

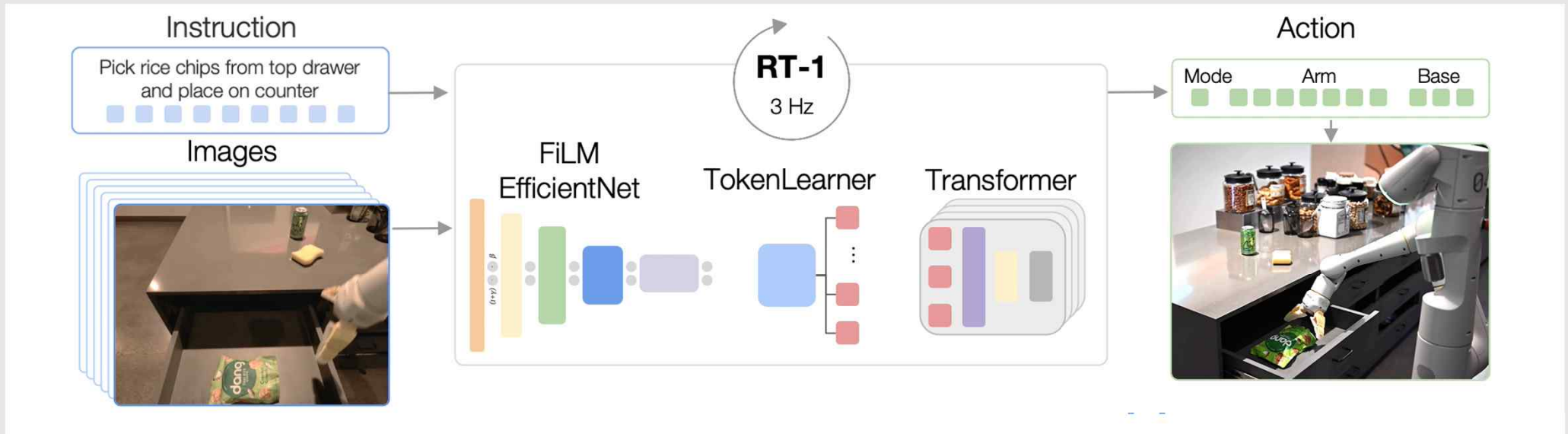
VLA 상제

RT-1,2/OpenVLA/Pi0/OpenVLA-OFT/Pi0.5/GR00TN1,N1.5

01

Robotics Transformer 1 (RT-1)

Robotics Transformer 1 (RT-1 2022)

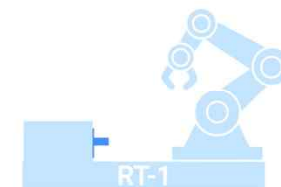
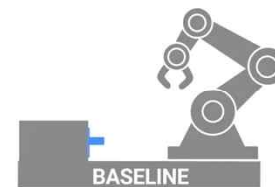


성과와 일반화 능력이 향상

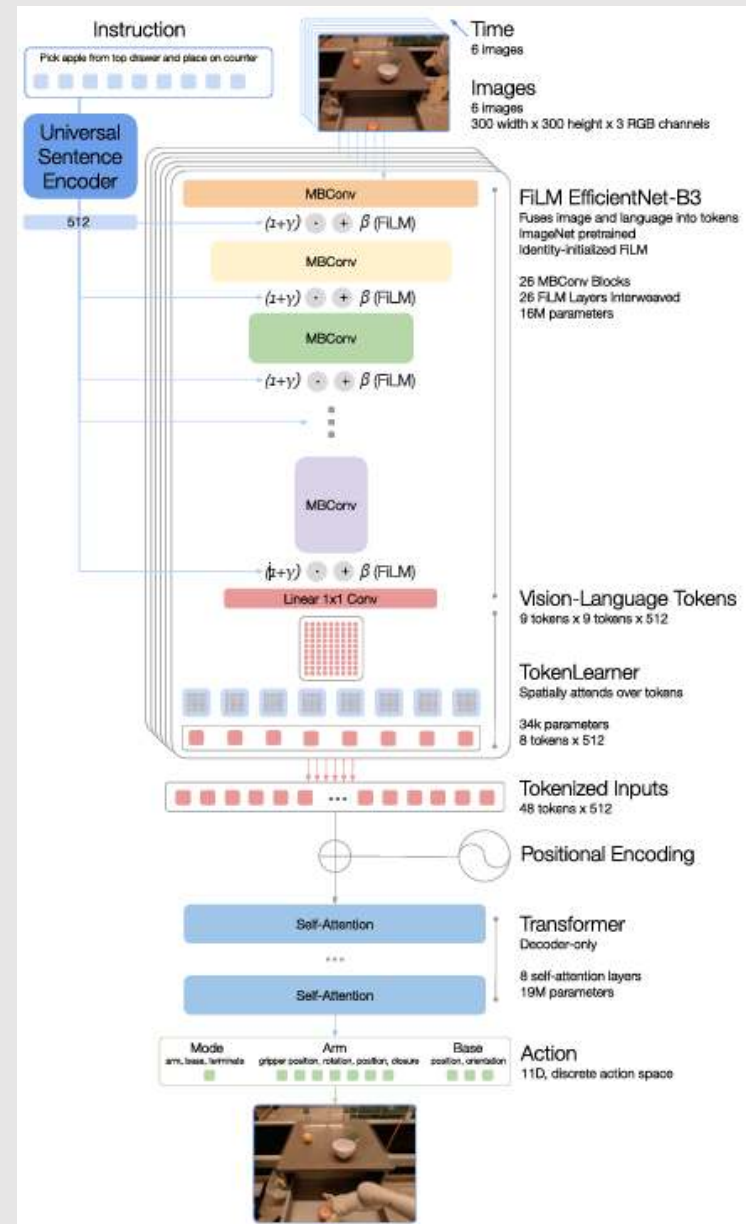
- 이유 : RT-1은 여러 작업, 물체, 환경을 포함하는 다양한 로봇 궤적 데이터를 대량 흡수

기존 베이스라인

- 큰 데이터셋을 완전히 활용하는 능력 제한적



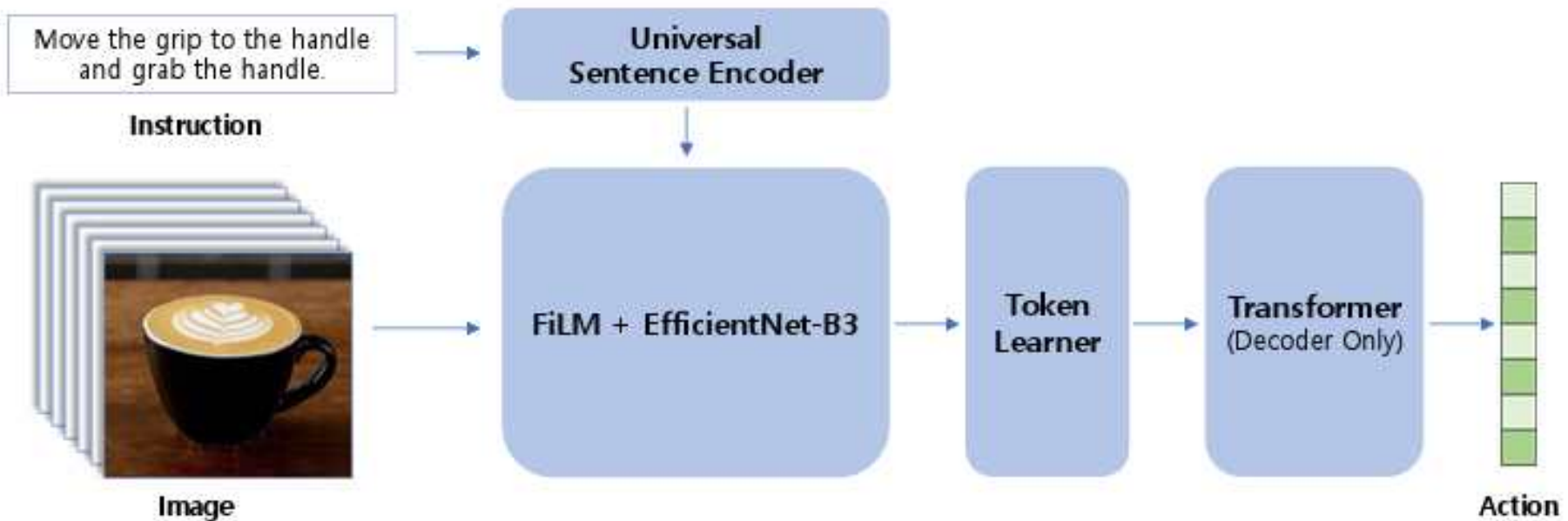
Robotics Transformer 1 (RT-1)



Robotics Transformer 1 (RT-1)

❖ RT-1: Robotics Transformer for Real-World Control at Scale (Google Research, 2022, 914회 인용)

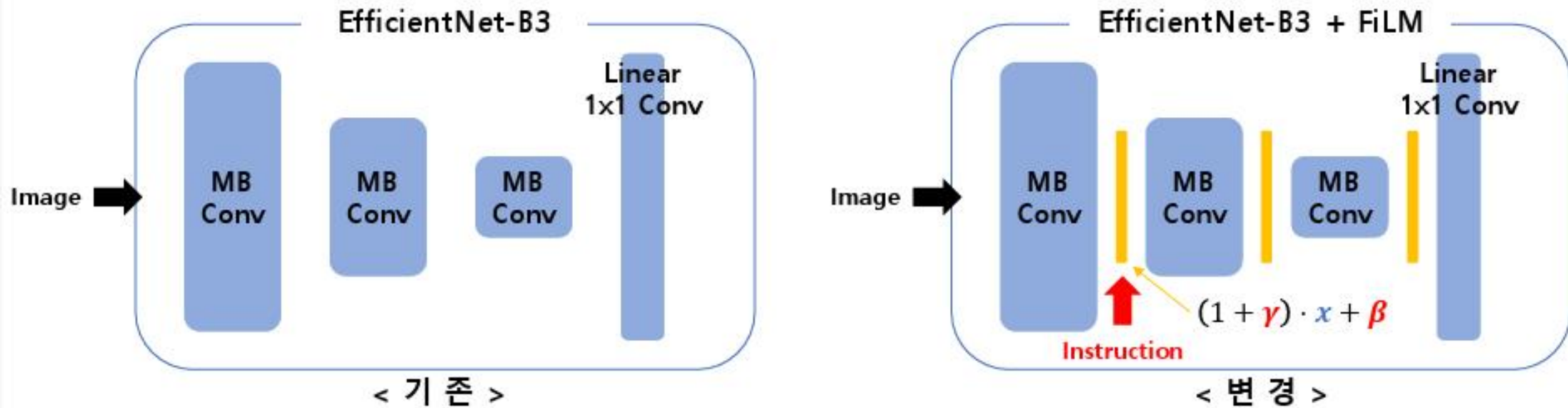
- 로봇 학습에서는 **일반화 성능과 추론 속도가 중요함** → RT-1은 **대규모 데이터셋과 모델 크기를 확장**하여 해당 목표를 달성하고자 함
- 보통 모델 파라미터의 크기가 증가할 수록 일반화 성능이 좋아지지만 동시에 추론 속도가 하락하는 경향이 있음
- RT-1은 이 문제를 완화할 수 있는 로봇 학습에 적합한 아키텍처를 제시함



Robotics Transformer 1 (RT-1)

❖ Detailed Architecture of RT-1

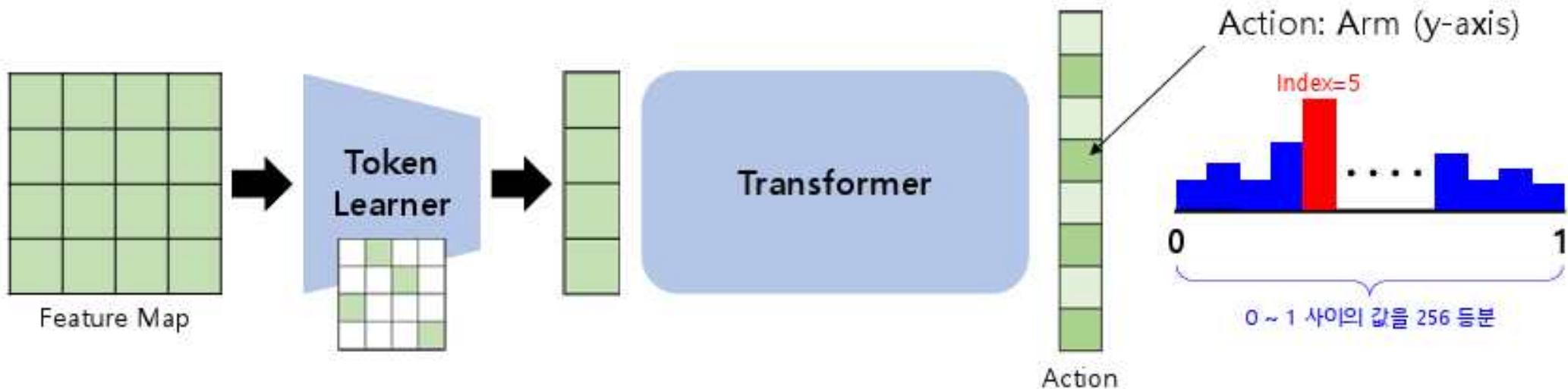
- Feature-wise linear modulation (FiLM)은 서로 다른 두 정보를 하나의 정보로 합치는데 효과적인 방법론
- EfficientNet-B3는 사전학습된 이미지 인코더이며 파라미터 수가 작지만 뛰어난 성능을 보여주고 추론 속도가 빠르다는 특징이 있음
- FiLM을 EfficientNet-B3의 각 블록 사이마다 삽입하여 multimodal learning을 수행
- 이 때 사전학습으로 얻은 성능을 훼손하지 않기 위하여 처음엔 FiLM의 가중치를 항등함수와 동일하도록 초기화한 뒤 점차 업데이트를 진행



Robotics Transformer 1 (RT-1)

❖ Detailed Architecture of RT-1

- 인코더에서 특징을 추출한 뒤 transformer에 입력하기 전, token learner를 사용하여 중요한 일부 토큰을 선별함
→ 연산량을 줄여 추론 속도 확보
- 이후 transformer를 사용하여 action token을 예측
- 원래 연속적인 값으로 행동이 구성되지만 이를 256개의 구간으로 나누어 이산적인 값으로 표현함
→ 해당テクニック이 더 복잡한 분포를 학습했다고 서술 (연속적인 값으로 학습할 경우 단일 모드의 정규 분포를 학습)



Robotics Transformer 1 (RT-1)

❖ Experiment Settings (dataset & robot)

- 17개월 동안 13대의 로봇(EverydayRobot)으로 현실 세계에서 수집한 13만 여개의 동작 데이터(총 744개의 과제)를 사용
- EverydayRobot에 학습한 정책을 탑재하여 성능 검증을 수행

Skill	Count	Description	Example Instruction
Pick Object	130	Lift the object off the surface	pick iced tea can
Move Object Near Object	337	Move the first object near the second	move pepsi can near rxbar blueberry
Place Object Upright	8	Place an elongated object upright	place water bottle upright
Knock Object Over	8	Knock an elongated object over	knock redbull can over
Open Drawer	3	Open any of the cabinet drawers	open the top drawer
Close Drawer	3	Close any of the cabinet drawers	close the middle drawer
Place Object into Receptacle	84	Place an object into a receptacle	place brown chip bag into white bowl
Pick Object from Receptacle and Place on the Counter	162	Pick an object up from a location and then place it on the counter	pick green jalapeno chip bag from paper bowl and place on counter
Section 6.3 and 6.4 tasks	9	Skills trained for realistic, long instructions	open the large glass jar of pistachios pull napkin out of dispenser grab scoop
모델에 입력되는 형식			
Total	744		실제 입력 예시

Dataset Description



Everyday Robot

Robotics Transformer 1 (RT-1)

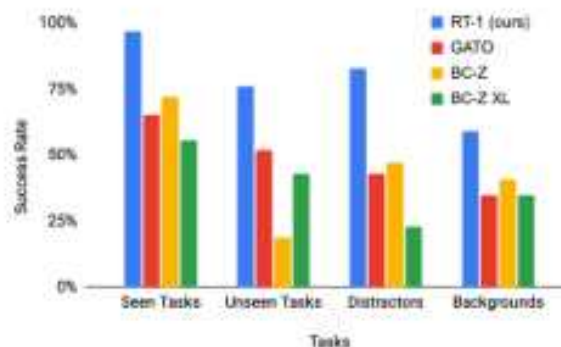
❖ Experiment Results

- RT-1의 학습 성능과 일반화 성능을 평가
 - ✓ Unseen Tasks는 학습에 사용된 적 없는 스킬과 물체를 가지고 지시하는 경우
 - ✓ Distractors는 지시사항에 없는 물체를 작업대에 진열하는 경우 (기본적으로 2~5개는 존재)
 - ✓ Backgrounds는 데이터에 없던 배경에서 작업을 수행하는 경우
- 비교 방법론에 비해서 더 좋은 일반화 성능을 가짐

목표 외 객체를 배치 본적 없는 배경으로 변경
 처음보는 과제 수행

Model	Seen Tasks	Unseen Tasks	Distractors	Backgrounds
Gato (Reed et al., 2022)	65	52	43	35
BC-Z (Jang et al., 2021)	72	19	47	41
BC-Z XL	56	43	23	35
RT-1 (ours)	97	76	83	59

일반화 성능 평가
(성공률)



Placing Distracting Objects



Changing the Texture of the Table

Table 2: Overall performance of RT-1 and baselines across seen tasks, generalization to unseen tasks, and robustness to distractors and backgrounds.

Robotics Transformer 1 (RT-1)

❖ Experiment Results

- Ablation study를 통해 연속적인 행동 값을 이산적으로 표현하여 학습하는 것이 보다 효과적임을 보임



Table 13: Various model ablations of RT-1 across seen tasks, generalization to unseen tasks, and robustness to distractors and backgrounds.

Robotics Transformer 1 (RT-1)

❖ Experiment Results

- 형태가 다른 종류의 로봇 데이터를 사용하여도 성능 향상이 나타남 (단, 본래의 데이터와 함께 학습 해야함)
- KUKA 로봇으로 쓰레기를 집는(pick object) 과제를 수행한 데이터를 학습한 결과 RT-1에서도 해당 과제에 대한 성능이 향상됨

Models	Training Data	Classroom eval	Bin-picking eval
RT-1	Kuka bin-picking data + EDR data	90(-2)	39(+17)
RT-1	EDR only data	92	22
RT-1	Kuka bin-picking only data	0	0

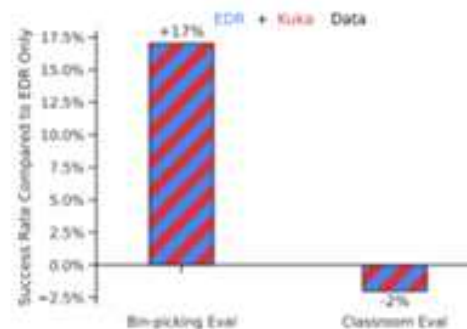


Table 5: Experimental results for mixing data from two different robots. Incorporating Kuka bin-picking data from QT-Opt (Kalashnikov et al., 2018) in RT-1 minimally impacts the standard classroom evaluation performance and results in almost a 2x improvement in generalization to the Bin-picking evaluation (that is similar to the setup in the Kuka data) on the Everyday Robots manipulator. This demonstrates an effective transfer across two different robot morphologies.



Kuka Robot

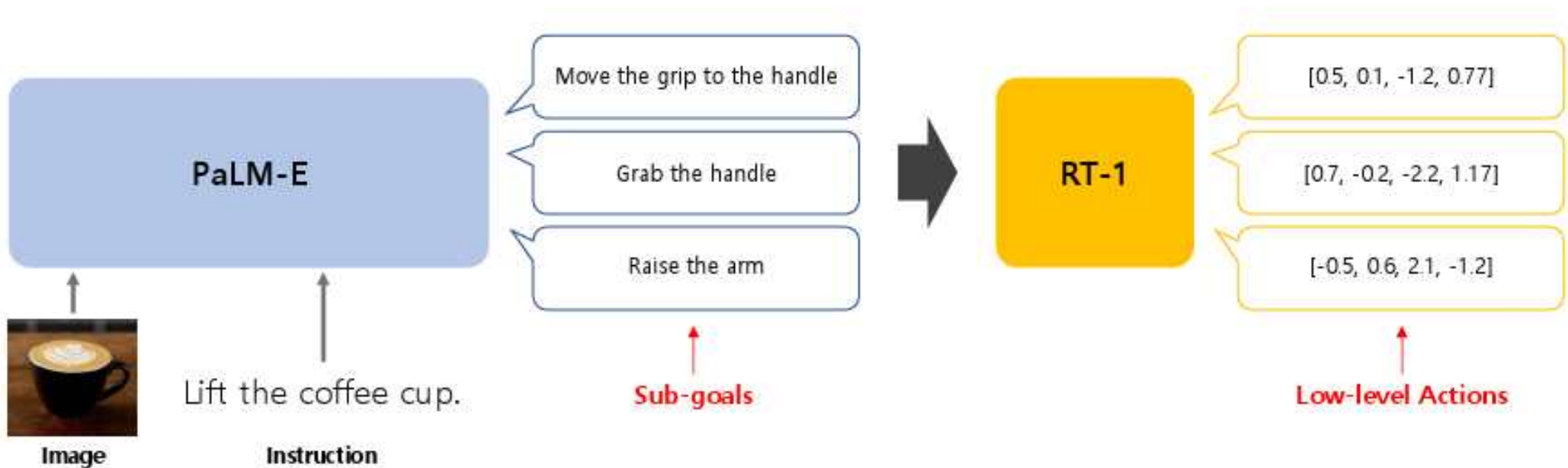
02

PaLM-E

Pathways Language Model-Embodied (PaLM-E)

❖ PaLM-E: An Embodied Multimodal Language Model (ICML, 2023, 1630회 인용)

- 사전학습된 거대언어모델 PaLM에 이미지-텍스트, 제어 데이터를 기반으로 multi-modal 학습
- 현실 세계를 효과적으로 표현할 수 있도록 vision-language-model을 활용하여 로봇 제어 수행
- 다만 PaLM-E는 지시사항을 다수의 세부 절차로 나누어 주는 역할을 하며 실질적인 움직임은 RT-1이 수행



Pathways Language Model–Embodied (PaLM-E)

❖ Co-training Strategy

- 사전학습된 PaLM (8B, 62B, 540B)에 이미지, 텍스트, 로봇 조종이 유기적으로 연관된 데이터를 학습하여 multi-modal large language model 생성

Language Only Task

Visual Q&A



Given `<img>`. Q: What's in the image? Answer in emojis.

A: 🍏 🍌 🍇 🍎 🍓 🍓

Captioning



Describe the following `<img>`:

A dog jumping over a hurdle at a dog show

Mobile Manipulation



Human: Bring me the rice chips from the drawer. Robot: 1. Go to the drawers, 2. Open top drawer. I see `<img>`.

3. Pick the green rice chip bag from the drawer and place it on the counter

Given `<emb>` ... `<img>` Q: How to grasp blue block? A: First, grasp yellow block



Large Language Model (PaLM)

After Training → PaLM-E

A: First, grasp yellow block

Pathways Language Model–Embodied (PaLM-E)

❖ Experiment Settings (dataset & robot)

- 제시된 데이터 조합으로 학습된 PaLM-E (8B, 62B, 540B)를 high level controller로 사용, low level control은 RT-1을 사용
- 클라우드 시스템에서 지시사항을 추론 한 뒤 EverydayRobot에 전송하는 방식으로 성능 검증을 수행

Dataset in full mixture	Sampling frequency	%
Webli (Chen et al., 2022)	100	52.4
VQ ² A (Changpinyo et al., 2022)	25	13.1
VQG (Changpinyo et al., 2022)	10	5.2
CC3M (Sharma et al., 2018)	25	13.1
Object Aware (Piergiovanni et al., 2022)	10	5.2
OKVQA (Marino et al., 2019)	1	0.5
VQAv2 (Goyal et al., 2017)	1	0.5
COCO (Chen et al., 2015)	1	0.5
Wikipedia text	1	0.5
(robot) Mobile Manipulator, real 주방에서의 과제를 수행	6	3.1
(robot) Language Table (Lynch et al., 2022), sim and real 물체를 밀어 이동하는 법	8	4.2
(robot) TAMP, sim 물체를 쌓는 법	3	1.6

Dataset Description

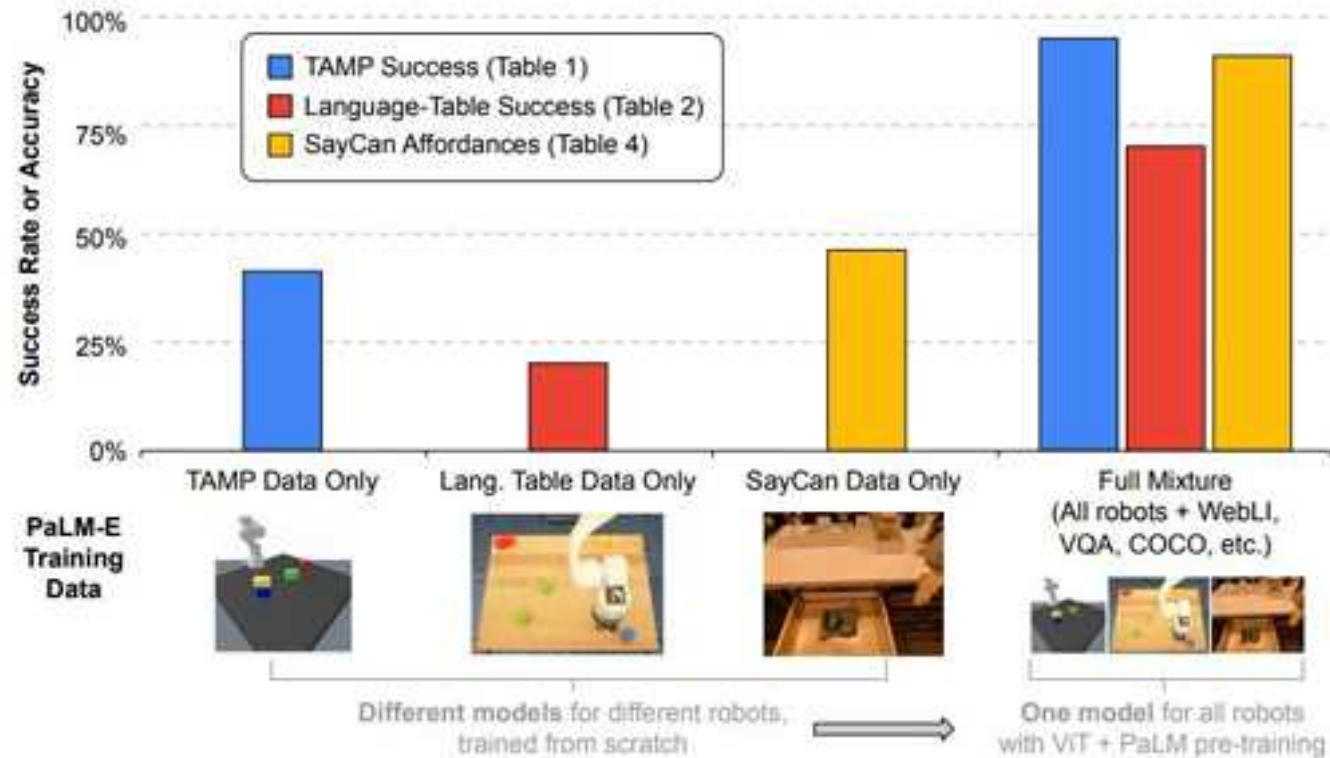


Everyday Robot

Pathways Language Model-Embodied (PaLM-E)

❖ Experiment Results

- 로봇 조종 과제에 있어서 이미지와 텍스트 데이터를 함께 학습하는 것이 성능 향상에 더 도움이 됨



Pathways Language Model – Embodied (PaLM-E)

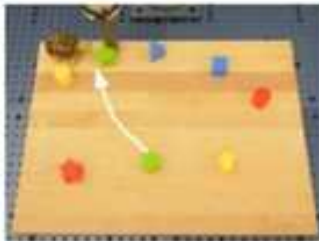
❖ Experiment Results

- Language Table 환경에서 PaLM-E의 일반화 성능을 검증: Robot 데이터에 없던 물체가 있을 때 + 환경에 갑작스런 인위적인 변화를 줄 때
- Robot manipulation 학습 데이터에는 '거북이'가 전혀 없었으나 잘 인지해서 주어진 지시사항을 성공적으로 수행

Move the green star to the top left corner



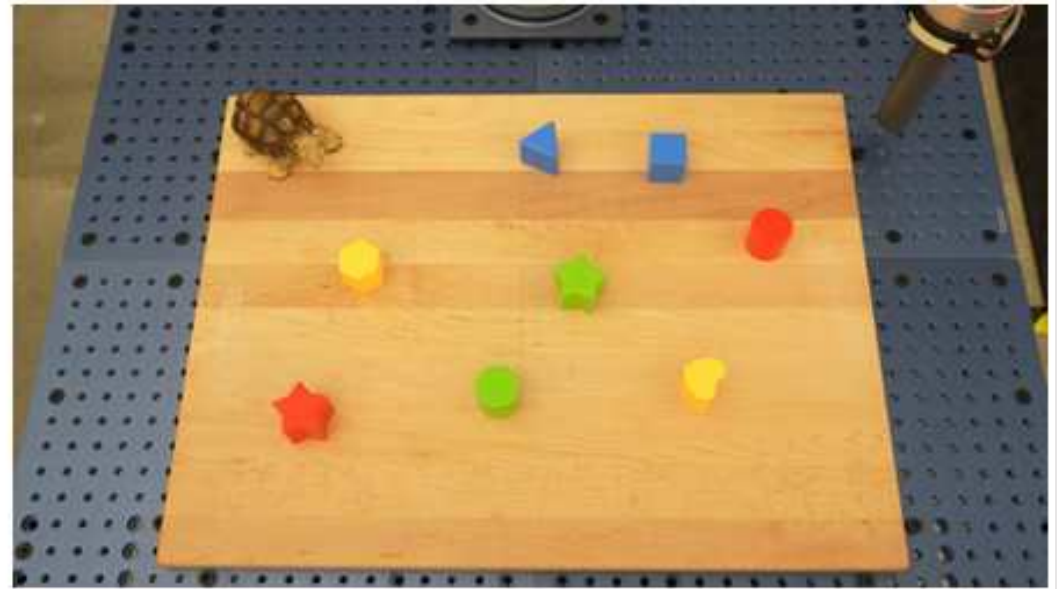
Move the green star to the green circle



SUCCESS



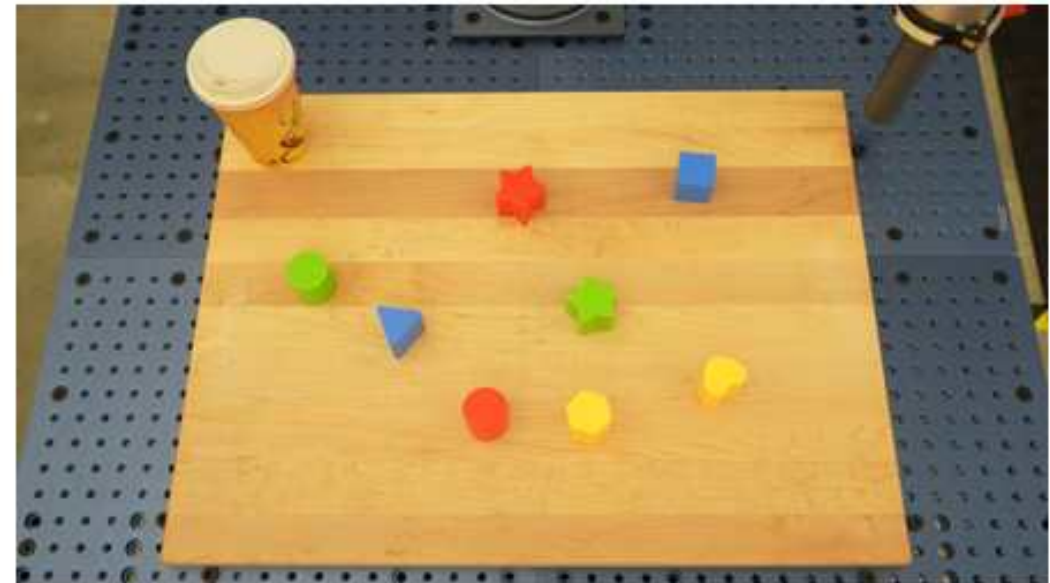
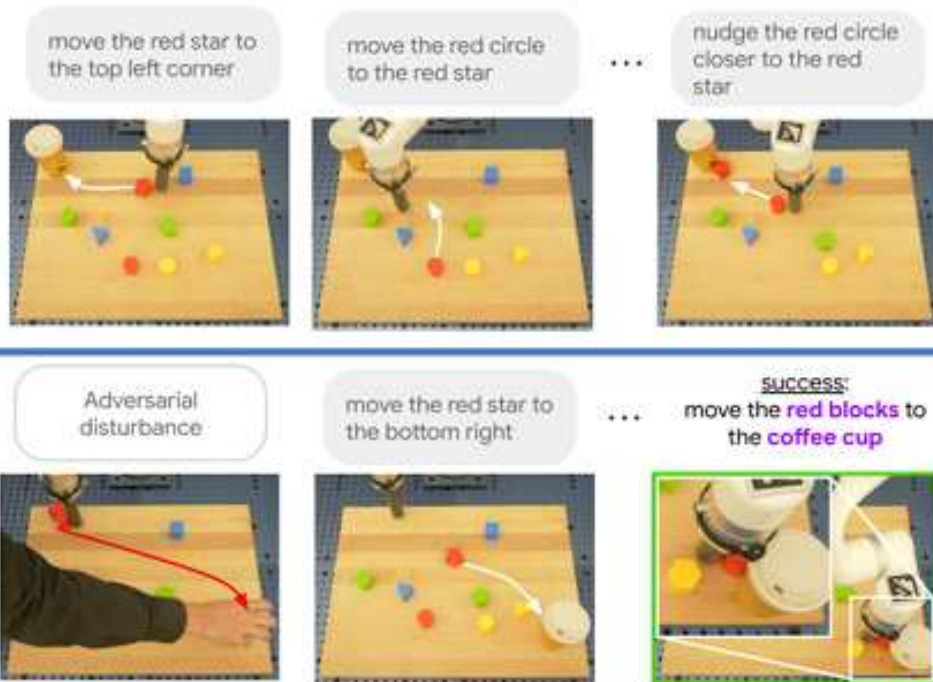
zero-shot: "Move the green blocks to *the turtle*"



Pathways Language Model – Embodied (PaLM-E)

❖ Experiment Results

- Language Table 환경에서 PaLM-E의 일반화 성능을 검증: Robot 데이터에 없던 물체 조합이 있을 때 + 환경에 갑작스런 인위적인 변화를 줄 때
- 학습 시 red block과 coffee cup이 하나의 과제 안에 있는 경우가 없었으나 성공적으로 수행

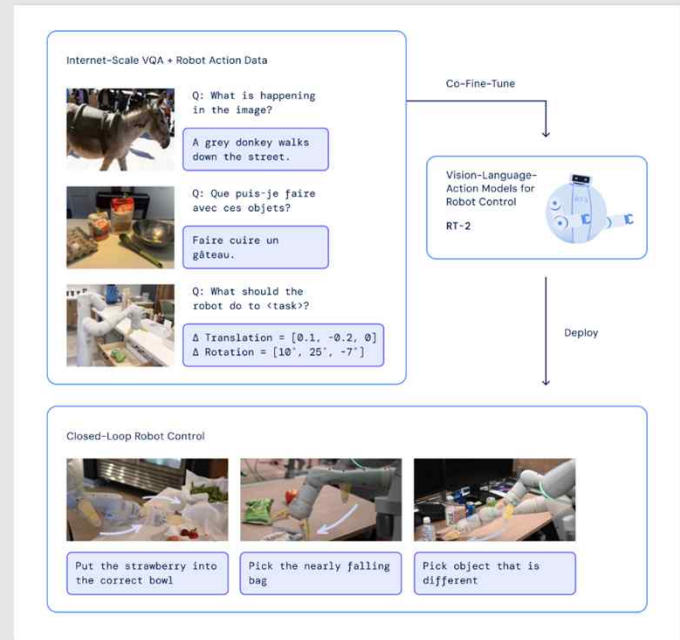


03

Robotics Transformer 2 (RT-2)

Robotics Transformer 2 (RT-2)

- **모델 유형:** 비공개 Vision-Language-Action 모델
- **백본:** PaLI-X(55B) or PaLM-E(12B)
- **입력:** 로봇 카메라 이미지 + 자연어 질의
- **출력:** 이산형 액션 토큰 → 디토큰나이즈 → 로봇 제어 코드
- **학습 데이터:** 웹 규모 시각·언어 데이터 + 로봇 행동 데이터 비중 강화
- **특징:** COT(Chain-of-Thought) 추론, 과제 일반화 능력, 작업별 정책 불필요
- **성과:** 훈련받지 않은 작업(사람 인식, 기호 이해 등) 수행 가능
- **의의:** 웹 스케일 사전학습 + 행동 토큰화를 통한 최초의 고도 일반화 VLA 사례



Prompt:
Given Instruction:
Bring me a drink.
Prediction:
Plan: pick 7up can.
Action: 1 143 129 123 145
114 115 127



Prompt:
Given Instruction:
Move all the objects
together.
Prediction:
Plan: move green can near
green rice chip bag.
Action: 1 128 126 127 135
123 119 127



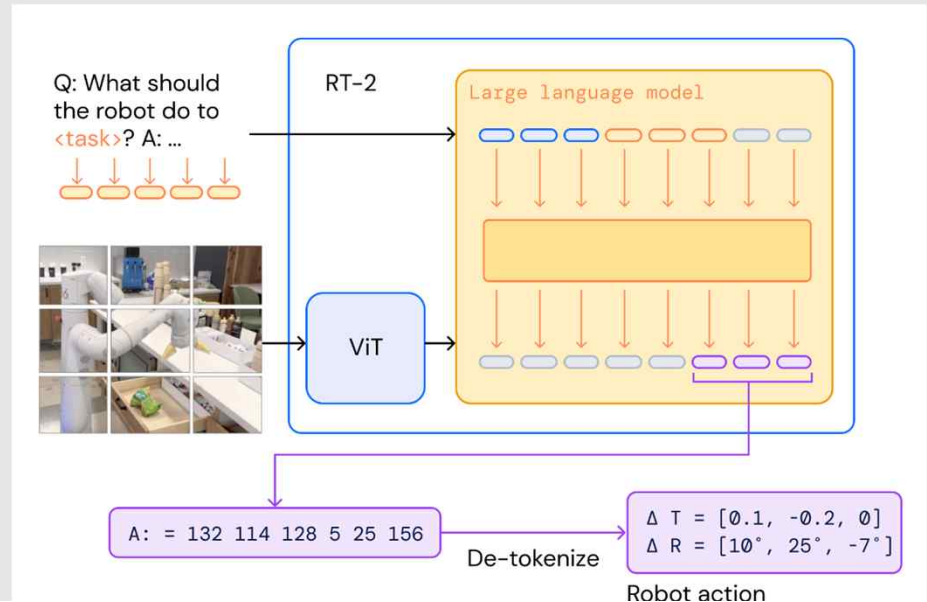
Prompt:
Given Instruction:
Pick the object that is
different from all other
objects
Prediction:
Plan: pick rxbar
chocolate. Action: 1 128
129 125 131 125 128 127



Prompt:
Given Instruction:
Move the green objects
together.
Prediction:
Plan: move green can near
green rice chip bag.
Action: 1 130 129 121 131
127 128 127



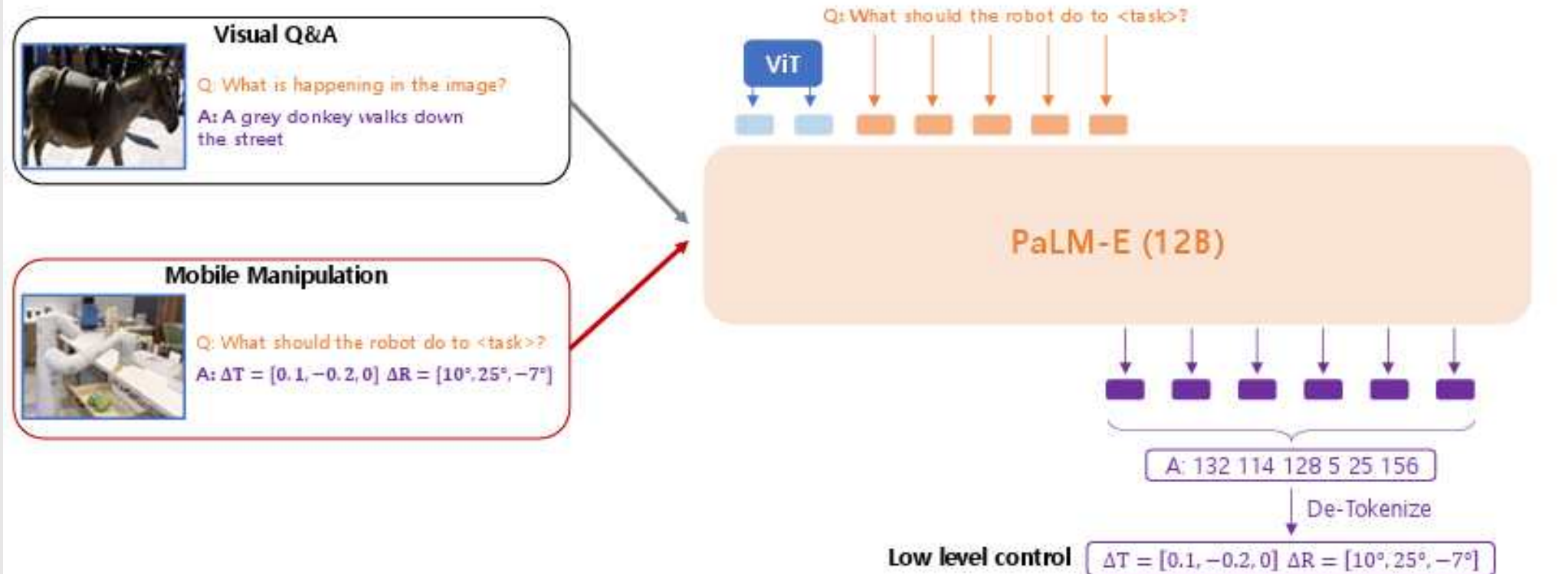
Prompt:
Given I need to
hammer a nail, what
object from the scene
might be useful?
Prediction:
Rocks. Action: 1 129 138
122 132 135 106 127



Robotics Transformer 2 (RT-2)

❖ RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control (Google DeepMind, 2023, 781회 인용)

- PaLM-E (또는 PaLM-X)가 sub-goals를 출력하는 것이 아닌 RT-1처럼 로봇 제어에 필요한 값을 직접 출력하도록 학습하는 방법론
- PaLM-E에서 진행한 co-training strategy를 수행하되 이번 manipulation 데이터는 정답에 low level action을 사용함



Robotics Transformer 2 (RT-2)

❖ Action tokens for RT-2

- RT-1과 동일하게 연속적인 값을 이산적인 값으로 표현 (action tokens)
- 실험에서 단일 로봇 기종(Everyday Robot)을 쓰기 때문에 action tokens 길이가 고정되어 있음
- 다만, RT-1과 달리 사용하려는 vision-language model의 토큰을 사용해야 하므로 수정이 필요
 - PaLM-E를 쓰는 경우 토큰 중 잘 쓰이지 않는 토큰을 action token으로 변경하여 사용
 - PaLI-X를 쓰는 경우 자체적으로 숫자를 표현하는 토큰이 존재하여 변경 X



Robotics Transformer 2 (RT-2)

❖ Experiment Settings (dataset & robot)

- 17개월 동안 13대의 로봇(EverydayRobot)으로 현실 세계에서 수집한 13만 여개의 동작 데이터(총 744개의 과제)를 사용
- EverydayRobot에 학습한 정책을 탑재하여 성능 검증을 수행

Skill	Count	Description	Example Instruction
Pick Object	130	Lift the object off the surface	pick iced tea can
Move Object Near Object	337	Move the first object near the second	move pepsi can near rxbar blueberry
Place Object Upright	8	Place an elongated object upright	place water bottle upright
Knock Object Over	8	Knock an elongated object over	knock redbull can over
Open Drawer	3	Open any of the cabinet drawers	open the top drawer
Close Drawer	3	Close any of the cabinet drawers	close the middle drawer
Place Object into Receptacle	84	Place an object into a receptacle	place brown chip bag into white bowl
Pick Object from Receptacle and Place on the Counter	162	Pick an object up from a location and then place it on the counter	pick green jalapeno chip bag from paper bowl and place on counter
Section 6.3 and 6.4 tasks	9	Skills trained for realistic, long instructions	open the large glass jar of pistachios pull napkin out of dispenser grab scooper
Total	744		

모델에 입력되는 형식

실제 입력 예시

Dataset Description



Everyday Robot

Robotics Transformer 2 (RT-2)

❖ Experiment Results

- 학습한 상황에서는 성능이 조금 높거나 비슷한 경향을 보임
- 하지만 학습 데이터에 없던 객체, 배경, 환경이 주어진 상황에서는 RT-2가 더 좋은 일반화 성능을 보임

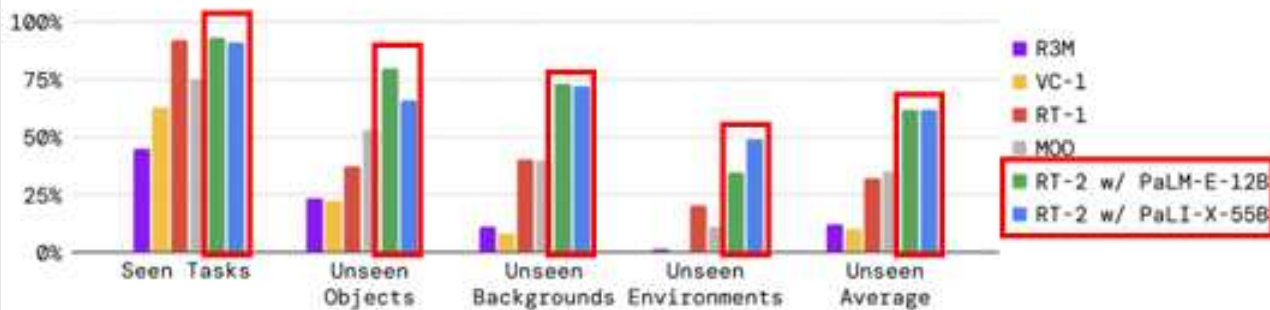
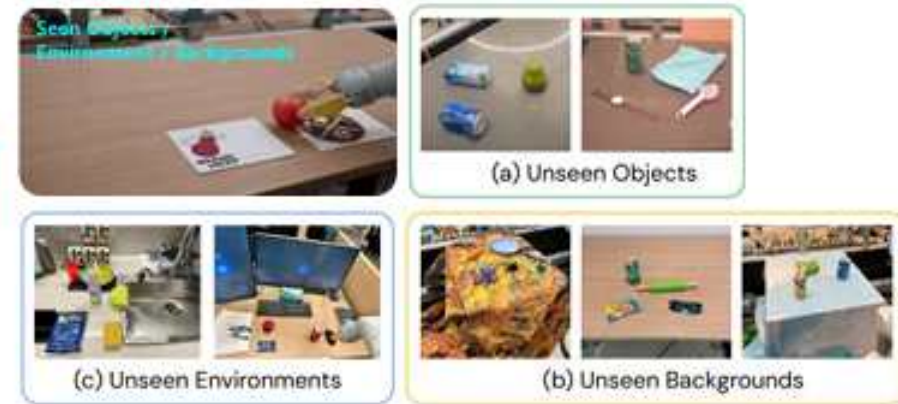


Figure 4 | Overall performance of two instantiations of RT-2 and baselines across seen training tasks as well as unseen evaluations measuring generalization to novel objects, novel backgrounds, and novel environments. Appendix Table 4 details the full results.



Robotics Transformer 2 (RT-2)

❖ Experiment Results

- 학습 과정에서 데이터에는 없었던 새로운 스킬을 찾아내는지 실험을 수행
- 학습 데이터에는 미는(push) 지시사항 존재하지 않으니 해당 지시를 내렸을 때 올바르게 수행하는 것을 확인
- 따라서 웹 규모의 사전 훈련으로 얻은 지식이 제어 관련 스킬과 연결되어 새로운 스킬을 획득할 수 있음



Figure 5 | Real-world out-of-distribution behaviors in the Language Table environment. Identical RT-2-PaLI-3B model checkpoint is used as in Tab. 1.

Model	Language-Table
BC-Zero (Jang et al., 2021)	72 ± 3
RT-1 (Brohan et al., 2022)	74 ± 13
LAVA (Lynch et al., 2022)	77 ± 4
RT-2-PaLI-3B (ours)	90 ± 10

Table 1 | Performance on the simulated Language-Table tasks (Lynch and Sermanet, 2020).

OpenVLA

OpenVLA

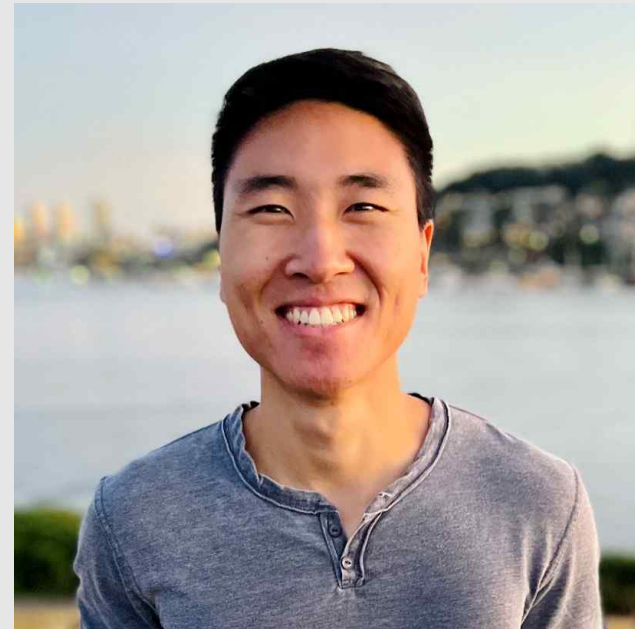
모두의연구소 PPT 표지 제목을 입력해주세요.

01

OpenVLA 개요

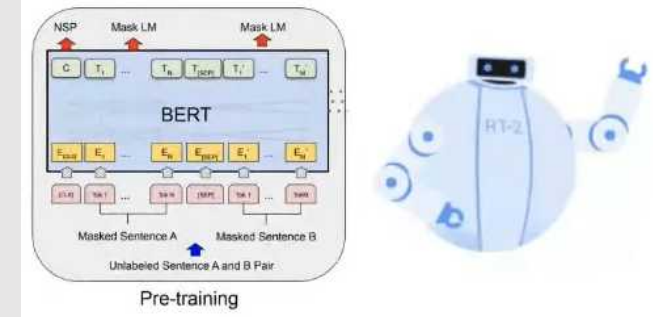
<https://moojink.com/>

<https://www.youtube.com/watch?v=-0s0v3q7mBk>



동기 (Motivation)

- NLP와 컴퓨터 비전 분야에서 성공적인 공식:
 1. 방대한 데이터로 대규모 "기반 모델(foundation model)"을 사전 학습
 2. 다운스트림 작업에 맞춰 미세 조정(fine-tune) 또는 제로샷(zero-shot) 배포
- 그러나, 이 접근법은 로봇틱스 분야에서는 역사적으로 그다지 인기가 없었음...
 - 데이터가 병목: 실제 세계에서 대규모로 수집하기 어려움
- 하지만 최근 동향은 로봇틱스에서도 유사한 공식을 사용할 가능성을 보여줌:
 - **Open X-Embodiment Dataset** : 200만 개 이상의 에피소드, 20개 이상의 로봇, 20개 이상의 기관에서 집계
 - **RT-2 / RT-2-X** : 사전 학습된 비전-언어 모델(VLMs)을 로봇 제어에 맞게 미세 조정
 - "VLA (Vision-Language-Action)"라는 용어를 처음 사용
 - 인터넷 규모의 사전 학습으로부터 넓은 의미론적 지식을 보유
 - 로봇 정책의 일반화 및 추론 능력을 강화 → 로봇 상호작용 데이터 그 이상으로 일반화 가능
 - 수십억 개(550억)의 파라미터를 가진 고용량 모델
 - 방대한 다양한 데이터셋에 적합
- RT-2 이후 더 많은 VLA 모델들이 개발됨
 - 예: **RFM-1, Interactive Agent Foundation Model, RoboFlamingo, LEO, 3D-VLA** 등
 - 그러나 이들 모델은 모두 비공개이거나 시뮬레이션된 로봇 데이터만으로 학습됨!



OpenVLA 프로젝트 목표

우리가 달성하고자 하는 것은:

1. 강력한 오픈소스 VLA 개발

- 실제 로봇 데이터를 대규모로 학습한 모델
- 범용 정책(**Generalist policy**) : 여러 종류의 로봇에서 다양한 작업을 바로 수행 가능

2. 다운스트림 작업에 적응할 수 있는 효과적인 프레임워크 개발

- 매개변수 효율적 미세 조정(**Parameter-efficient fine-tuning**) : 낮은 연산 자원으로도 미세 조정 가능

3. VLA의 사전학습 및 미세조정 코드, 모델 가중치, 데이터 혼합물 전부 공개

- 다른 연구자들이 대규모 사전학습 로봇 모델 연구를 발전시킬 수 있도록 지원



place the red vegetable
in the silver pot



route cable



pick up the blue cup and
put it into the brown cup



place the black bowl
in the oven



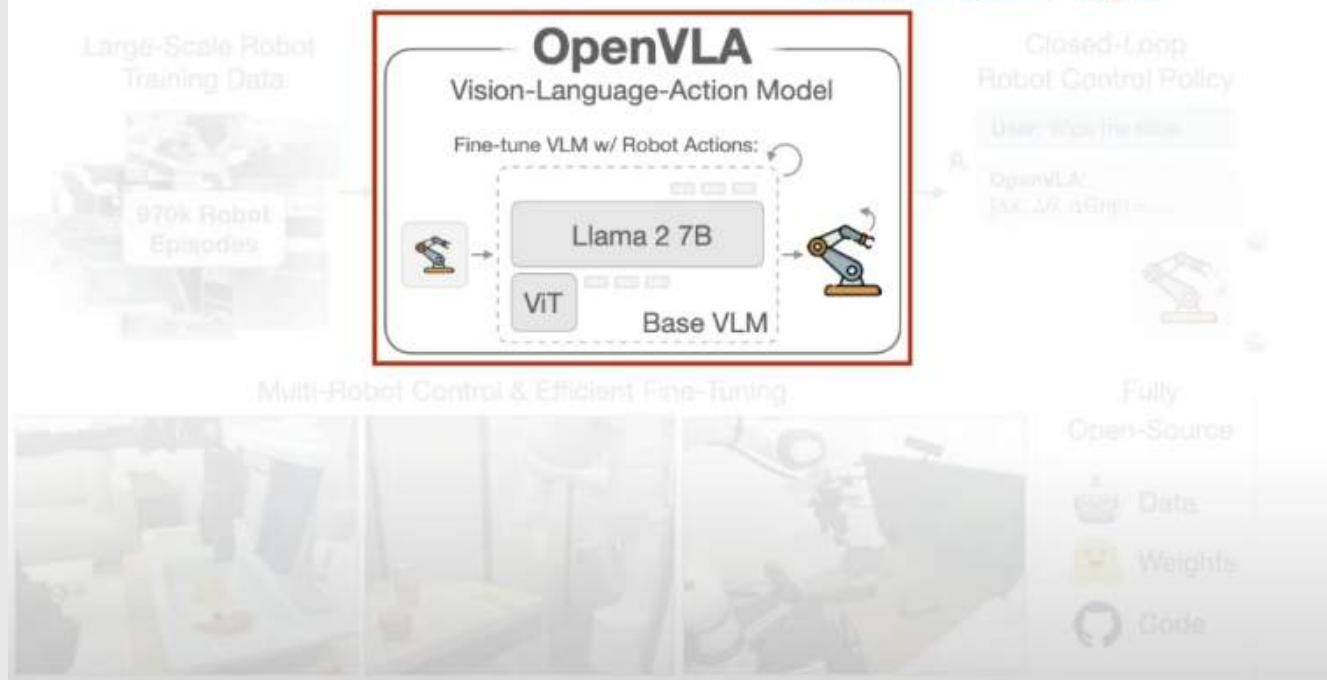
place green can upright

(Videos from the Open X-Embodiment dataset)

OpenVLA 프로젝트 목표

What is OpenVLA?

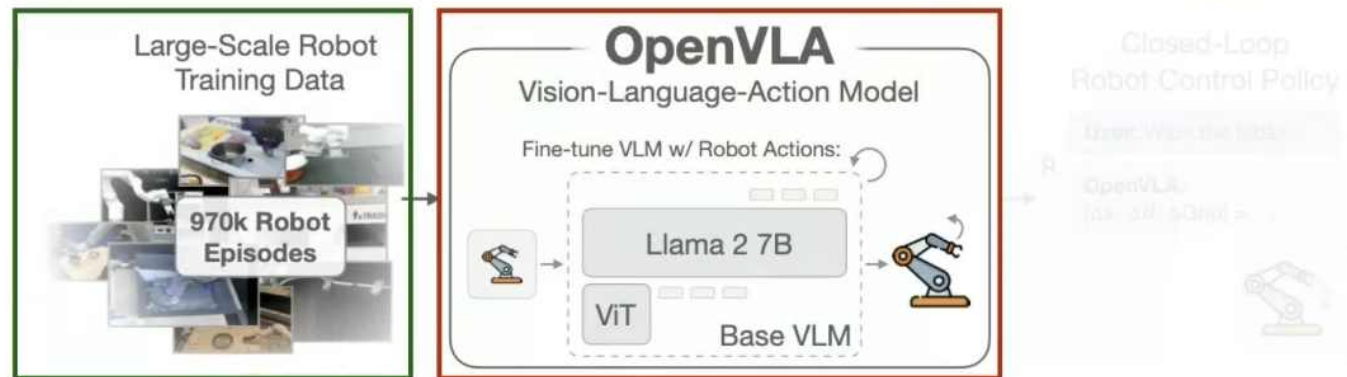
- 7B VLA built on pretrained “Prismatic VLM” backbone
- LLM: Llama 2
- Vision: DINOv2 + SigLIP



OpenVLA 프로젝트 목표

What is OpenVLA?

- 7B VLA built on pretrained “Prismatic VLM” backbone
- LLM: Llama 2
- Vision: DINOv2 + SigLIP



What is OpenVLA trained on?

- 970k episodes from Open X-Embodiment dataset
 - 27 real robot datasets
- Trained on 15 more datasets than RT-2-X

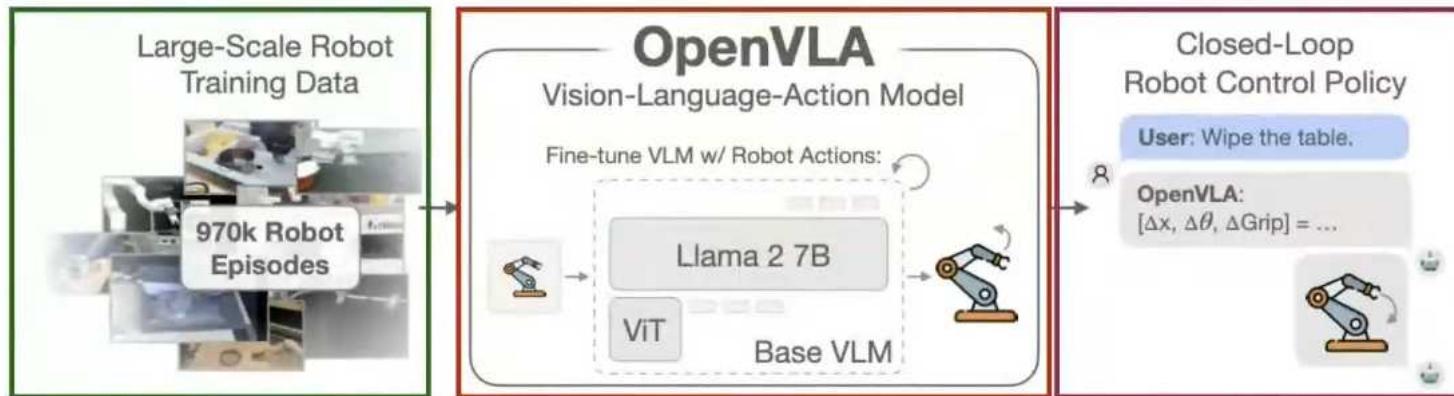
Fully
Open-Source

- Data
- Weights
- Code

OpenVLA 프로젝트 목표

What is OpenVLA?

- 7B VLA built on pretrained “Prismatic VLM” backbone
- LLM: Llama 2
- Vision: DINOv2 + SigLIP



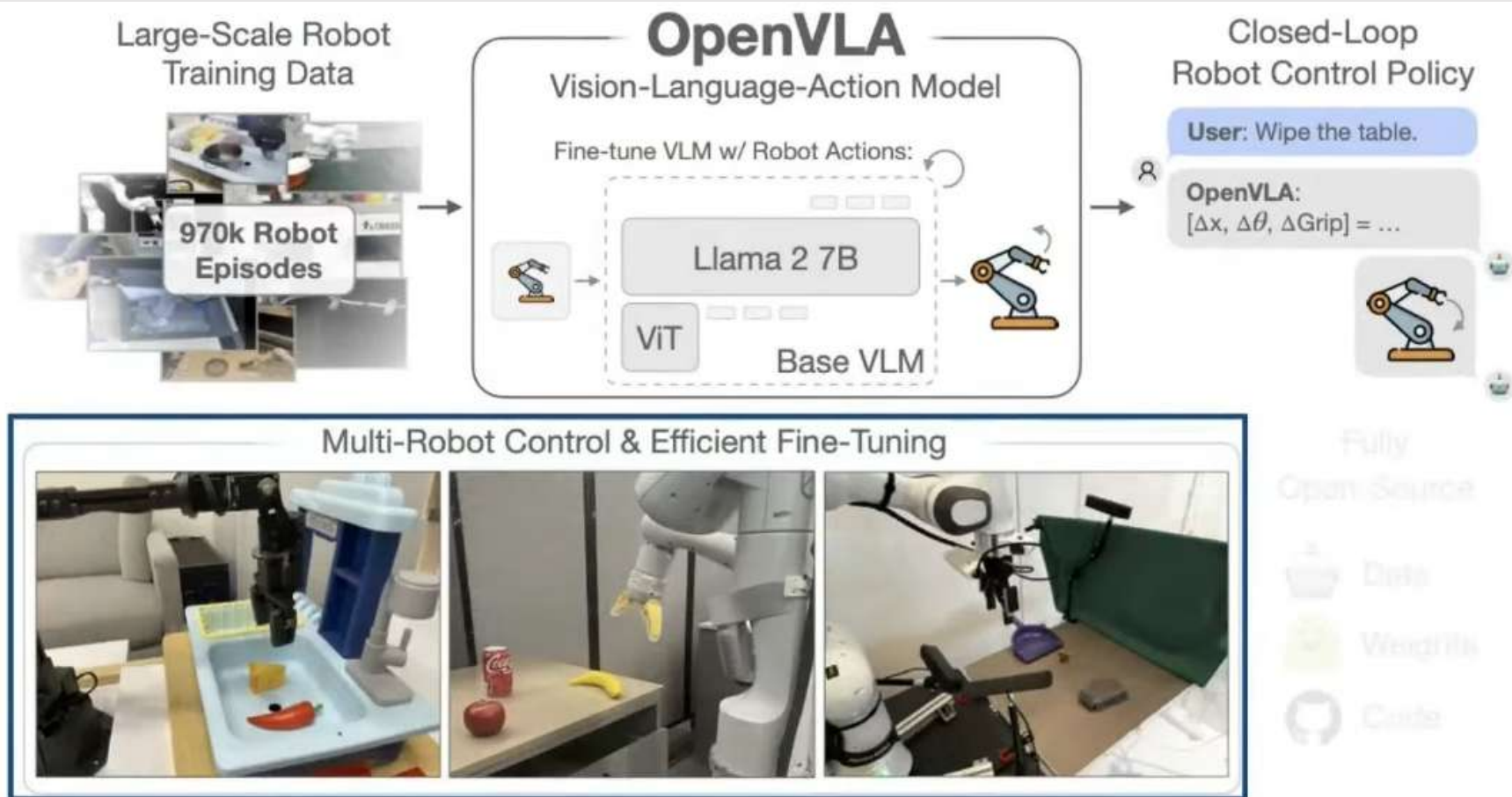
What is OpenVLA trained on?

- 970k episodes from Open X-Embodiment dataset
 - 27 real robot datasets
- Trained on 15 more datasets than RT-2-X

OpenVLA Inputs / Outputs?

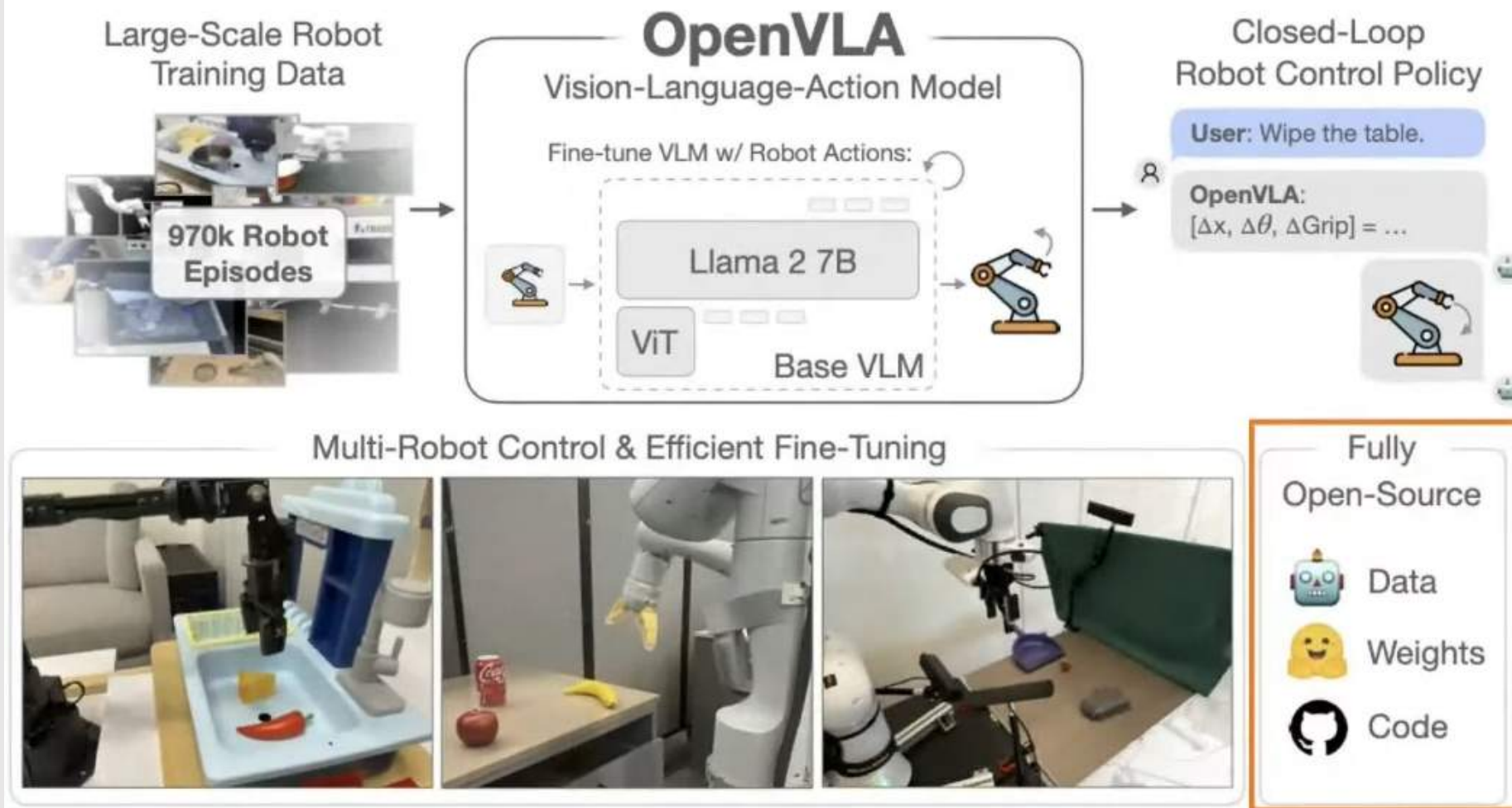
- Inputs: Prompt w/ single image & language instruction
 - $\langle \text{Image} \rangle$ In: What action should the robot take to put eggplant into pot?
- Out:
- Outputs: Tokenized robot action

OpenVLA 프로젝트 목표



Which robots can OpenVLA control? •WidowX, Google robot, Franka, & several more!

OpenVLA 프로젝트 목표



All open-sourced! :)

OpenVLA는 어떻게 학습되는가?

- **모방 학습(Imitation learning):** 전문가 시연을 바탕으로 한 행동 복제 (behavioral cloning)
 - **주어진 것(Given):** 관측값 o_t 와 행동 a_t 의 쌍으로 이루어진 궤적(trajecory), $t = 1, \dots, T$
 - **목표(Goal):** 모델 π_θ 를 학습하여, $\pi_\theta(\cdot|o_t) = \hat{a}_t \approx a_t$ 가 되도록 예측
- 각 행동 $\hat{a}_t$ 는 **이산화(discretized)** 되어, 7개의 토큰 문자열(string of 7 tokens)**로 표현됨
 - 이를 통해 OpenVLA는 마치 LLM을 학습하듯이, **다음 토큰 예측(next-token prediction)** 방식과 크로스 엔트로피 손실(cross-entropy loss)**로 훈련할 수 있음
 - 특별한 아키텍처 수정은 필요 없음!

OpenVLA는 어떻게 학습되는가?(2)

Prompt and response



← 3rd-person image

Prompt template

Language instruction

In: What action should the robot take to put eggplant into pot?

Out: 0.00 0.03 -0.82 0.00 -0.14 0.57 1.00

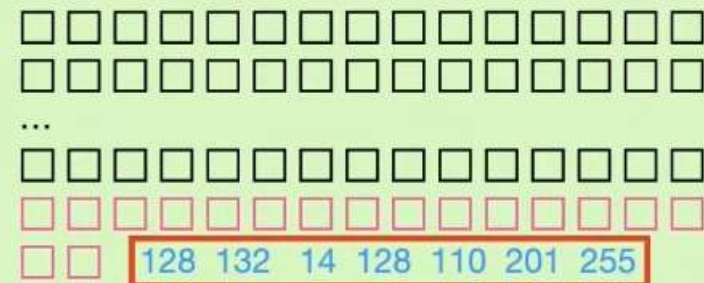
Robot action:

$$a_t = [\Delta pos_x \ \Delta pos_y \ \Delta pos_z \ \Delta rot_x \ \Delta rot_y \ \Delta rot_z \ \Delta gripper]$$

Note: This is for a single timestep t !

Tokenize

Prompt and response



Trained to predict these 7 robot action tokens only
via autoregressive next-token prediction

How is the robot action tokenized into 7 tokens?

OpenVLA 로봇 행동 토큰화 (Robot Action Tokenization)

- **로봇 행동 공간: 7차원**
 - 6자유도(6-DoF) 엔드이펙터 위치 변화:
 $\Delta pos_x, \Delta pos_y, \Delta pos_z, \Delta rot_x, \Delta rot_y, \Delta rot_z$
 - 1자유도(1-DoF) 그리퍼 제어: $\Delta gripper$ (이진값: 0 = 닫힘, 1 = 열림)
- 각 차원은 $[-1, +1]$ 범위로 스케일링된 후, **255개의 균일한 구간(bin)** 으로 이산화(discretization)
 - 예:
 - $\Delta pos_x \rightarrow -1 [1|2|\dots|254|255] + 1$
 - $\Delta pos_y \rightarrow -1 [1|2|\dots|254|255] + 1$
 - ...
- 따라서, 각 행동 $\hat{a}_t$ 는 **7개의 토큰 문자열(string of 7 tokens)** 로 표현 가능
 - 예시:
 - 원본 행동(raw action): $[0.00, 0.03, -0.82, 0.00, -0.14, 0.57, 1.00]$
 - 토큰화(tokenized): "128 132 14 128 110 201 255"
- 전체 행동 공간을 표현하는 데 **255개의 토큰만 필요**
- 실제 적용에서는, 자주 사용되지 않는 255개의 토큰을 덮어쓰워(vocab override) 활용
- 참고: RT-2 (PaLM-E 변형 모델)도 유사한 토큰화 방식을 사용

실험 (Experiments)

- Main experiments:
 - **(1) How does OpenVLA compare to prior generalist robot policies when deployed out of the box?**
 - **(2) Can OpenVLA be effectively fine-tuned on a new robot setup/task?**
- Additional experiments:
 - (3) Can we use parameter-efficient fine-tuning methods (e.g., LoRA) to fine-tune OpenVLA efficiently with a smaller compute budget?
 - (4) Can we use inference-time quantization to load & run OpenVLA with limited GPU memory?

실험 #1 – Out-of-the-Box 평가

- **(1) How does OpenVLA compare to prior generalist robot policies when deployed out of the box?**
- Comparisons
 - **RT-1-X (35M params)**
 - Transformer model with FiLM-conditioned EfficientNet image encoder
 - Trained on 12 datasets from OpenX (mostly from scratch)
 - **Octo (93M params) – Prior SOTA open-source generalist policy**
 - Transformer model with flexible input/output spaces & diffusion action head
 - Trained on 25 datasets from OpenX (mostly from scratch)
 - **RT-2-X (55B params) – Prior SOTA closed-source generalist policy**
 - Pretrained PaLI-X VLM with 22B ViT image encoder & 32B encoder-decoder LLM
 - Fine-tuned on 12 datasets from OpenX (same as RT-1-X)
 - **OpenVLA (7B params)**
 - Pretrained Prismatic VLM with fused ~300M DINOv2 + ~400M SigLIP image encoders & 7.1B Llama 2 decoder-only LLM
 - Fine-tuned on 27 datasets from OpenX

실험 #1 – Out-of-the-Box 평가

- We evaluate on two setups: BridgeData V2 WidowX robot + Google RT-X robot
- Evaluation setup #1: BridgeData V2 WidowX
 - 17 tasks total, 10 rollouts per task
 - Various axes of OOD generalization: visual, motion, physical, semantic
 - Language grounding tasks
 - Given multiple objects in scene, can policy manipulate correct target object (specified in user's prompt)?

실험 #1 - Out-of-the-Box 평가

Sample visual generalization tasks

Put Cup from Counter into Sink



(unseen cup color)

Put Eggplant into Pot



(unseen pot)

Put Yellow Corn on Pink Plate



(unseen clutter / distractor objects)

실험 #1 - Out-of-the-Box 평가

동작 일반화

Sample motion generalization tasks

Lift Eggplant



(unseen object position)

Put Carrot on Plate



(unseen plate height)

실험 #1 - Out-of-the-Box 평가

Sample physical generalization tasks

Flip Pot Upright



(unseen pot size/shape)

Lift AAA Battery



(unseen target object size)

실험 #1 - Out-of-the-Box 평가

Sample semantic generalization tasks

Move Skull into Drying Rack



(unseen target object)

Stack Blue Cup on Pink Cup



(unseen language instruction)

실험 #1 - Out-of-the-Box 평가

Sample language grounding tasks

Put Eggplant into Pot

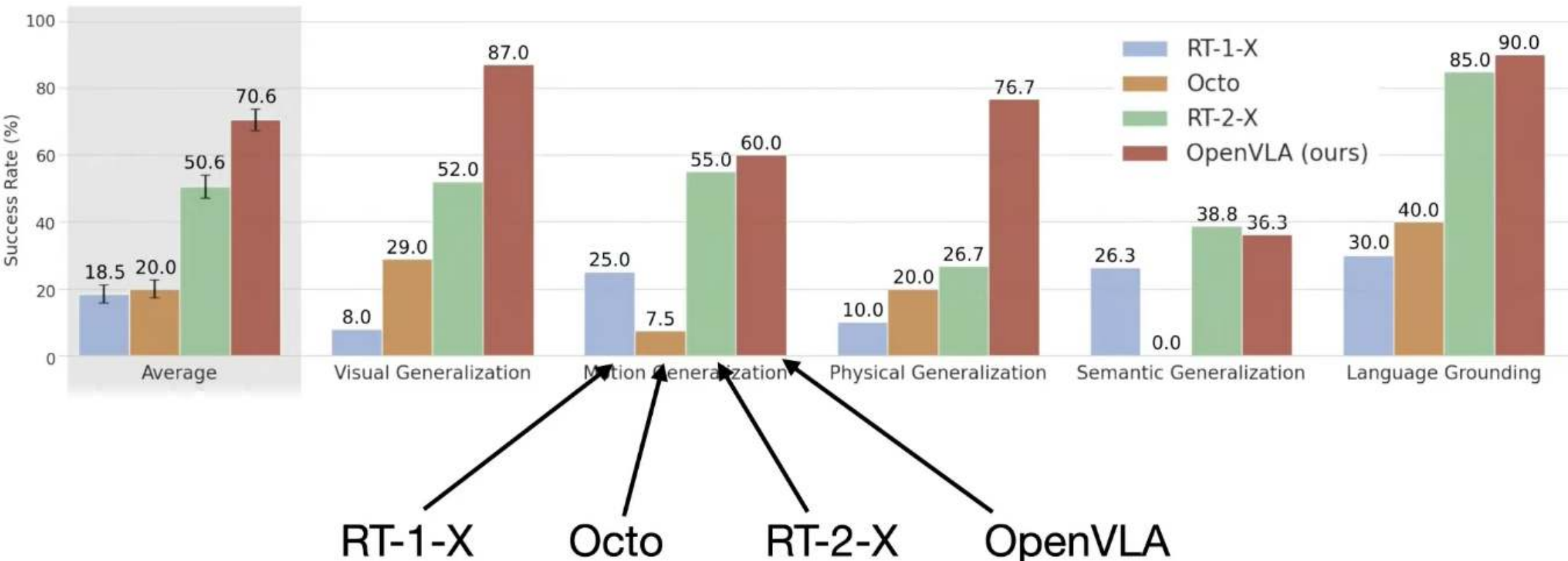


Put Red Bottle into Pot



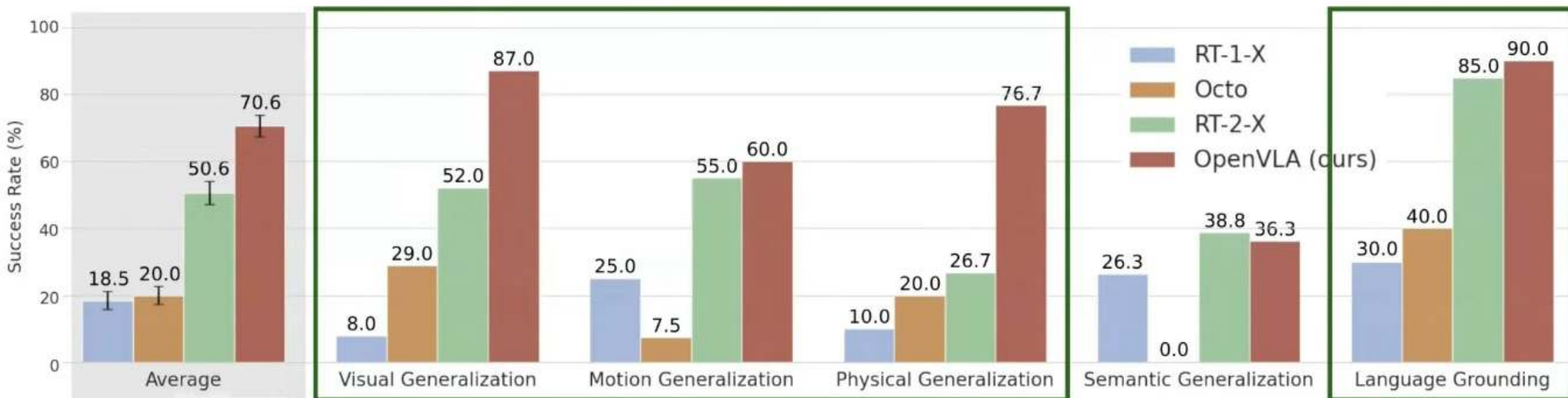
실험 #1 - Out-of-the-Box 평가

BridgeData V2 Results



실험 #1 - Out-of-the-Box 평가

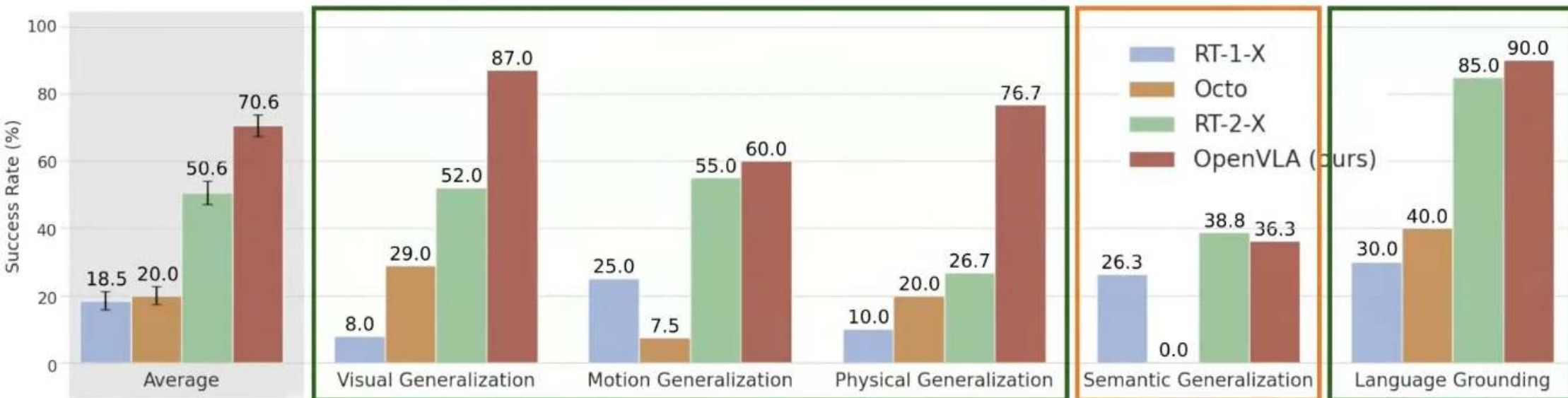
BridgeData V2 Results



OpenVLA is
strongest in most
task categories

실험 #1 - Out-of-the-Box 평가

BridgeData V2 Results



OpenVLA is
strongest in most
task categories

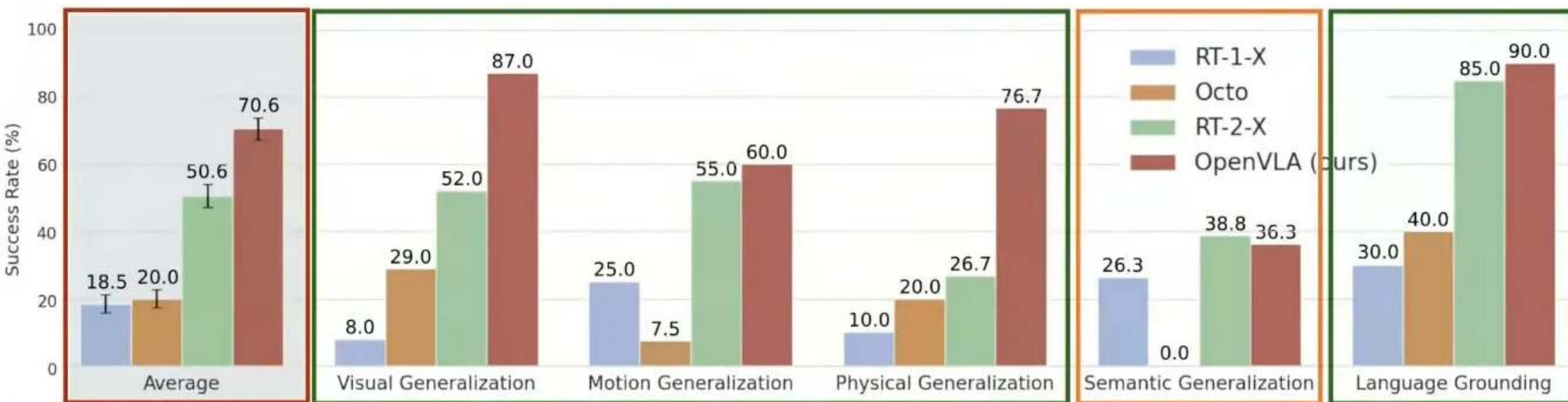
Exception: For semantic
gen, RT-2-X performs best

Potentially due to:

- (1) Larger-scale Internet pretraining
- (2) Co-fine-tuning on both robot data + VLM pretraining data

실험 #1 - Out-of-the-Box 평가

BridgeData V2 Results



OpenVLA performs best overall, outperforming next best model (RT-2-X) by +20%

OpenVLA is strongest in most task categories

Exception: For semantic gen, RT-2-X performs best

Potentially due to:

- (1) Larger-scale Internet pretraining
- (2) Co-fine-tuning on both robot data + VLM pretraining data

실험 #1 - Out-of-the-Box 평가

Qualitative Results

In a scene with many distractors, OpenVLA can approach and manipulate the correct target object

Put Eggplant into Pot



Put Yellow Corn on Pink Plate



실험 #1 - Out-of-the-Box 평가

Qualitative Results

Similarly, we can prompt OpenVLA to manipulate a different target object given same initial states

Lift Red Chili Pepper



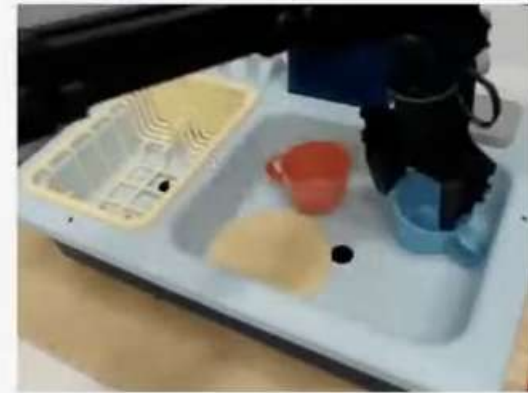
Lift Cheese



Put Pink Cup on Plate



Put Blue Cup on Plate

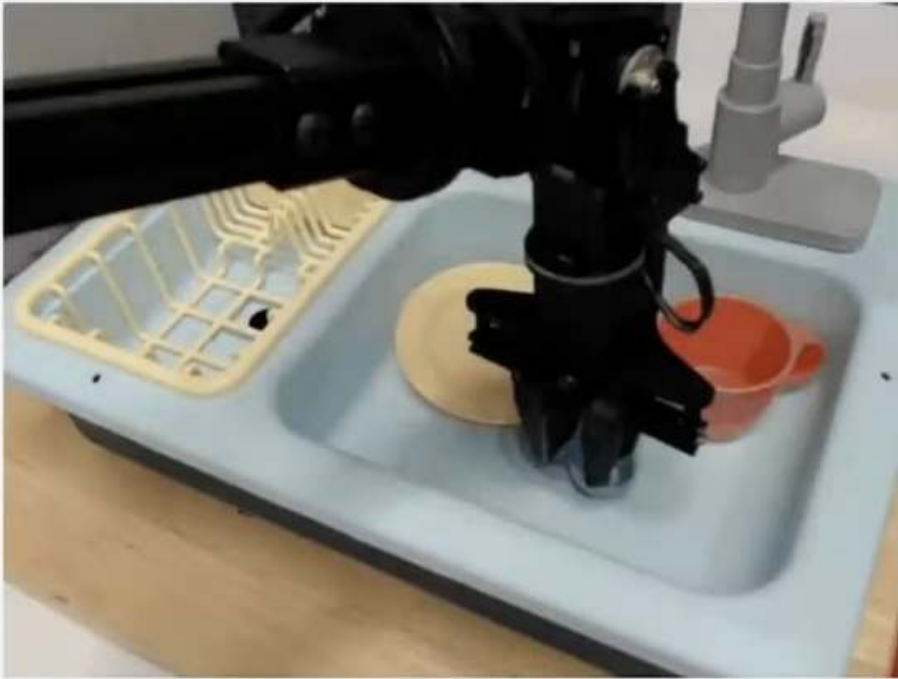


실험 #1 - Out-of-the-Box 평가

Qualitative Results

In some cases, after an initial mistake, OpenVLA can recover and successfully complete the task

Put Blue Cup on Plate



Put Eggplant into Pot



실험 #1 - Out-of-the-Box 평가

- Evaluation setups:
 - BridgeData V2 WidowX Robot 
 - Google RT-X manipulator
 - 12 tasks total, 5 rollouts per task
 - 5 in-distribution tasks
 - 7 OOD tasks
 - Unseen objects, backgrounds, object relations, web data concepts

실험 #1 - Out-of-the-Box 평가

Sample in-distribution tasks

Pick Coke Can



Move Apple near Green Can



Open Middle Drawer



실험 #1 - Out-of-the-Box 평가

Sample **OOD** generalization tasks

Move Orange near Brown Chip Bag



(unseen objects & backgrounds)

Place Banana in Pan



(unseen objects & instruction)

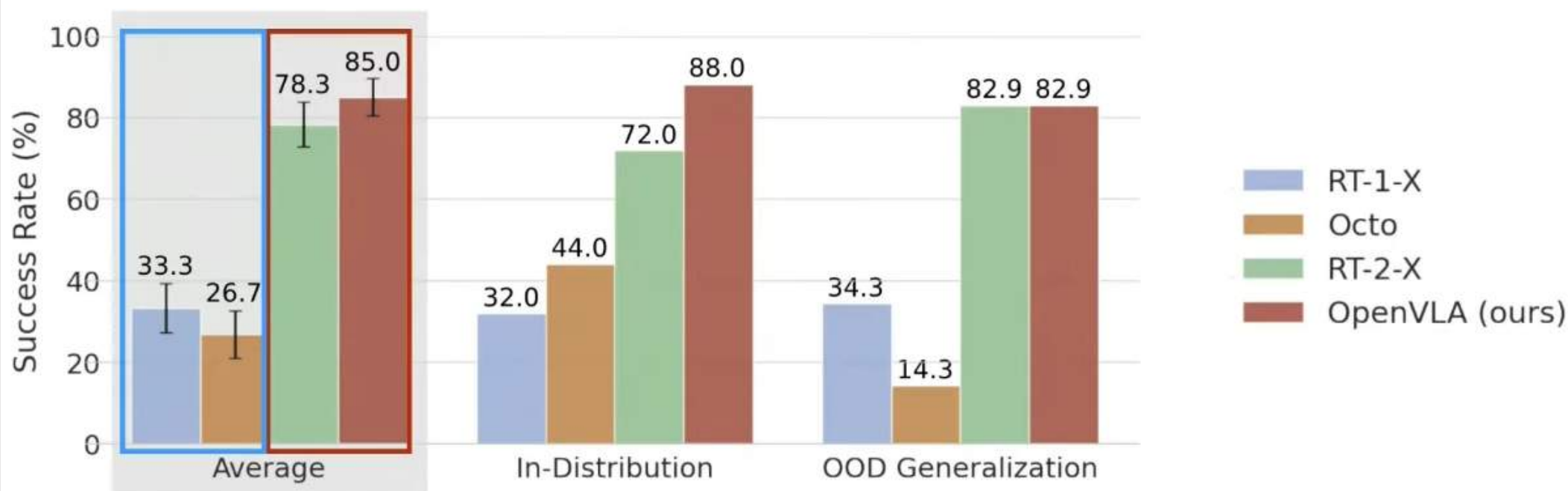
Move Coke Can to Taylor Swift



(unseen photos from Internet)

실험 #1 - Out-of-the-Box 평가

Google RT-X Robot Results



OpenVLA and RT-2-X perform comparably (overlapping error bars)

RT-1-X and Octo performance is also comparable

Both underperform compared to the VLA models

실험 #1 - Out-of-the-Box 평가

Qualitative Results

Both RT-2-X & OpenVLA perform reliably on in-distribution + basic OOD tasks

RT-2-X:

Place Banana on Plate

(OOD: unseen target object & instruction)



OpenVLA:

Place Banana on Plate

(OOD: unseen target object & instruction)



실험 #1 - Out-of-the-Box 평가

Qualitative Results

As in Bridge evals, RT-2-X performs better on difficult semantic generalization tasks

RT-2-X:

Move Coke Can near Taylor Swift
(OOD: unseen concept from Internet)



OpenVLA:

Move Coke Can near Taylor Swift
(OOD: unseen concept from Internet)



실험 (Experiments)

- Main experiments:
 - **(1) How does OpenVLA compare to prior generalist robot policies when deployed out of the box?**
 - **(2) Can OpenVLA be effectively fine-tuned on a new robot setup/task?**
- Additional experiments
 - (3) Can we use parameter-efficient fine-tuning methods (e.g., LoRA) to fine-tune OpenVLA efficiently with a smaller compute budget?
 - (4) Can we use inference-time quantization to load & run OpenVLA with limited GPU memory?

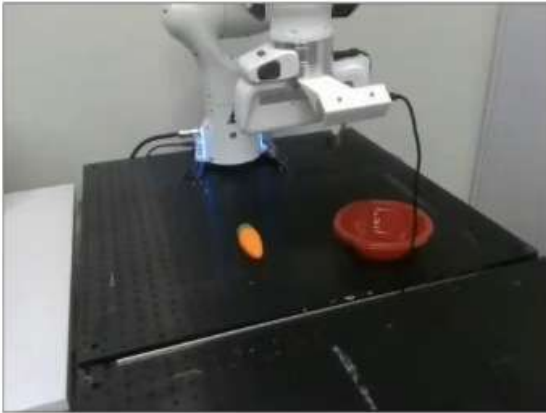
실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

- **(2) Can OpenVLA be effectively fine-tuned on a new robot setup/task?**
- Comparisons
 - **“Diffusion Policy”**: Full Diffusion Policy, trained from scratch
 - Additional bells & whistles compared to OpenVLA — robot proprio state, 2-step observation history, action chunking
 - **“Diffusion Policy (matched)”**: Version of DP that matches OpenVLA inputs/outputs and action space, trained from scratch
 - **“Octo”**: Pretrained Octo-Base model, fine-tuned
 - **“OpenVLA (scratch)”**: (Ablation) OpenVLA model with no OpenX pretraining, fine-tuned
 - I.e., fine-tuning the base pretrained VLM directly
 - **“OpenVLA (ours)”**: Full OpenVLA model, fine-tuned
- Evaluation setups:
 - **“Franka-Tabletop”**: 7-DoF table-mounted Franka Panda robot, 5 Hz control
 - 3 narrow single-instruction tasks
 - 3 diverse multi-instruction tasks
 - **“Franka-DROID”**: Same hardware setup from DROID dataset paper, 15 Hz control
 - 1 single-instruction task

실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

Franka-Tabletop: Narrow Single-Instruction Tasks

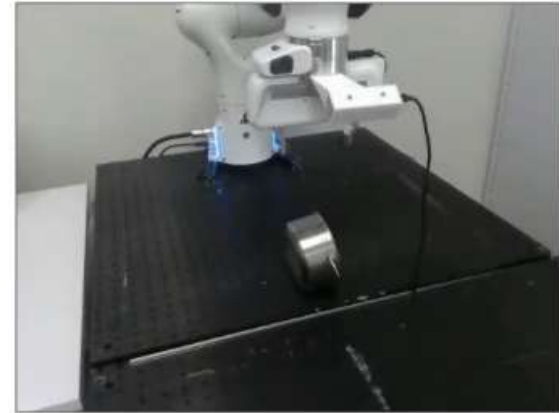
Put Carrot in Bowl (50 demos)



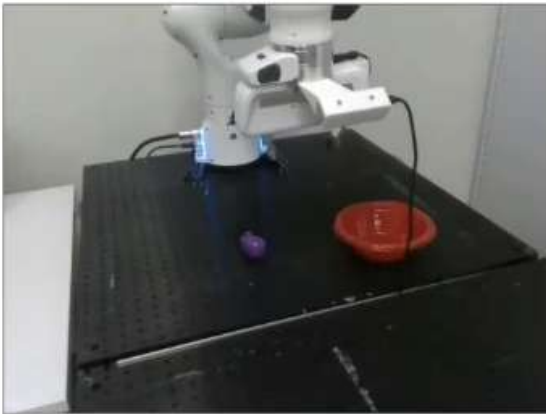
Pour Corn into Pot (50 demos)



Flip Pot Upright (10 demos)



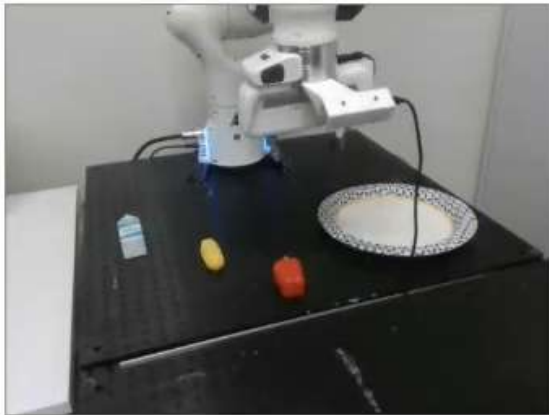
OOD Variants



실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

Franka-Tabletop: Diverse Multi-Instruction Tasks

Move <object> onto Plate (150 demos)



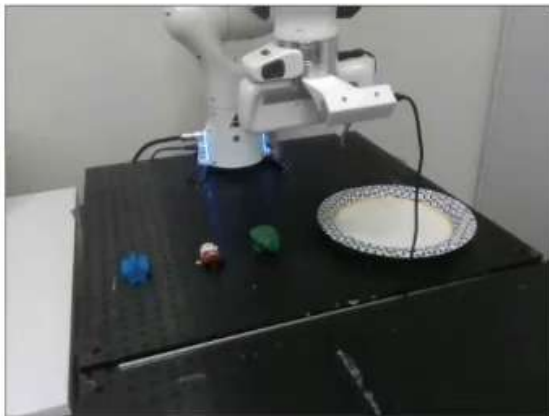
Knock <object> Over (70 demos)



Cover <object> with Towel (45 demos)



OOD Variants



실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

Franka-DROID Task

Wipe Table (70 demos)

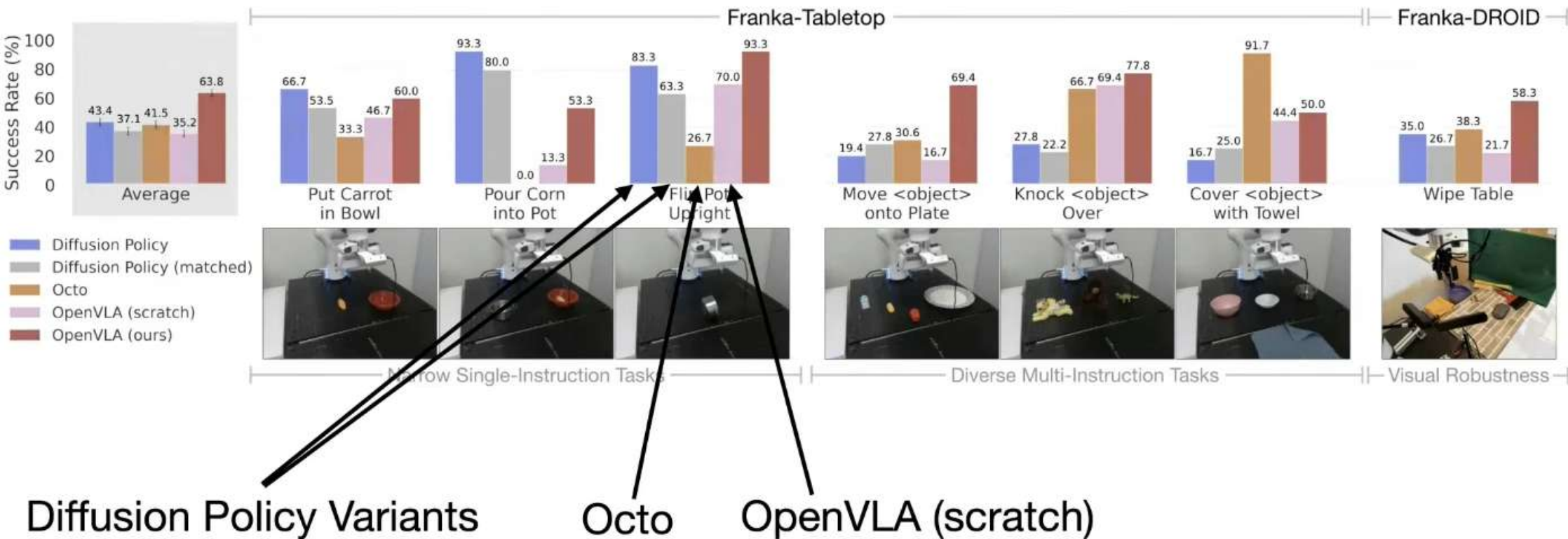


Wipe Table w/ Distractors (OOD)



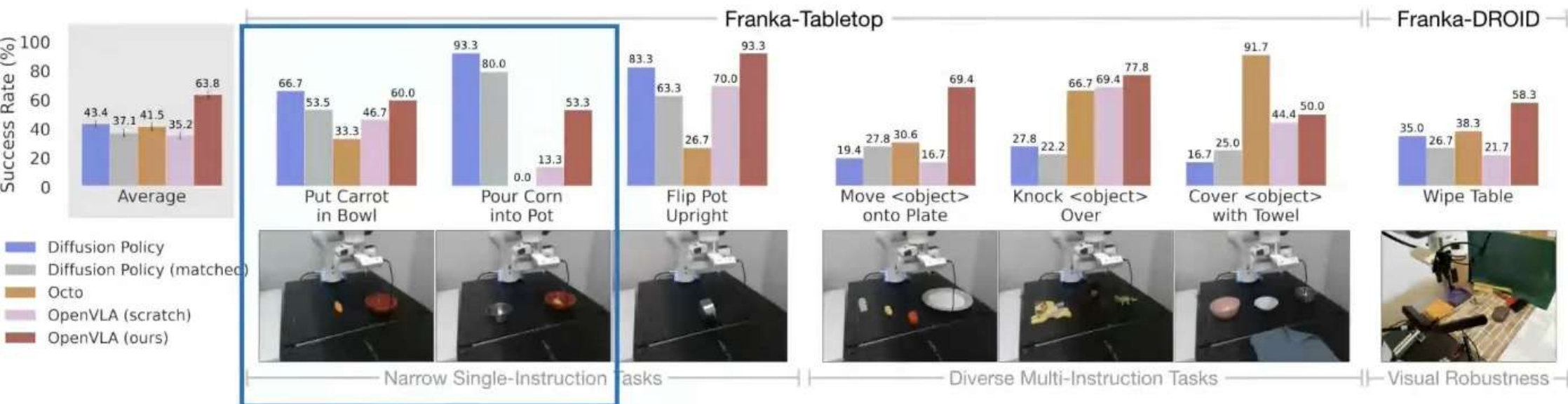
실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

Fine-Tuning Results



실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

Fine-Tuning Results



Diffusion Policy variants perform best in 2 narrow single-instruction tasks, where language grounding is not important

실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

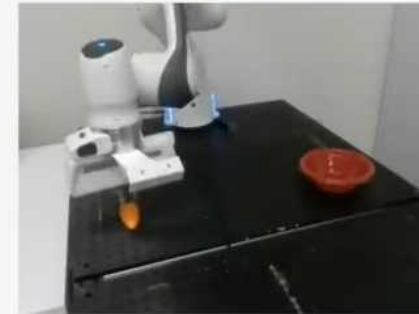
Diffusion Policy:
Put Carrot in Bowl



Octo:
Put Carrot in Bowl



OpenVLA:
Put Carrot in Bowl



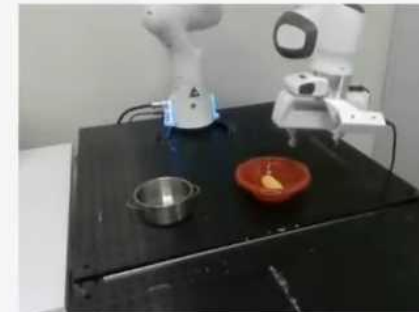
Diffusion Policy:
Pour Corn into Pot



Octo:
Pour Corn into Pot

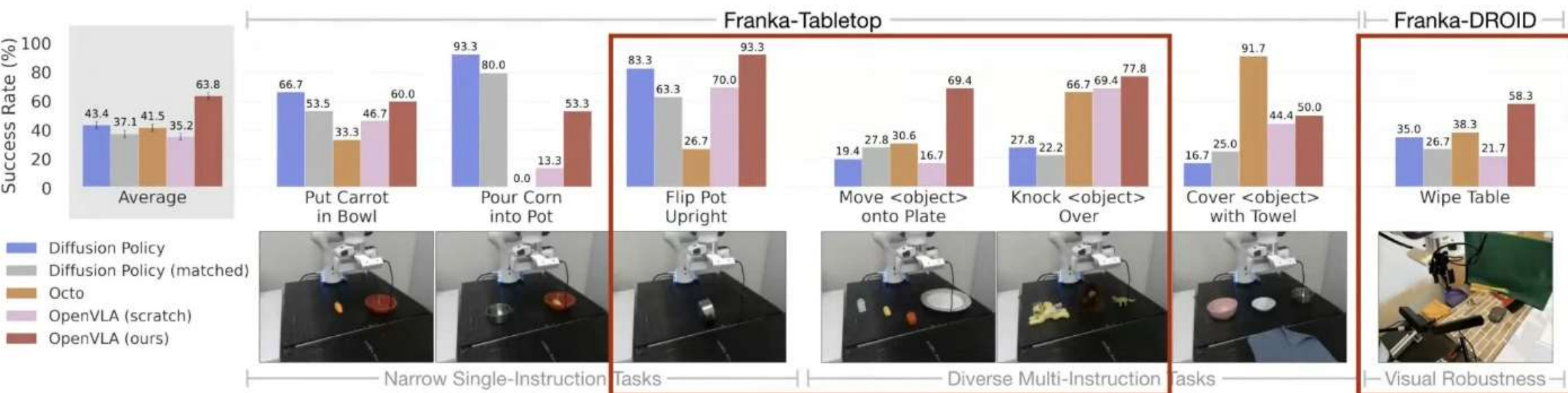


OpenVLA:
Pour Corn into Pot



실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

Fine-Tuning Results



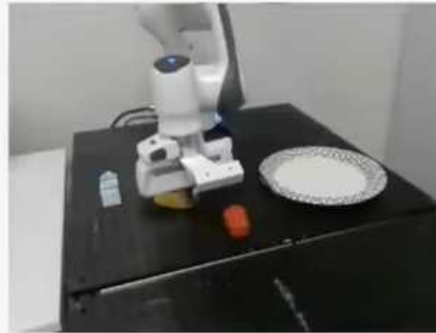
OpenVLA performs best in 4 of the 7 tasks, especially in diverse multi-instruction tasks where language grounding is important

실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

Diffusion Policy:
Move Yellow Corn onto Plate



Octo:
Move Yellow Corn onto Plate



OpenVLA:
Move Yellow Corn onto Plate



Diffusion Policy:
Knock Brown Bear Over



Octo:
Knock Brown Bear Over

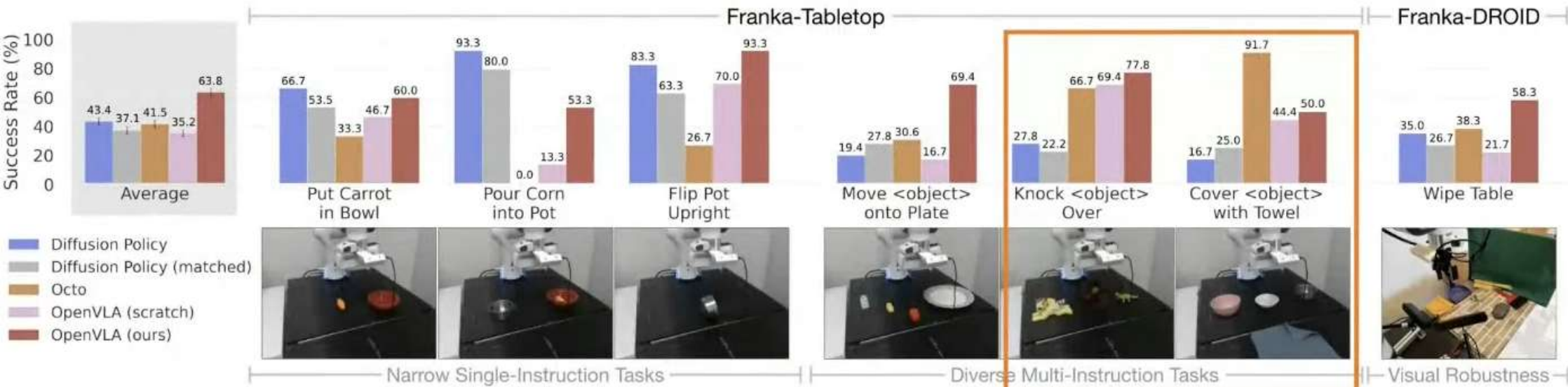


OpenVLA:
Knock Brown Bear Over



실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

Fine-Tuning Results

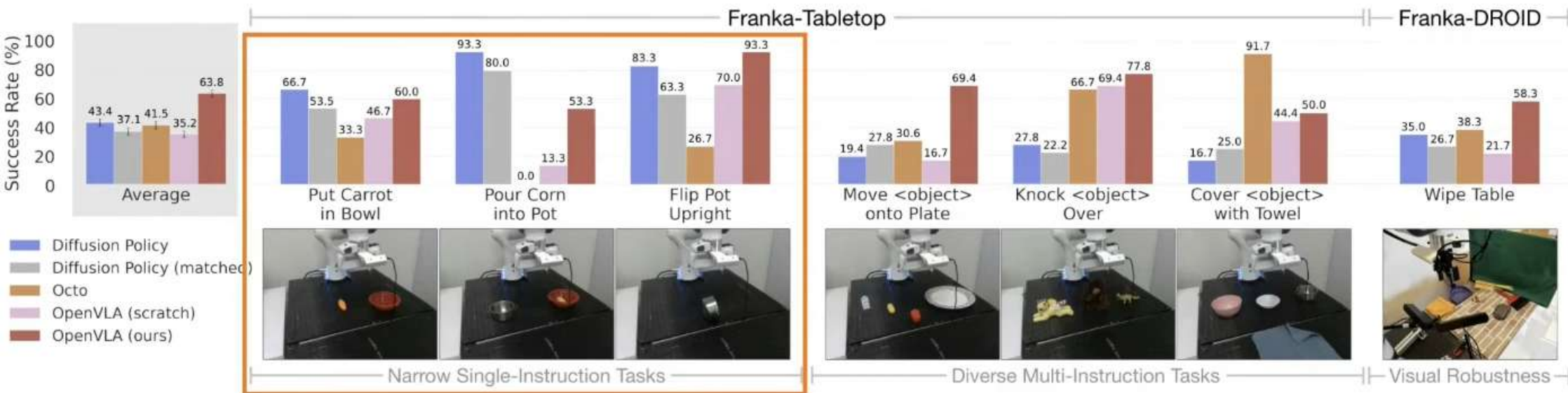


We observe that Octo exhibits strong language grounding as well

- Performs well on 2 of 3 “diverse multi-instruction” tasks

실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

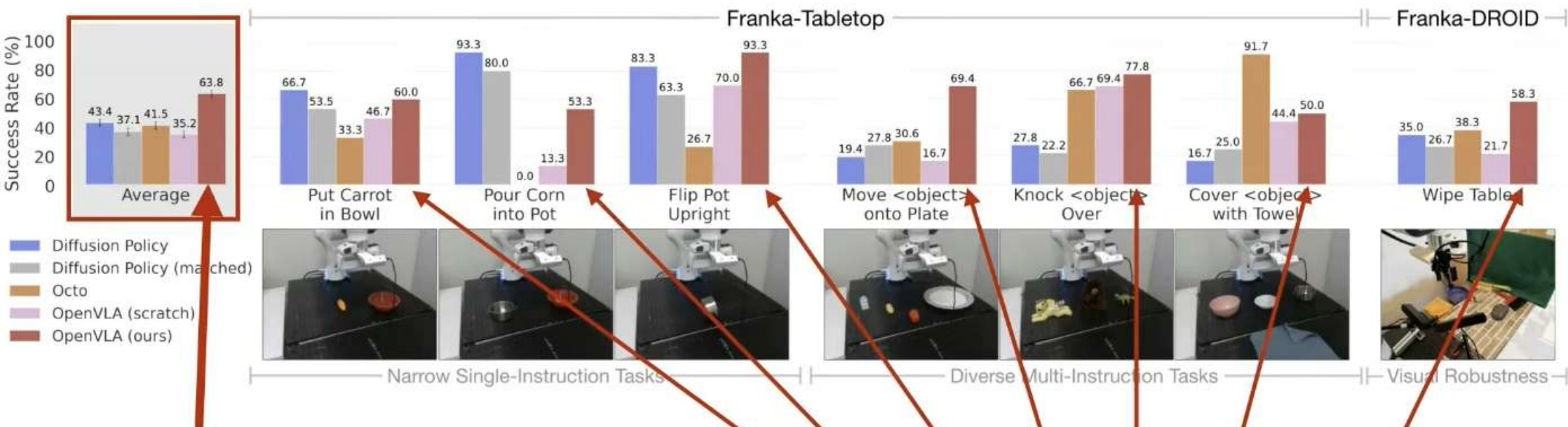
Fine-Tuning Results



However, Octo struggles on the “narrow single-instruction” tasks

실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

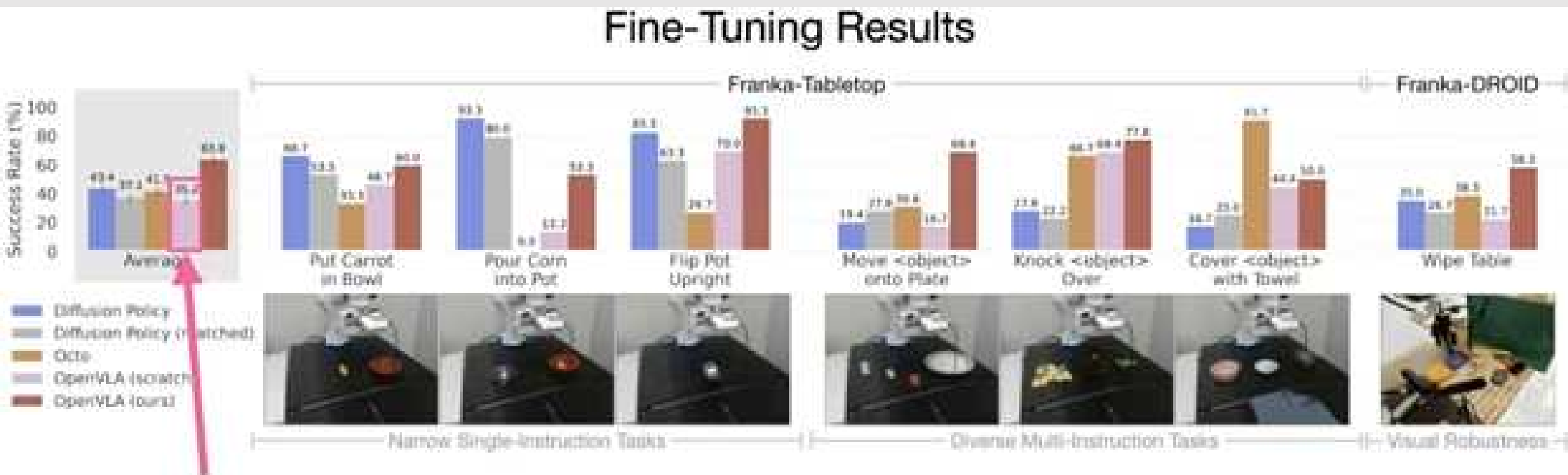
Fine-Tuning Results



OpenVLA shows strongest performance on average: outperforms next best baseline by +20.4% (absolute)

Also: Only method to get at least 50% success in all tasks

실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)



OpenVLA trained from scratch (no OpenX pretraining) struggles overall — showing that robot pretraining is crucial for good fine-tuning performance

실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

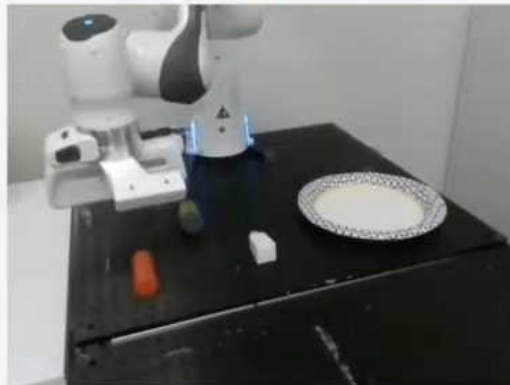
OpenVLA (Scratch):
Flip Pot Upright



OpenVLA:
Flip Pot Upright



OpenVLA (Scratch):
Move White Salt Shaker onto Plate

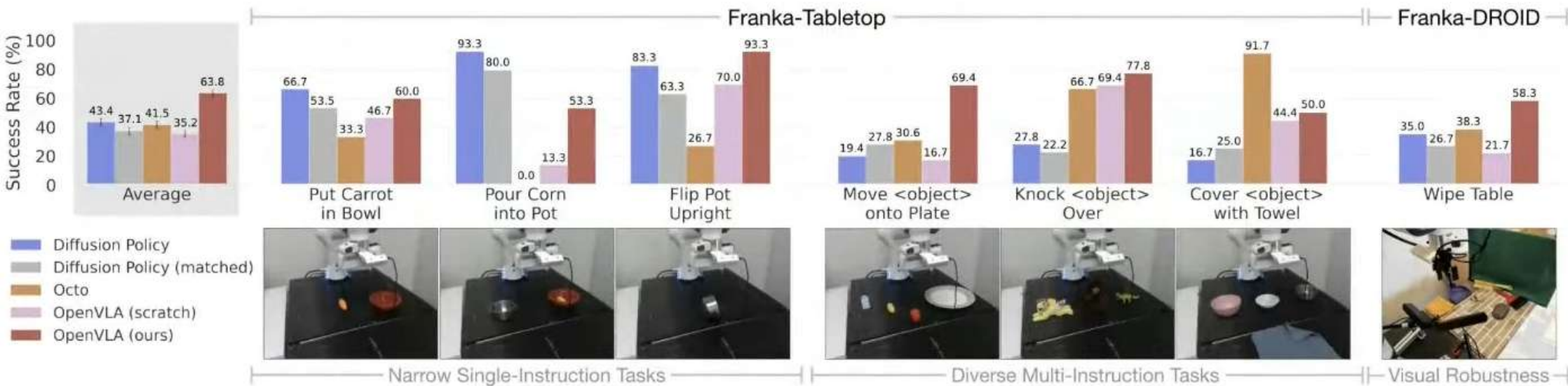


OpenVLA:
Move White Salt Shaker onto Plate



실험 #2 — 파인튜닝 평가 (Fine-Tuning Evals)

Fine-Tuning Results



(See openvla.github.io for more qualitative results!)

실험

- Main experiments:
 - (1) How does OpenVLA compare to prior generalist robot policies when deployed out of the box?
 - (2) Can OpenVLA be effectively fine-tuned on a new robot setup/task?
- Additional experiments
 - (3) Can we use parameter-efficient fine-tuning methods (e.g., LoRA) to fine-tune OpenVLA efficiently with a smaller compute budget?
 - (4) Can we use inference-time quantization to load & run OpenVLA with limited GPU memory?



추가 실험 (Additional Experiments)

- Q: Do we need to fully fine-tune OpenVLA to get good performance on downstream tasks?
- **Parameter-efficient fine-tuning**
 - We compare various partial fine-tuning strategies:
 - “**Last layer only**”: Fine-tune only last layer of LLM backbone
 - “**Frozen vision**”: Freeze vision encoder, unfreeze all else
 - “**Sandwich**”: Fine-tune vision encoder + last LLM layer, freeze all else
 - “**LoRA**”: Low-rank adaptation method, applied to all linear layers of vision/LLM backbones
 - We evaluate each method on a subset of Franka-Tabletop tasks

추가 실험 (Additional Experiments)

Parameter-Efficient Fine-Tuning Results

Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	69.7 ± 7.2 %	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	68.2 ± 7.5 %	97.6	59.7 GB
rank=64	68.2 ± 7.8 %	195.2	60.5 GB

LoRA is most effective: With rank=32, trains only 1.4% of model params but matches full fine-tuning performance!

추가 실험 (Additional Experiments)

Parameter-Efficient Fine-Tuning Results

Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	69.7 ± 7.2 %	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	68.2 ± 7.5 %	97.6	59.7 GB
rank=64	68.2 ± 7.8 %	195.2	60.5 GB

Freezing the vision encoder leads to poor performance — even if you fine-tune the entire LLM backbone!

추가 실험 (Additional Experiments)

Parameter-Efficient Fine-Tuning Results

Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	69.7 ± 7.2 %	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	68.2 ± 7.5 %	97.6	59.7 GB
rank=64	68.2 ± 7.8 %	195.2	60.5 GB

“Sandwich fine-tuning” only fine-tunes vision encoder + LLM last layer
Boost in performance demonstrates importance of fine-tuning vision encoder

추가 실험 (Additional Experiments)

- Q: Do we need a GPU with large memory to run OpenVLA models?
- **4-bit quantization (at inference time)**
 - Normally, we cast OpenVLA to bfloat16 at test time → Requires ~16 GB VRAM
 - What if we use int4 quantization?
 - Results (on subset of BridgeData V2 tasks)

Precision	Bridge Success	VRAM
bfloat16	$71.3 \pm 4.8\%$	16.8 GB
int8	$58.1 \pm 5.1\%$	10.2 GB
int4	$71.9 \pm 4.7\%$	7.0 GB

- **TLDR:**
 - Only need 7 GB VRAM to run OpenVLA with 4-bit quantization
 - Did not observe degradation in performance in our evals!
 - (8-bit quantization performs poorly... possibly due to low inference speed)

OpenVLA 설계 결정 및 인사이트 (OpenVLA Design Decisions & Insights)

- Which VLM to fine-tune?
 - Tried IDEFICS (v1), LLaVA (v1.5), and Prismatic VLM
 - IDEFICS << LLaVA (+35% *absolute* SR in Bridge) < Prismatic VLM (+10%)
- Which image resolution?
 - Tried 224x224 and 384x384
 - Observed no difference in performance in Bridge evals, but 384x384 takes 3x longer to train → We opted for 224x224
- Freeze or fine-tune vision encoder?
 - Prior work on VLMs found freezing vision to be better
 - We found the opposite: Fine-tuning was crucial for both zero-shot + fine-tuning evals
- How long to train?
 - Consistently found that training for more epochs was better (e.g. 30 epochs!)
- Which learning rate?
 - LR sweep → Same LR used in VLM pretraining ($2e-5$) led to best results
 - Used constant LR schedule for all experiments

OpenVLA 한계점 (OpenVLA Limitations)

- Only supports single-frame input and single-step action output
 - (Because large model → low training/inference speed with more inputs/outputs)
 - Q: How would performance change with additional inputs/outputs (e.g., robot proprio state, observation history, action chunking)?
- Lack of support for high-frequency control / bi-manual manipulation robots
 - E.g., ALOHA
 - (Difficult to run OpenVLA at >7 Hz control without special tricks — e.g. if we are using local GPUs or even remote H100 servers)
 - Q: Can VLM inference speedup tricks enable high-frequency VLA control?
- VLM fine-tuned only on robot action data (not co-fine-tuned on robot data + VQA data, as done in RT-2/RT-2-X)
 - Potential catastrophic forgetting of Internet-pretraining concepts that do not appear in robot interaction data
 - Q: Would OOD generalization performance improve significantly if OpenVLA is also fine-tuned on VQA data?
- We hope researchers build off of OpenVLA to answer some of these questions!

Thankful for my amazing collaborators!

OpenVLA: An Open-Source Vision-Language-Action Model

Moo Jin Kim^{*,1}, Karl Pertsch^{*,1,2}, Siddharth Karamcheti^{*,1,3},

Ted Xiao⁴, Ashwin Balakrishna³, Suraj Nair³, Rafael Rafailov¹, Ethan Foster¹, Grace Lam,

Pannag Sanketi⁴, Quan Vuong⁵, Thomas Kollar³, Benjamin Burchfiel³, Russ Tedrake^{3,6}, Dorsa Sadigh¹,

Sergey Levine², Percy Liang¹, Chelsea Finn¹,

^{*}Equal contribution

¹Stanford University, ²UC Berkeley, ³Toyota Research Institute, ⁴Google DeepMind, ⁵Physical Intelligence,
⁶MIT,

Thank you for listening! :) Happy to take any questions.

Language Grounding

로봇이나 AI가 언어로 주어진 지시(명령, 설명)를 실제 세계의 대상(object), 속성(attribute), 동작(action)과 연결하는 능력

◆ 정의

- **Language (언어)**: 사람이 내린 지시 → “빨간 컵을 집어라”
- **Grounding (기반/연결)**: 언어를 실제 환경에 맞게 해석 → 빨간 컵이 무엇인지, 어디 있는지 인식
- 로봇이 이 해석을 기반으로 **정확히 행동을 실행**하는 것

◆ 예시

1. 시각적 grounding

1. “파란색 공을 잡아라” → 카메라 이미지에서 파란 공을 찾아냄

2. 행동 grounding

1. “공을 상자 안에 넣어라” → ‘집기(pick)’ 동작 + ‘이동(move)’ 동작 + ‘놓기(place)’ 동작으로 연결

3. 맥락적 grounding

1. 테이블 위에 여러 개의 물체가 있을 때
2. “오른쪽에 있는 컵을 들어 올려라” → 상대적 위치까지 반영해 올바른 물체 선택

◆ OpenVLA와 Language Grounding

• 테스트 방식: 동일한 초기 환경에서

- Episode 1: “컵을 집어라” → 컵 조작 성공?
- Episode 2: “병을 집어라” → 병 조작 성공?

• 같은 환경에서 ****지시문(prompt)****만 달라졌을 때, 로봇이 올바른 대상(object)을 고르고 조작할 수 있는지 평가 → 이것이 **language grounding 성능**

✓ 정리하면,

로봇이 언어 지시를 실제 세계의 대상, 속성, 행동을 올바르게 연결하는 능력

오토리그레시브(Autoregressive) 액션 생성 - 1

◆ 1. Autoregressive의 기본 개념

Autoregressive = 자기회귀

이전 단계에서 생성한 출력을 다음 단계의 입력으로 사용하는 방식.
즉, ****시퀀스(sequence)****를 한 토큰씩 순차적으로 생성.

LLM의 텍스트 생성과 같은 구조:

"오늘은" → "날씨가" → "맑다" → ... 이런 식으로 하나씩 생성.

◆ 2. 로봇 액션에서의 Autoregressive

로봇 제어에서 ****행동(action)****도 시퀀스로 표현할 수 있습니다.

예: Δx , Δy , Δz , $\Delta roll$, $\Delta pitch$, Δyaw , gripper (총 7개의 action 토큰)

- Autoregressive 방식에서는 이 7개를 **순서대로 예측**합니다.

Step 1: Δx 생성

Step 2: Δy 생성 (Δx 결과를 조건으로)

Step 3: Δz 생성 (Δx , Δy 를 조건으로)

... 반복해서 모든 액션 토큰을 채움

따라서 **각 액션 토큰이 직전 결과에 의존** → LLM의 텍스트 생성과 동일한 구조.

◆ 3. 장점

언어모델 구조와 호환

LLM처럼 “다음 토큰 예측”으로 훈련 가능

시퀀스 모델(Transformer)에 그대로 적용 가능

복잡한 분포 학습 가능

액션 토큰 간 **조건부 의존성**을 잘 반영 가능

오토리그레시브(Autoregressive) 액션 생성 -2

◆ 4. 단점

속도 문제

액션 하나를 만들기 위해 7회(forward pass) 추론 필요 → 지연(latency)↑
실시간 제어에 부담

에러 누적

앞 단계 토큰이 잘못되면 이후 모든 토큰에 오차 전파
따라서 **action chunking**(여러 step을 한 번에 예측) 같은 개선 기법 연구됨

◆ 5. 비교

Autoregressive (RT-1, OpenVLA)

한 토큰씩 순서대로 생성
느리지만 안정적, LLM 친화적

Parallel Decoding / Diffusion Policy

전체 액션 벡터를 한 번에 생성 (병렬)
빠르지만 학습 난이도 ↑

✓ 정리하면,

Autoregressive 액션 생성은 로봇의 행동을 시퀀스 토큰으로 쪼개어 이전 토큰에 조건부로 다음 토큰을 생성하는 방식입니다.

즉, “LLM이 문장을 단어 단어 생성하듯이, 로봇은 액션을 토큰 단위로 순차 생성”한다고 보면 됩니다.

OpenVLA-OFT+

OpenVLA-OFT+

Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success



Moo Jin Kim



Chelsea Finn



Percy Liang

Sound On



Paper, code, models at
openvla-oft.github.io



Center for
Research on
Foundation
Models

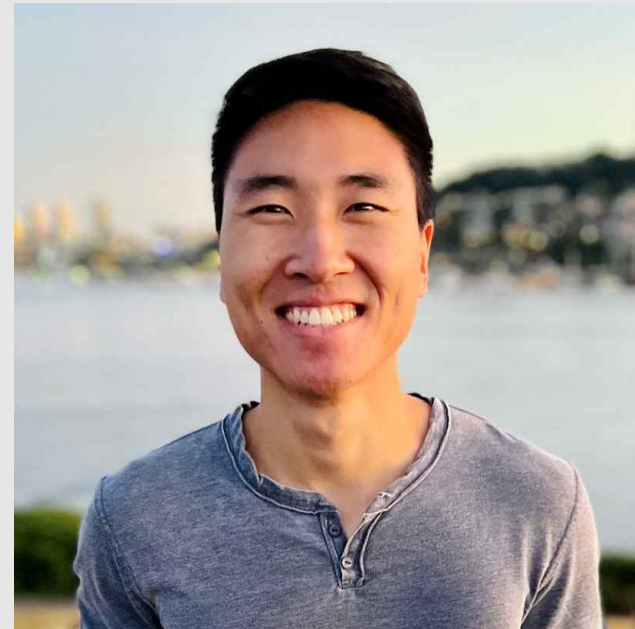


01

OpenVLA-OFT+ 개요

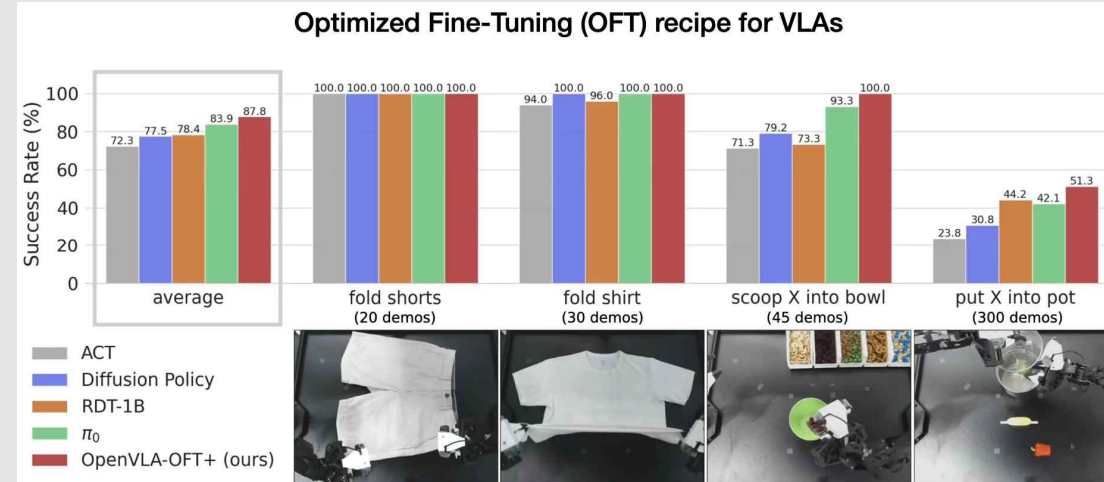
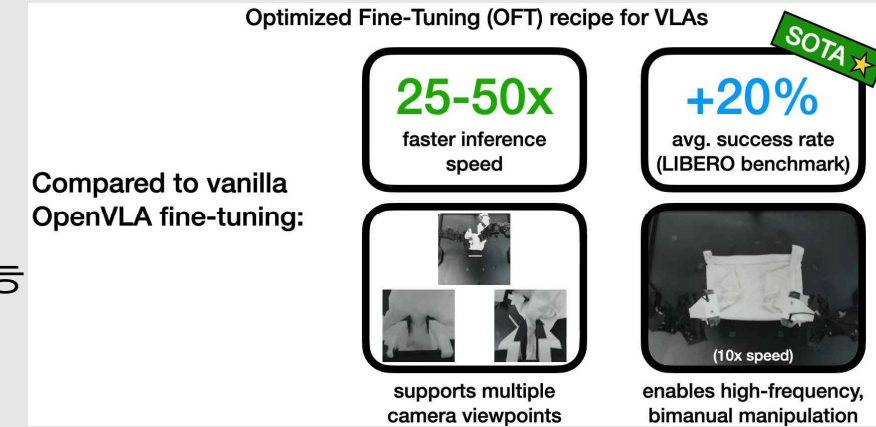
<https://moojink.com/>

<https://www.youtube.com/watch?v=-0s0v3q7mBk>



동기 (Motivation)

- 연구 목적 : 비전-언어-액션(VLA) 모델의 파인튜닝 과정에서 **속도와 성공률**을 동시에 개선.
- 제안 방법 : **OpenVLA-OFT+**라는 새로운 최적화 파인튜닝 레시피.
- 성과:
 - 기존 OpenVLA 대비 **추론 속도 25~50배 향상**
 - Libero 벤치마크에서 **평균 성공률 20% 증가 (SOTA 성능)**
- 특징:
 - 다중 이미지 입력 지원**
 - 실제 Aloha 로봇에서 고주파수(bimanual) 조작 작업 수행 가능**
- 비교 평가:
 - 기존 모방학습 기법 (ACT, Diffusion Policy)
 - 다른 VLA 정책 (RDT-1B, Pi-Z)
- 결과: OpenVLA-OFT가 가장 높은 평균 성공률 달성.



OpenVLA vs OpenVLA-OFT 비교 인포그래픽

학습 방식	Autoregressive Discrete tokens Next-token prediction	Parallel decoding Action chunking Continuous + L1 (선택적 FiLM)
추론 속도	느림 (순차 토큰 생성)	25~50배 빠름
성공률	보통 (Liber0 기준)	+20% 향상 (97%, SOTA)
입력 처리	단일 이미지 입력	다중 이미지 입력
실제 로봇 성능	저주파수 작업 한계	고주파수(bimanual) 작업 가능
언어 지시 능력	기본 언어 grounding	언어 따르기 강화 (FiLM)
비교 대상 성능	IL 기법 대비 낮거나 비슷	모든 비교군 대비 최고 성능

OpenVLA vs OpenVLA-OFT 비교

구분	기존 OpenVLA	개선된 OpenVLA-OFT
학습 방식	오토리그레시브(autoregressive) 액션 생성 이산적(discrete) 액션 토큰 다음 토큰 예측 (cross-entropy loss)	병렬 디코딩(parallel decoding) + 액션 청킹(action chunking) 연속적(continuous) 액션 + L1 회귀 최적화 선택적 FiLM 적용 (언어 지시 강화)
추론 속도	느림 (순차 토큰 생성으로 인한 지연)	25~50배 빠름 (병렬 액션 예측)
성공률	Libero 벤치마크 기준 평균 성능	+20% 향상 (97% 달성, SOTA)
입력 처리	단일 이미지 입력 중심	다중 이미지 입력 지원
실제 로봇 성능	저주파수(bimanual) 작업에 한계	고주파수(bimanual) 조작 가능 (Aloha 로봇)
언어 지시 능력	기본적인 언어 grounding만 지원	FiLM을 통해 언어 따르기(language following) 능력 강화 (필요 시 적용)
비교 대상 성능	기존 imitation learning 기법 대비 낮거나 비슷	ACT, Diffusion Policy, RDT-1B, Pi-Z 모두 대비 최고 성능

📌 정리하면, **OpenVLA-OFT는 OpenVLA의 구조적 한계를 극복하여**

- 훨씬 빠른 속도,
- 더 높은 성공률,
- 실제 로봇 적용성 강화

라는 세 가지 측면에서 큰 진전을 이뤘습니다.

OpenVLA vs OpenVLA-OFT 비교 인포그래픽

Optimized Fine-Tuning (OFT) recipe for VLAs



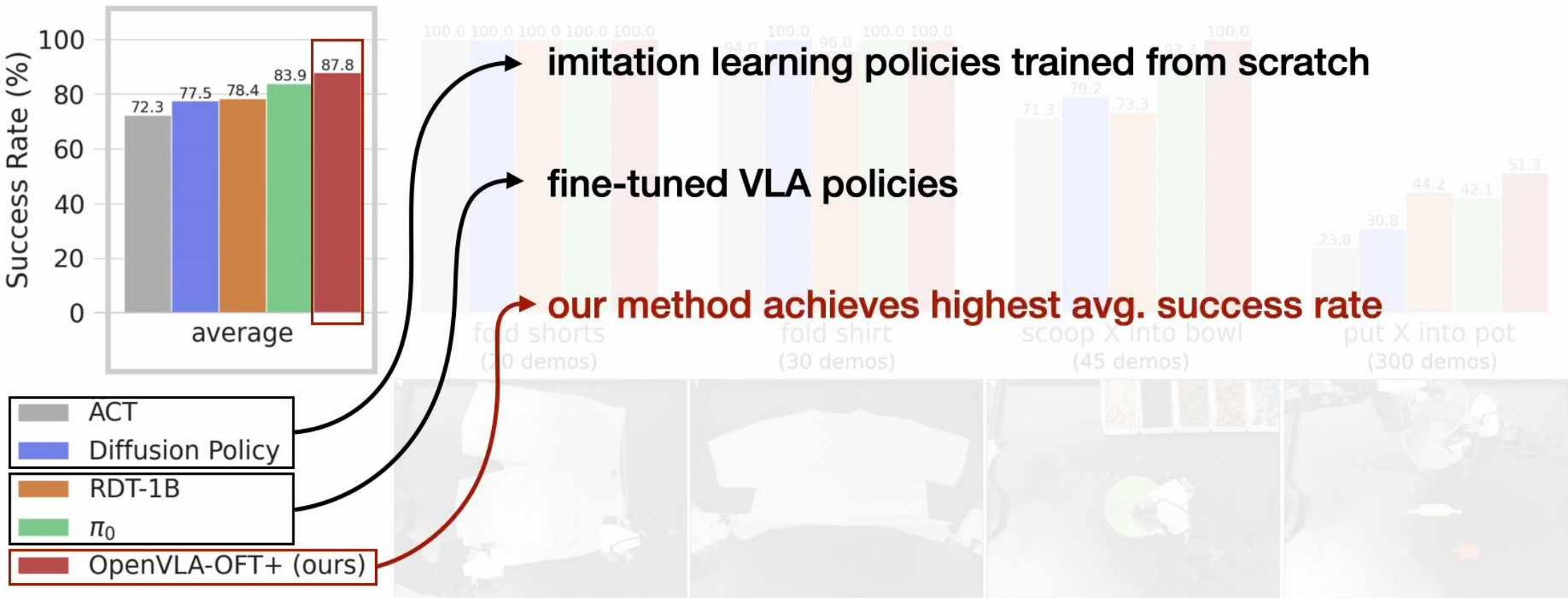
OpenVLA vs OpenVLA-OFT 비교 인포그래픽

Optimized Fine-Tuning (OFT) recipe for VLAs



OpenVLA vs OpenVLA-OFT 비교 인포그래픽

Optimized Fine-Tuning (OFT) recipe for VLAs



OpenVLA와 비교

• 기존 OpenVLA 파인튜닝 방식

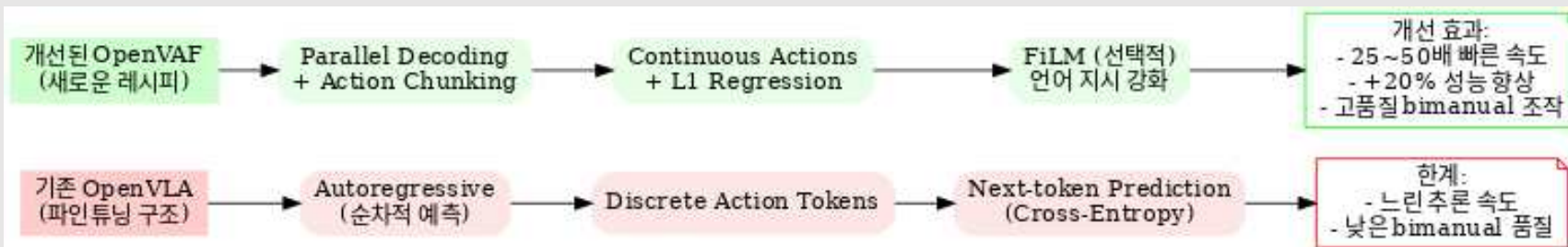
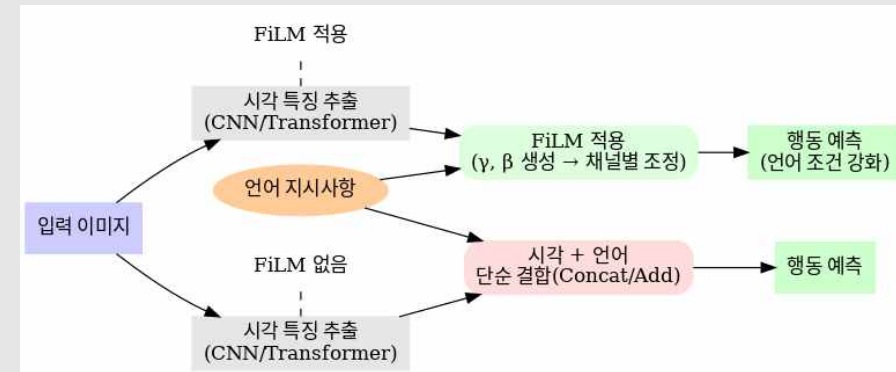
- 오토리그레시브(autoregressive) 액션 생성
- 이산적(discrete) 액션 토큰 기반
- 다음 토큰 예측(next-token prediction) 목표
- 한계: 추론 속도 느림, 양팔(bimanual) 로봇 조작 시 품질 저하

• OpenVLA-OFT+ 새로운 레시피

- 병렬 디코딩 + 액션 청킹 → 추론 속도 및 작업 성능 크게 향상
- 연속 액션 + L1 회귀 최적화 → 성능 추가 개선
- FiLM 적용(선택적) → 언어 지시사항(language following) 능력 강화

• 버전 구분

- OpenVLA-OFT+: FiLM 미적용
- OpenVLA-OFT+: FiLM 적용
- 단, 언어 grounding이 충분하다면 FiLM은 선택 사항



Original OpenVLA formulation:

- autoregressive generation
- discrete actions
- next-token prediction

Limitations: slow inference, poor quality on bimanual manipulation tasks

OpenVLA-OFT+ (ALOHA Robot)

- parallel decoding w/ action chunking
- continuous actions
- L1 regression objective
- feature-wise linear modulation (FiLM)

OpenVLA vs OpenVLA-OFT 비교 인포그래픽

OpenVLA-OFT+

(ALOHA Robot)

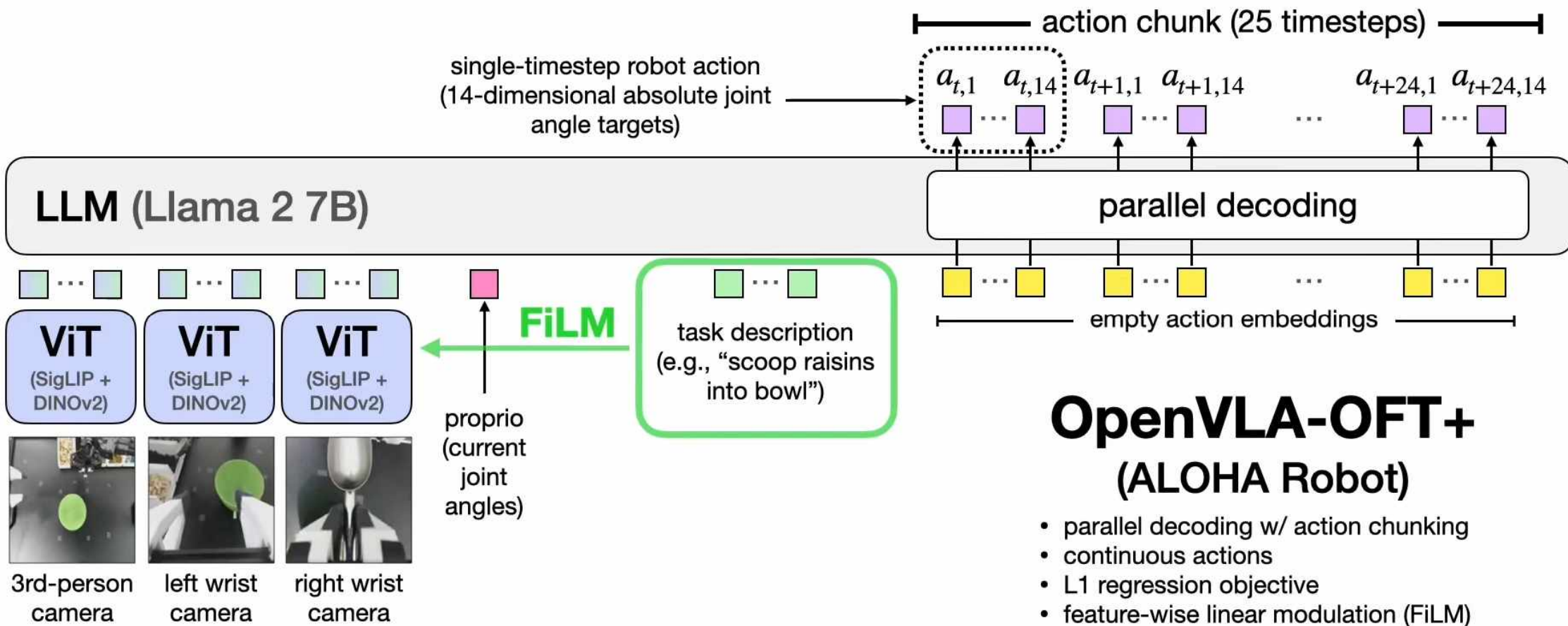
“OpenVLA-OFT+” (with FiLM)

“OpenVLA-OFT” (no FiLM)

- parallel decoding w/ action chunking → boosts inference speed & task performance
- continuous actions
- L1 regression objective } → improves task performance further
- feature-wise linear modulation (FiLM) → enhances language following ability

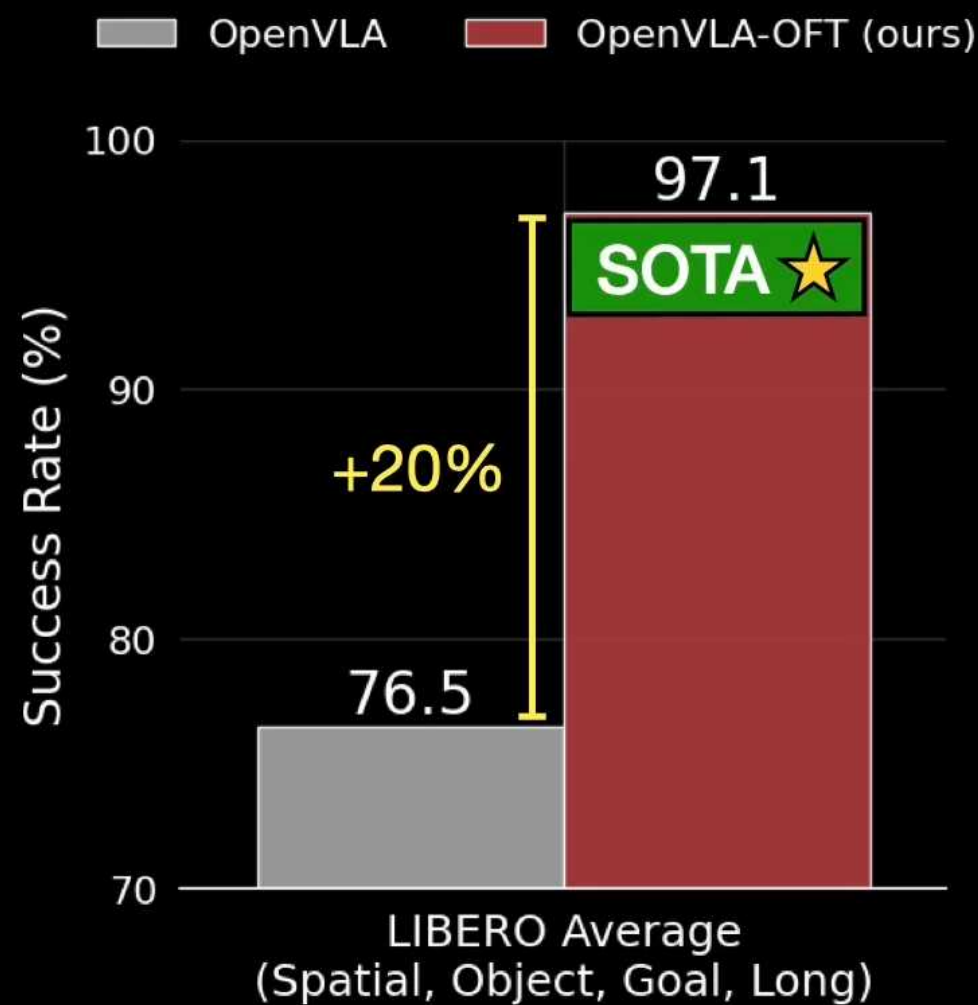
Note: FiLM is not always necessary!
(E.g., not needed in LIBERO sim environments)

OpenVLA vs OpenVLA-OFT 비교 인포그래픽



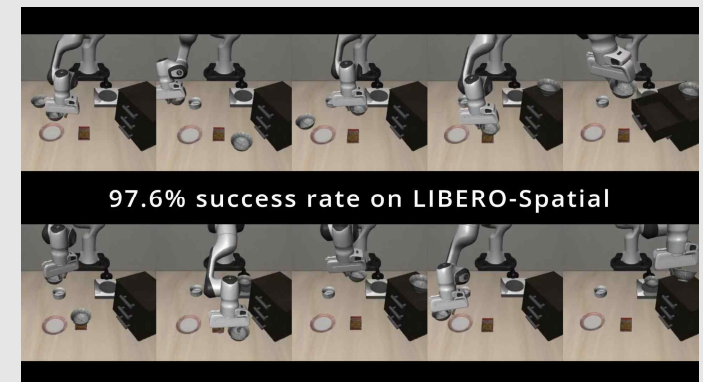
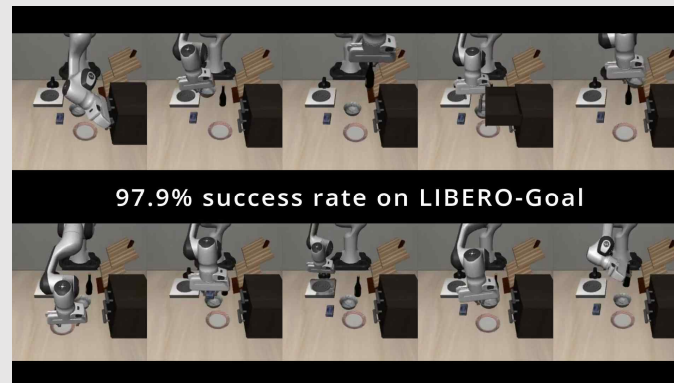
OpenV

OpenVLA-OFT in Simulation



OpenVLA-OFT 성능 결과

- OpenVLA-OFT 성능 결과
시뮬레이션 Libero 벤치마크에서 평균 성공률 97% 달성
기존 OpenVLA 파인튜닝 정책 대비 +20% 성능 향상
SOTA(최첨단 성능) 달성
- 적용 사례
실제 Aloha 로봇에 적용 가능 (영상/다이어그램 확인 가능)
- 세부 평가 항목
Libero Spatial : 공간 내에서 위치 제어 정확도
Libero Object : 다양한 물체 조작
Libero Goal : 목표 지점 도달 여부
Libero Long : 장기적/다단계 행동 수행 능력



요약

OpenVLA-OFT+는

병렬 디코딩 + 액션 청킹

연속 액션 + L1 회귀 최적화

필요 시 FiLM 적용

을 통해 **25~50배 빠른 추론 속도**와 **20% 향상된 성공률**을 달성했으며,
실제 **Aloha** 로봇과 **Libero** 벤치마크에서 최신 성능을 보여주었습니다.

Libero 벤치마크란

Libero 벤치마크란?

• 정식 명칭: LIBERO (Language-Instructed Benchmark for Robotic Control)

• 출시 배경:

- 최근 로봇 연구에서는 단순한 조작 성공률뿐 아니라 언어 지시를 이해하고 따르는 능력이 중요해졌습니다.
- Libero는 **비전-언어-액션(VLA) 모델**이 실제 로봇 작업에서 얼마나 잘 일반화할 수 있는지를 측정하기 위해 제안된 **표준 벤치마크 세트**입니다.

Libero의 주요 특징

1. 언어 지시 기반

1. "컵을 테이블 위에 놓아라", "노란 블록을 잡아 올려라" 같은 **자연어 지시**를 입력으로 사용.
2. 모델이 단순 조작뿐 아니라 **언어를 행동으로 연결**할 수 있는지 평가.

2. 시뮬레이션 + 실제 로봇 연결

1. Mujoco 기반 시뮬레이터에서 다양한 환경을 구성.
2. 연구자들이 동일한 조건에서 결과를 비교 가능.
3. 일부 태스크는 실제 로봇(Aloha, Franka 등)으로도 이전 가능.

3. 다양한 작업 시나리오

Libero는 4가지 하위 벤치마크로 나뉘어 있음:

1. **Libero-Spatial**: 물체의 위치/공간적 배치 조작 능력 평가
2. **Libero-Object**: 다양한 물체 유형을 구별하고 조작하는 능력
3. **Libero-Goal**: 목표 지점 도달 능력 (ex. 컵을 특정 위치로 옮기기)
4. **Libero-Long**: 여러 단계가 필요한 장기적/복합 행동 수행 능력

Libero 벤치마크란

왜 중요한가?

기존 로봇 벤치마크는 특정 동작이나 물리적 성공률만 보았다면,

Libero는 언어-시각-행동을 통합적으로 평가한다는 점에서,

→ ****VLA 모델(OpenVLA, OpenVLA-OFT, RT-1/2, Pi0 등)**의 성능을 비교하는 표준 기준이 되고 있음.**

특히 OpenVLA-OFT가 **Libero 전체 평균 97% 성공률**을 기록했다는 것은,

"언어로 지시된 다양한 조작을 거의 완벽하게 수행할 수 있다"는 것을 의미합니다.

Libero 벤치마크란

Libero는 데이터셋(dataset)이 아니라, “로봇 학습 성능을 평가하기 위한 벤치마크(benchmark suite)”입니다.

📦 데이터셋(dataset) → 학습에 사용되는 원천 데이터 (시뮬레이션 환경의 조작 데이터, 로봇 시연 기록, 영상-행동 쌍 등).

🔍 벤치마크(benchmark) → 특정한 태스크 묶음과 평가 프로토콜을 정의해서, 여러 모델/알고리즘을 같은 조건에서 비교할 수 있게 만든 것.

Libero 벤치마크는

여러 가지 시뮬레이션 환경(가구 조립, 가전제품 조작, 문 열기 등)을 포함하고,

각 과제(task)에 대한 성공률(success rate) 같은 메트릭을 제공합니다.

연구자들은 자신들의 VLA나 정책(policy)을 학습한 뒤, Libero 환경에서 실행시켜 성능을 비교합니다.

즉, **Libero 자체가 “훈련 데이터셋”을 제공하는 건 아니고, 테스트·평가용 환경과 과제 집합이라고 이해하면 정확합니다.**

👉 정리하면:

Open X-Embodiment, BridgeData → 데이터셋 중심.

Libero, Meta-World → 벤치마크(평가 환경) 중심.

L2 대신 L1을 쓰는 핵심 목적

- **이상치(Outlier) 강건성**: 제곱 페널티가 없어 큰 오차의 영향이 과도하게 커지지 않습니다.
- **노이즈 라벨에 안정적**: 사람 데모·센서 노이즈 등 불균질/heavy-tailed 분포에 덜 민감합니다.
- **그래디언트 안정성**: $\partial|e|=\text{sign}(e)$ 로 큰 오차에서도 기울기 폭주가 줄어 학습이 안정적입니다.
- **전형적 사례 최적화**: 몇몇 큰 오차보다 전체 구간의 균등한 오차 감소에 초점을 맞춥니다.
- **분포 가정 적합**: L1은 **Laplace(라플라스) 오차 가정**에 최적(중앙값 추정)입니다.
- **연속 제어에 유리**: 미세 보정($\Delta\text{pose}/\Delta\text{joint}$)에서 과별점 없이 세밀한 추종이 가능합니다.

PI-Zero

Pi-Zero

Physical Intelligence

1. 모델 개요

1. π_0 : 범용 로봇 제어를 위한 **VLA + Diffusion Flow Model**
2. 백본: **Paligemma VLM (~3B) + 액션 전문가(300M)**
3. 학습 데이터: **π Cross-Embodiment Dataset**(다양한 로봇·환경)

2. VLM 구성

1. **SigLIP**(이미지 인코더) + **Gemma**(언어 명령)
2. 비전·언어 입력 처리, **액션 토큰 미포함** → 시각 이해 명확화

3. Action Expert (Flow Matching)

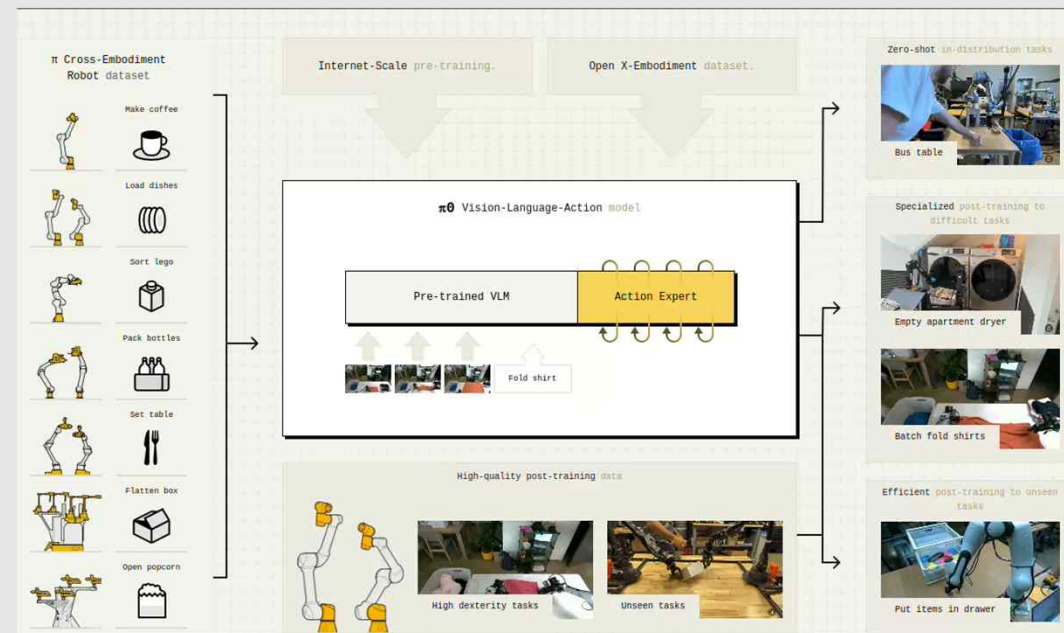
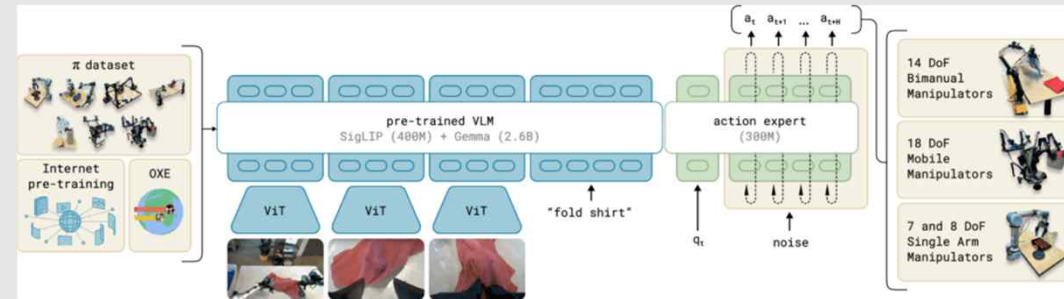
1. 비전·언어·로봇 상태 토큰 기반
2. **연속형 액션 시퀀스(H=50)** 생성 → 부드러운 모션
3. 디퓨전 기반 점진적 노이즈 제거로 정밀 제어

4. 기존 VLA와 차별점

1. **연속형 액션 토큰나이제이션** → 고주파 제어(50Hz) 가능
2. 고난이도 작업에서 높은 효율성과 적응성

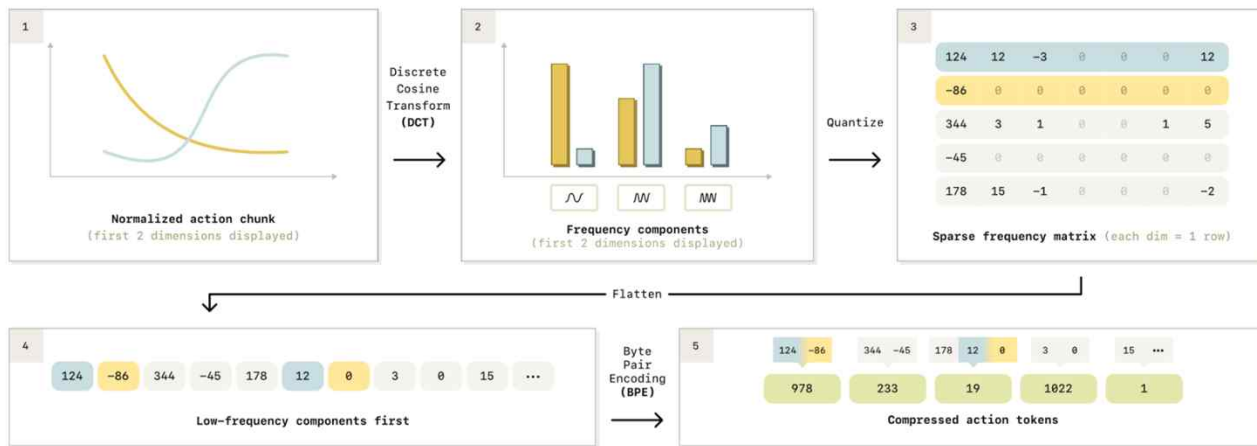
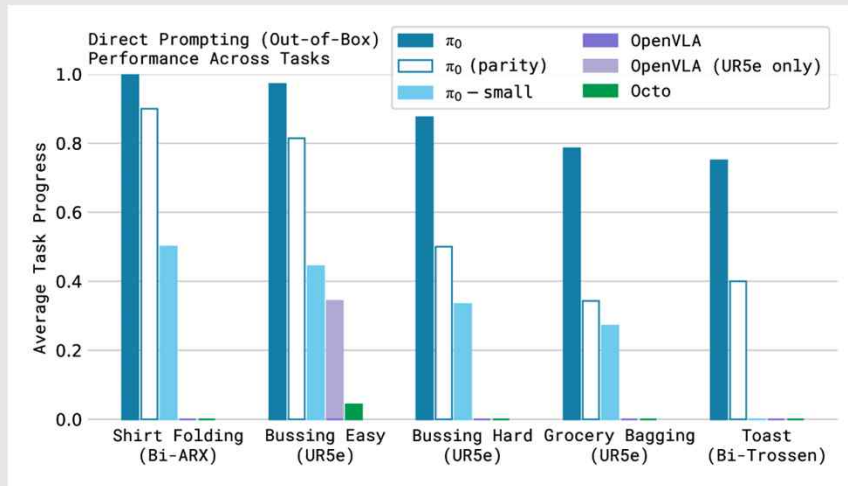
5. π_0 -FAST 확장판

1. **DCT 기반 시퀀스 압축** (저주파/고주파 분리)
2. 오토리그래시브 방식, 효율·속도 향상
3. 불필요한 고주파 성분 제거, 핵심 동작 정보 유지



Physical Intelligence

- 소형 버전도 쉬운 과업부터 어려운 과업까지 **성능 우위를 확보**
- **DCT** : 연속된 행동 토큰을 시간 영역에서 주파수 영역으로 변환
 - 저주파 성분(느리고 부드러운 변화)
 - 고주파 성분(빠르고 급격한 변화)
- 실제 환경 : **저주파 성분이 주로 의미 있는 정보**
 - 고주파 성분은 무시하거나 추가 압축 가능
- **π_0 -FAST**
 - π_0 의 확장 버전
 - 자기회귀 방식과 **FAST** 방식
 - 행동 시퀀스를 주파수 영역에서 토크나이징하여 중복을 줄임
 - > 효율성과 정밀도 향상



OpenVLA 대비 Pi0의 성능 우위 요인

🔑 OpenVLA 대비 Pi0의 성능 우위 요인

1. 사전학습 규모 차이 (VLM)

- OpenVLA: Open X-Embodiment(약 100만 에피소드) 중심.
- Pi0: 더 방대한 영상+텍스트 데이터로 VLM을 사전학습.
→ 시각-언어 표현력이 강화되어 더 다양한 환경에서 일반화 가능.

2. VLA 사전학습 데이터 품질

- OpenVLA: 범용 데이터셋 기반.
- Pi0: 10,000시간 규모의 정교한 로봇 시연 데이터 추가.
→ 단순히 양뿐만 아니라 고품질 trajectory 학습을 통해 정확성·안정성 향상.

3. 액션 표현 방식

- OpenVLA: Discrete action tokens (7차원 심볼릭 토큰) 기반 autoregressive 예측.
- Pi0: Flow Matching 기반 연속 action trajectory 생성.
 - Action-level 예측보다 연속적·세밀한 제어 가능.
 - 노이즈 제거 + smooth trajectory 보장 → 저수준 동작에서 안정적 성능 확보.
→ Low-level 제어 품질이 크게 향상.

4. 경량화 vs 성능

- 모델 크기는 OpenVLA(수십억 파라미터)보다 작음.
- 그러나 데이터 품질 + 학습 기법(Flow Matching) → 효율성 대비 성능 역전.
- 즉, "작지만 더 똑똑한 정책"이 됨.

📊 결론

- 사용자가 정리하신 대로
 1. 더 많은 VLM 사전학습 데이터,
 2. 정제된 10,000시간 로봇 데이터,
 3. Flow Matching 기반 저수준 제어이 세 가지가 OpenVLA보다 Pi0이 우수한 핵심 이유입니다.

Pi0 특징 요약

1. Physical Intelligence 설립 배경

- 로봇 애플리케이션마다 새로운 회사·연구실 수준의 노력이 필요 → 범용 로봇틱 파운데이션 모델 필요성 대두.
- LLM과 VLM처럼 범용적 지식 모델을 로봇에 적용 → “SF에서 보던 범용 로봇”의 기초 마련.
- 단기보다는 장기적 비전. 하지만 최근 수년간 필요한 기술적 요소들이 준비되었음.

2. 로봇틱 학습의 핵심 도전 과제

1. 데이터 부족

- 인터넷에는 이미지·텍스트 데이터는 풍부하지만, 로봇 데이터는 없음.
- 로봇 학습을 위해서는 직접 데이터 수집 필요 → 비용·노력 커짐.

2. 일반화와 상식 부족

- 로봇은 실제 환경에서 예상치 못한 상황 처리 필요.
- VLM을 통해 언어·시각 기반 상식을 주입 가능.

3. 성능·신뢰성

- 강화학습(RL) 발전이 정밀성·성능 향상에 기여.

3. Reinforcement Learning(강화학습)과 파운데이션 모델

- $\pi 0$ 자체에는 RL이 핵심 요소는 아님 → “초기 단계 모델”으로 위치.
- LLM이 먼저 사전학습·SFT·RLHF 단계를 밟은 것처럼, 로봇도 나중에 RL이 통합될 가능성 큼.
- RL은 특히 테스트 시점 추론(test-time reasoning), 세밀한 행동 최적화에 유용할 것.

4. $\pi 0$ 모델 아키텍처

- 기본 구조: VLM(Gemma 기반) + Action Expert.
- 문제: VLM은 이미지·텍스트 이해/생성은 잘하지만, 정밀한 연속 동작 표현은 어렵다.
- 해결: Diffusion 기반 Action Expert를 붙여, 연속 trajectory(action chunks)를 생성.
- 결과: 빨래 개기 등 복잡한 작업에서 안정적 성능. ---

5. 데이터 Recipe (Pre-training & Post-training)

• Pre-training (약 10,000시간)

- 인간 텔레오퍼레이션을 통해 수집한 다양한 과제 데이터.
- 실패·실수 데이터를 포함 → 로봇이 오류에서 복구하는 능력 학습.
- OXE(Open X-Embodiment) 데이터도 포함.

• Post-training (2~20시간, 소규모)

- 고품질·일관된 데이터, 특정 전문가가 시연한 전략 포함.
- 과제를 가장 잘 수행하는 방식을 학습.

- 교훈: 좋은 데이터만 있으면 오히려 일반화에 약점. 다양한 품질 데이터가 필요.

6. 데모 사례 – 빨래 개기

- 과제별 코드 변경 없음 → 같은 모델이 박스 조립, 테이블 청소에도 적용.
- 전략은 포스트 트레이닝 데이터로 학습.
- 예측 불가 상황은 프리 트레이닝 데이터에서 복구 능력을 습득.
- 실제 실험 중, 연구원이 셔츠를 방해 요소로 던지자 로봇이 스스로 치워 버리는 emergent behavior 관찰. ---

Pi0 특징 요약

7. FAST (토크나이제이션 혁신)

- 기존 VLA는 액션을 단순 숫자 토큰으로 표현 → 비효율적.
- FAST**: 주파수 도메인 압축(JPEG 유사) → 액션 토큰 균등화.
- 효과:
 - 학습 속도 **4배** 향상.
 - 동일 성능을 더 적은 토큰으로 구현.
 - 언어 지시 일반화 성능 개선.
- Hugging Face 공개, 누구나 적용 가능.

8. Open Sourcing (오픈소스 공개)

- 제공 내용:
 - 기본 체크포인트,
 - FAST** 버전 체크포인트 + 토크나이저,
 - 파인튜닝 코드,
 - 예시 모델 (DROID, Aloha).
- 주요 목적: 연구자·커뮤니티가 직접 데이터 수집 → 자신의 로봇에 파인 튜닝.
- 기대 효과: 창의적 실험 유도 → 새로운 레시피·데이터 수집 방식 발견.

9. 지원 로봇 유형

- 팔 기반 로봇(단일/이중 팔), 휴머노이드, 모바일 플랫폼 지원.
- 32차원** 이하 액션 공간이면 대부분 적용 가능.
- 실제 사례: 다섯 손가락 로봇 손, Franka 팔, 저가형 로봇 팔 등.
- Aloha 실험 장비: 약 \$20K, 팔 하나당 \$6K.
- 하드웨어 비용 지속 감소 → **향후 대중화** 가능성 큼.

9. 지원 로봇 유형

- 팔 기반 로봇(단일/이중 팔), 휴머노이드, 모바일 플랫폼 지원.
- 32차원** 이하 액션 공간이면 대부분 적용 가능.
- 실제 사례: 다섯 손가락 로봇 손, Franka 팔, 저가형 로봇 팔 등.
- Aloha 실험 장비: 약 \$20K, 팔 하나당 \$6K.
- 하드웨어 비용 지속 감소 → **향후 대중화** 가능성 큼.

10. 향후 연구 방향

- 복잡한 지시사항 이해
 - 단순 과제 지시("셔츠 접어") → 복합 지시("파티 준비, 접시는 두고 쓰레기만 치워라") 가능하게.
 - LLM처럼 상세 태스크 설명 기반 실행 목표.
- 기술 재활용·응용
 - 새로운 과제를 기존 기술 조합으로 해결하는 능력.
- 일반화 성능 확장
 - 과제(Task), 환경(Environment), 객체(Object), 플랫폼(Platform), 지시사항(Instruction) 전반에 걸친 일반화.
 - 목표: "아하 모멘트(aha moment)"를 더 자주 제공.
 - 예: 모델을 다른 대학의 로봇에 올려 실행 → 바로 동작하는 경험.

✅ 결론:

π_0 는 아직 초기 단계지만,

- Diffusion** 기반 **Action Expert**,
- Pre-training + Post-training** 레시피,
- FAST** 토크나이제이션,
- 오픈소스 공개

PI-Zero

Pi-Zero

Physical Intelligence

1. 모델 개요

1. π_0 : 범용 로봇 제어를 위한 VLA + Diffusion Flow Model
2. 백본: Paligemma VLM (~3B) + 액션 전문가(300M)
3. 학습 데이터: π Cross-Embodiment Dataset(다양한 로봇·환경)

2. VLM 구성

1. SigLIP(이미지 인코더) + Gemma(언어 명령)
2. 비전·언어 입력 처리, 액션 토큰 미포함 → 시각 이해 명확화

3. Action Expert (Flow Matching)

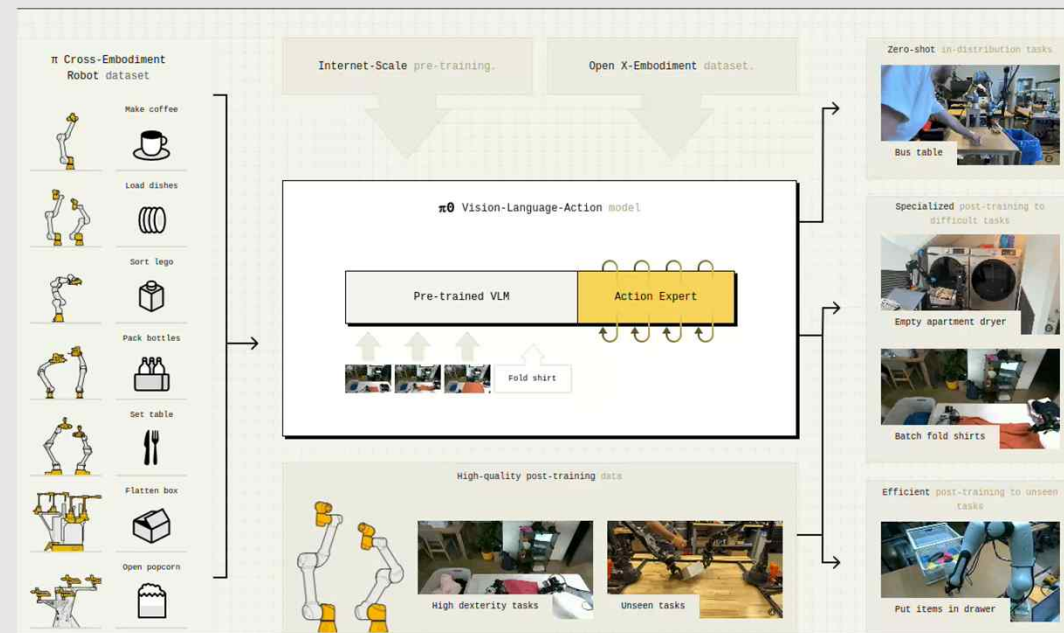
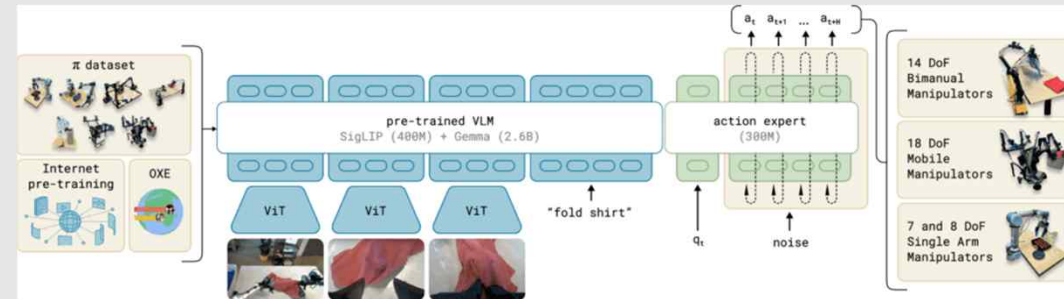
1. 비전·언어·로봇 상태 토큰 기반
2. 연속형 액션 시퀀스($H=50$) 생성 → 부드러운 모션
3. 디퓨전 기반 점진적 노이즈 제거로 정밀 제어

4. 기존 VLA와 차별점

1. 연속형 액션 토큰나이제이션 → 고주파 제어(50Hz) 가능
2. 고난이도 작업에서 높은 효율성과 적응성

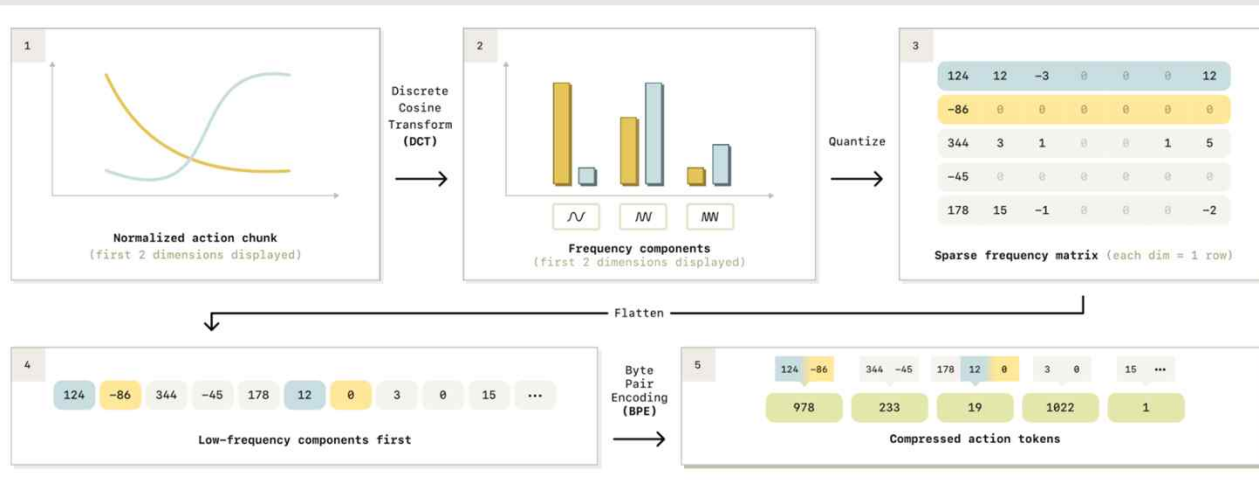
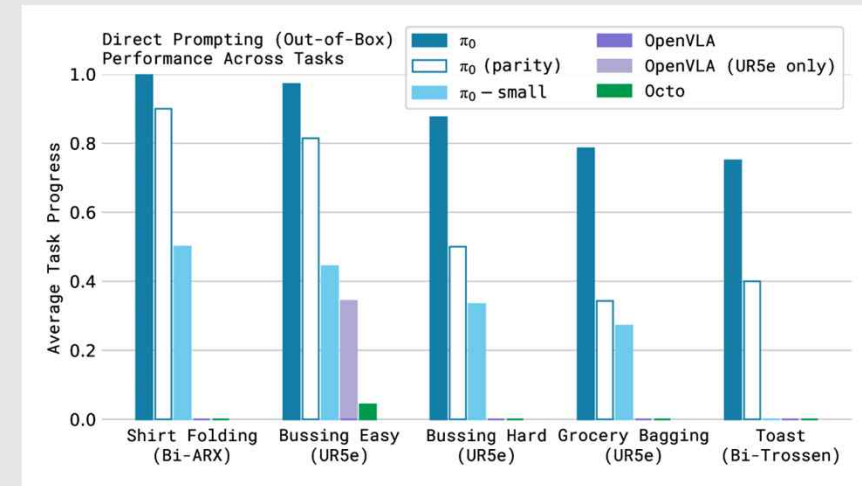
5. π_0 -FAST 확장판

1. DCT 기반 시퀀스 압축 (저주파/고주파 분리)
2. 오토리그래시브 방식, 효율·속도 향상
3. 불필요한 고주파 성분 제거, 핵심 동작 정보 유지



Physical Intelligence

- 소형 버전도 쉬운 과업부터 어려운 과업까지 **성능 우위를 확보**
- **DCT** : 연속된 행동 토큰을 시간 영역에서 주파수 영역으로 변환
 - 저주파 성분(느리고 부드러운 변화)
 - 고주파 성분(빠르고 급격한 변화)
- 실제 환경 : **저주파 성분이 주로 의미 있는 정보**
 - 고주파 성분은 무시하거나 추가 압축 가능
- **π_0 -FAST**
 - π_0 의 확장 버전
 - 자기회귀 방식과 **FAST** 방식
 - 행동 시퀀스를 주파수 영역에서 토크나이징하여 중복을 줄임
 - > 효율성과 정밀도 향상



OpenVLA 대비 Pi0의 성능 우위 요인

🔑 OpenVLA 대비 Pi0의 성능 우위 요인

1. 사전학습 규모 차이 (VLM)

- OpenVLA: Open X-Embodiment(약 100만 에피소드) 중심.
- Pi0: 더 방대한 영상+텍스트 데이터로 VLM을 사전학습.
→ 시각-언어 표현력이 강화되어 더 다양한 환경에서 일반화 가능.

2. VLA 사전학습 데이터 품질

- OpenVLA: 범용 데이터셋 기반.
- Pi0: 10,000시간 규모의 정교한 로봇 시연 데이터 추가.
→ 단순히 양뿐만 아니라 고품질 trajectory 학습을 통해 정확성, 안정성 향상.

3. 액션 표현 방식

- OpenVLA: Discrete action tokens (7차원 심볼릭 토큰) 기반 autoregressive 예측.
- Pi0: Flow Matching 기반 연속 action trajectory 생성.
 - Action-level 예측보다 연속적·세밀한 제어 가능.
 - 노이즈 제거 + smooth trajectory 보장 → 저수준 동작에서 안정적 성능 확보.
→ Low-level 제어 품질이 크게 향상.

4. 경량화 vs 성능

- 모델 크기는 OpenVLA(수십억 파라미터)보다 작음.
- 그러나 데이터 품질 + 학습 기법(Flow Matching) → 효율성 대비 성능 역전.
- 즉, "작지만 더 똑똑한 정책"이 됨.

📊 결론

- 사용자가 정리하신 대로
 1. 더 많은 VLM 사전학습 데이터,
 2. 정제된 10,000시간 로봇 데이터,
 3. Flow Matching 기반 저수준 제어이 세 가지가 OpenVLA보다 Pi0이 우수한 핵심 이유입니다.

Pi0 특징 요약

1. Physical Intelligence 설립 배경

- 로봇 애플리케이션마다 새로운 회사·연구실 수준의 노력이 필요 → 범용 로봇틱 파운데이션 모델 필요성 대두.
- LLM과 VLM처럼 범용적 지식 모델을 로봇에 적용 → “SF에서 보던 범용 로봇”의 기초 마련.
- 단기보다는 장기적 비전. 하지만 최근 수년간 필요한 기술적 요소들이 준비되었음.

2. 로봇틱 학습의 핵심 도전 과제

1. 데이터 부족

- 인터넷에는 이미지·텍스트 데이터는 풍부하지만, 로봇 데이터는 없음.
- 로봇 학습을 위해서는 직접 데이터 수집 필요 → 비용·노력 커짐.

2. 일반화와 상식 부족

- 로봇은 실제 환경에서 예상치 못한 상황 처리 필요.
- VLM을 통해 언어·시각 기반 상식을 주입 가능.

3. 성능·신뢰성

- 강화학습(RL) 발전이 정밀성·성능 향상에 기여.

3. Reinforcement Learning(강화학습)과 파운데이션 모델

- $\pi 0$ 자체에는 RL이 핵심 요소는 아님 → “초기 단계 모델”으로 위치.
- LLM이 먼저 사전학습·SFT·RLHF 단계를 밟은 것처럼, 로봇도 나중에 RL이 통합될 가능성 큼.
- RL은 특히 테스트 시점 추론(test-time reasoning), 세밀한 행동 최적화에 유용할 것.

4. $\pi 0$ 모델 아키텍처

- 기본 구조: VLM(Gemma 기반) + Action Expert.
- 문제: VLM은 이미지·텍스트 이해/생성은 잘하지만, 정밀한 연속 동작 표현은 어렵다.
- 해결: Diffusion 기반 Action Expert를 붙여, 연속 trajectory(action chunks)를 생성.
- 결과: 빨래 개기 등 복잡한 작업에서 안정적 성능. ---

5. 데이터 Recipe (Pre-training & Post-training)

• Pre-training (약 10,000시간)

- 인간 텔레오퍼레이션을 통해 수집한 다양한 과제 데이터.
- 실패·실수 데이터를 포함 → 로봇이 오류에서 복구하는 능력 학습.
- OXE(Open X-Embodiment) 데이터도 포함.

• Post-training (2~20시간, 소규모)

- 고품질·일관된 데이터, 특정 전문가가 시연한 전략 포함.
- 과제를 가장 잘 수행하는 방식을 학습.

- 교훈: 좋은 데이터만 있으면 오히려 일반화에 약점. 다양한 품질 데이터가 필요.

6. 데모 사례 – 빨래 개기

- 과제별 코드 변경 없음 → 같은 모델이 박스 조립, 테이블 청소에도 적용.
- 전략은 포스트 트레이닝 데이터로 학습.
- 예측 불가 상황은 프리 트레이닝 데이터에서 복구 능력을 습득.
- 실제 실험 중, 연구원이 셔츠를 방해 요소로 던지자 로봇이 스스로 치워 버리는 emergent behavior 관찰. ---

Pi0 특징 요약

7. FAST (토크나이제이션 혁신)

- 기존 VLA는 액션을 단순 숫자 토큰으로 표현 → 비효율적.
- **FAST**: 주파수 도메인 압축(JPEG 유사) → 액션 토큰 균등화.
- 효과:
 - 학습 속도 **4배** 향상.
 - 동일 성능을 더 적은 토큰으로 구현.
 - 언어 지시 일반화 성능 개선.
- Hugging Face 공개, 누구나 적용 가능.

8. Open Sourcing (오픈소스 공개)

- 제공 내용:
 - 기본 체크포인트,
 - **FAST** 버전 체크포인트 + 토크나이저,
 - 파인튜닝 코드,
 - 예시 모델 (DROID, Aloha).
- 주요 목적: 연구자·커뮤니티가 직접 데이터 수집 → 자신의 로봇에 파인 튜닝.
- 기대 효과: 창의적 실험 유도 → 새로운 레시피·데이터 수집 방식 발견.

9. 지원 로봇 유형

- 팔 기반 로봇(단일/이중 팔), 휴머노이드, 모바일 플랫폼 지원.
- 32차원 이하 액션 공간이면 대부분 적용 가능.
- 실제 사례: 다섯 손가락 로봇 손, Franka 팔, 저가형 로봇 팔 등.
- Aloha 실험 장비: 약 \$20K, 팔 하나당 \$6K.
- 하드웨어 비용 지속 감소 → **향후 대중화** 가능성 큼.

9. 지원 로봇 유형

- 팔 기반 로봇(단일/이중 팔), 휴머노이드, 모바일 플랫폼 지원.
- 32차원 이하 액션 공간이면 대부분 적용 가능.
- 실제 사례: 다섯 손가락 로봇 손, Franka 팔, 저가형 로봇 팔 등.
- Aloha 실험 장비: 약 \$20K, 팔 하나당 \$6K.
- 하드웨어 비용 지속 감소 → **향후 대중화** 가능성 큼.

10. 향후 연구 방향

1. 복잡한 지시사항 이해
 - 단순 과제 지시("셔츠 접어") → 복합 지시("파티 준비, 접시는 두고 쓰레기만 치워라") 가능하게.
 - LLM처럼 상세 태스크 설명 기반 실행 목표.
2. 기술 재 활용·응용
 - 새로운 과제를 기존 기술 조합으로 해결하는 능력.
3. 일반화 성능 확장
 - 과제(Task), 환경(Environment), 객체(Object), 플랫폼(Platform), 지시사항(Instruction) 전반에 걸친 일반화.
 - 목표: "아하 모멘트(aha moment)"를 더 자주 제공.
 - 예: 모델을 다른 대학의 로봇에 올려 실행 → 바로 동작하는 경험.

✅ 결론:

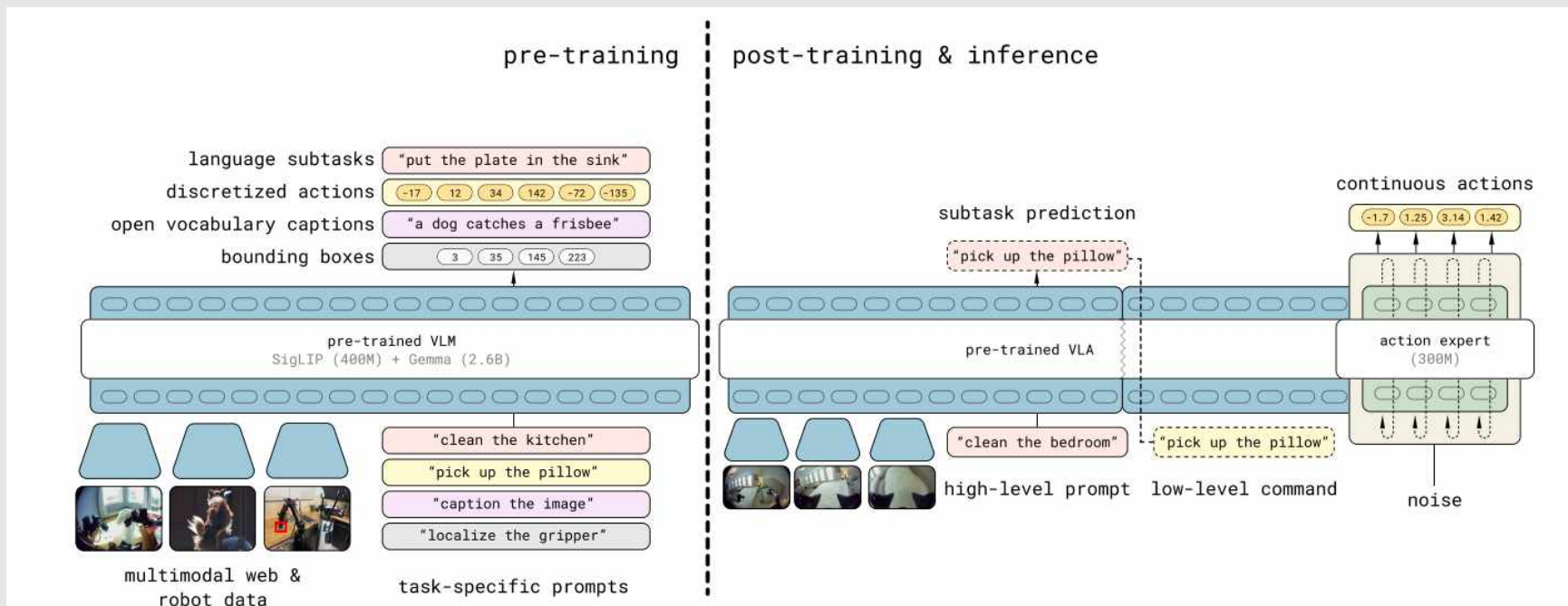
$\pi 0$ 는 아직 초기 단계지만,

- **Diffusion** 기반 **Action Expert**,
- **Pre-training + Post-training** 레시피,
- **FAST** 토크나이제이션,
- 오픈소스 공개

$\pi_{0.5}$: a VLA with Open-World Generalization

개요

- $\pi_{0.5}$: VLA 기반 로봇 지능 모델
새로운 가정 환경에서도 일반화된 행동 수행
- 기존 π_0 모델 확장
이동 조작, 언어 명령 수행,
다양한 데이터 소스로부터의 공동 학습



$\pi_{0.5}$: a VLA with Open-World Generalization

1. 이중 제어 체계 (High- & Low-Level Control)

- **고수준 제어**: 자연어 명령 이해 및 추론
(예: "접시를 싱크대에 넣어")
- **저수준 제어**: 실제 관절 제어 동작 생성
(50단계 연속 명령)
- 동일한 모델이 두 기능을 "**Chain-of-Thought**" 방식으로 수행



2. Co-Training 기반 학습 전략

- 웹 데이터(캡셔닝, QA),
- 고정형 로봇의 다환경 시연 데이터(ME),
- 크로스 엠보디먼트 데이터(CE)
- 다양한 데이터 조합을 통해 **범용성 강화**

3. 다양한 환경에서의 일반화 (Generalization)

- 학습에 포함되지 않은 **완전히 새로운 가정에서** 로봇 투입
- 접시 정리, 침대 정돈, 바닥 청소 등 복잡한 장기 과업 수행
- 약 100개의 환경만으로 **테스트 환경 학습 모델에** 근접한 성능 달성

GR00NT N1

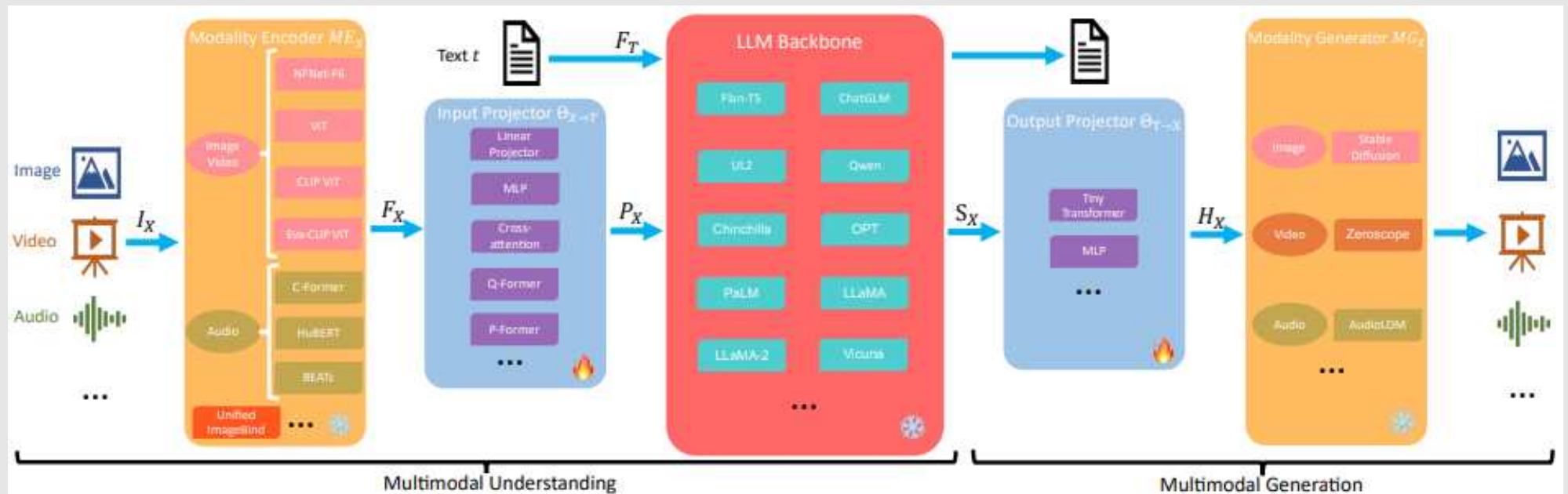
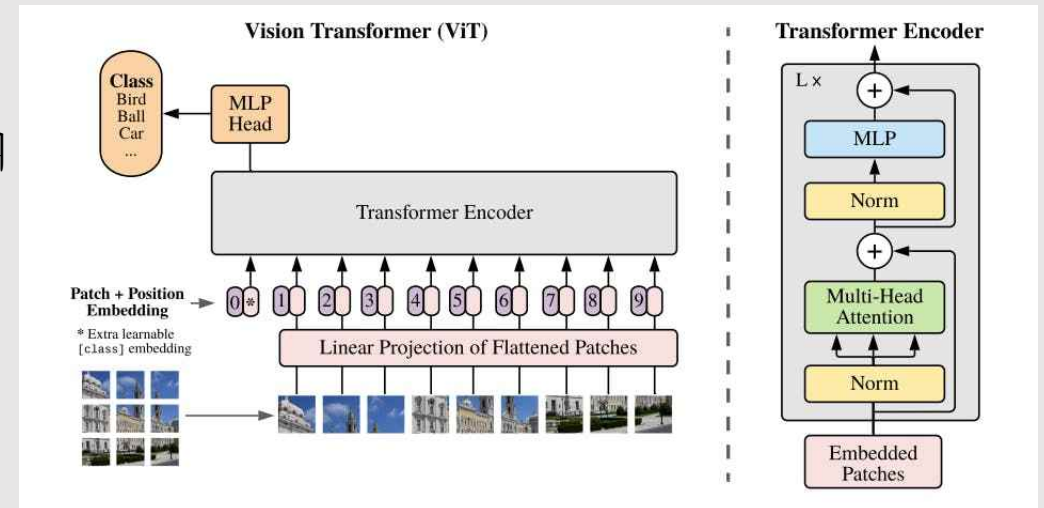
GR00NT N1

Multimodal LLM

텍스트뿐만 아니라 이미지, 비디오, 오디오, 3D 등 다양한 형식의 데이터를 이해하고 생성할 수 있는 LLM

- 텍스트와 이미지 처리 활발
- 비디오, 오디오, 3D는 초기 단계

- Image encoder : ViT/ResNet/DINO



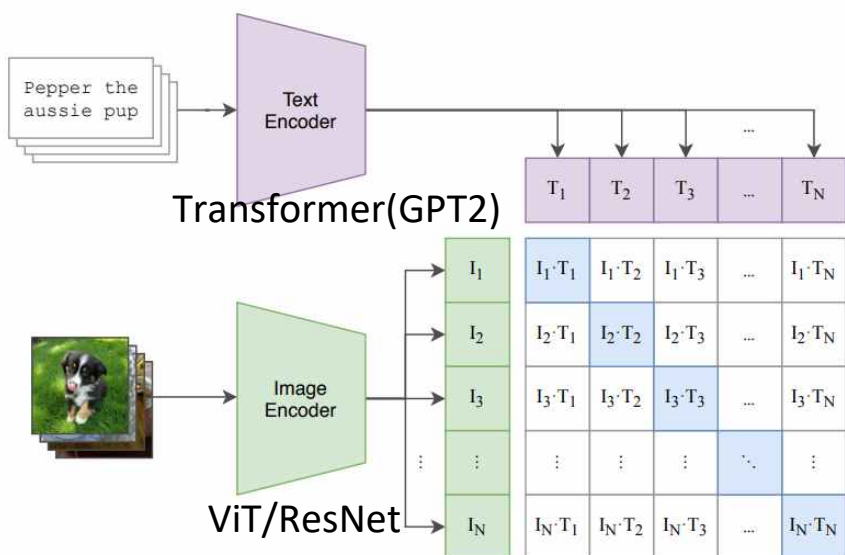
CLIP(Contrastive Language-Image Pretraining)

- CLIP은 OpenAI에서 2021년에 공개한 모델
- 일반 모델: 이미지 → 특징 추출 + 선형 분류기 → 레이블 예측
- CLIP: 이미지 인코더 + 텍스트 인코더 → (이미지, 텍스트) 매칭 학습
- 테스트 시: 텍스트 인코더로 클래스 설명을 임베딩 → 제로샷 분류 가능
즉, CLIP은 텍스트를 활용해 새로운 클래스에 대해 학습 없이도 분류가 가능하다.

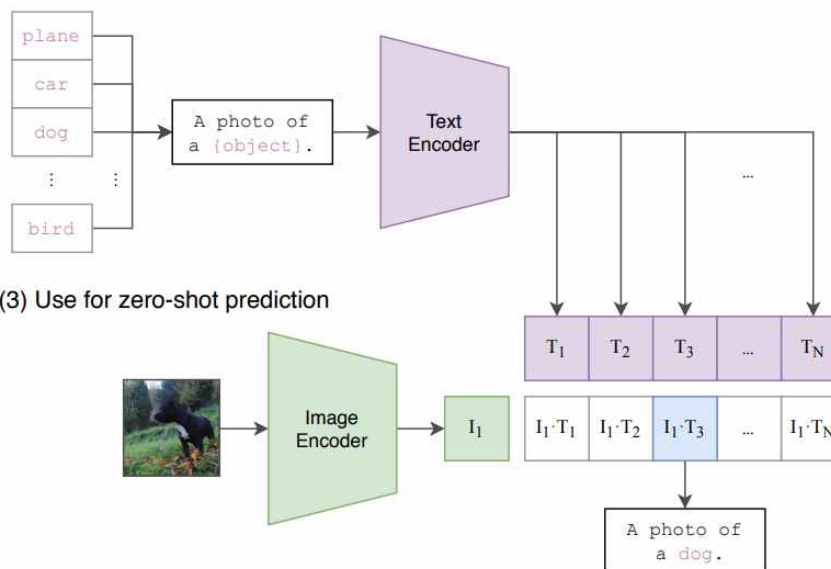
SigLIP(Google, 2023)은 CLIP의 대안
Sigmoid loss를 사용해 학습 안정성과 성능 개선

더 간단한 학습 구조로 CLIP보다 효율적
대규모 데이터에서 강력한 성능을 보임

(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

CLIP: SoftMax

$$-\frac{1}{2|B|} \sum_{i=1}^{|B|} \left(\log \frac{e^{t\mathbf{x}_i \cdot \mathbf{y}_i}}{\sum_{j=1}^{|B|} e^{t\mathbf{x}_i \cdot \mathbf{y}_j}} + \log \frac{e^{t\mathbf{x}_i \cdot \mathbf{y}_i}}{\sum_{j=1}^{|B|} e^{t\mathbf{x}_j \cdot \mathbf{y}_i}} \right)$$

SigLIP: Sigmoid

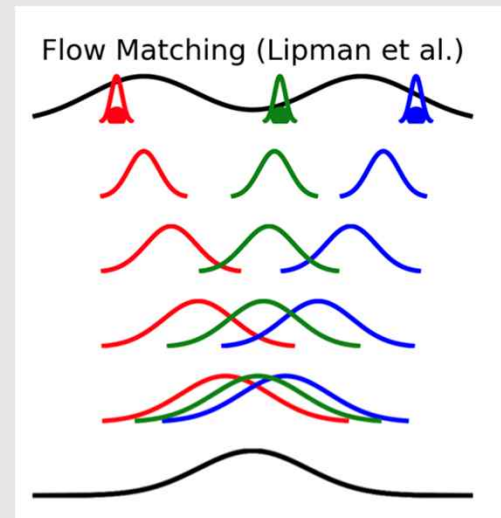
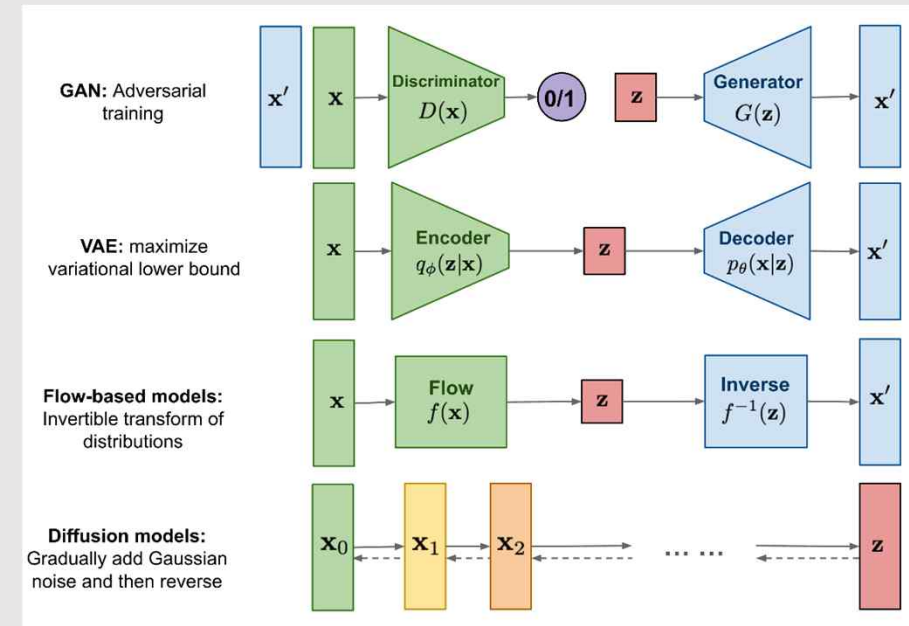
$$-\frac{1}{|B|} \sum_{i=1}^{|B|} \sum_{j=1}^{|B|} \log \frac{1}{1 + e^{z_{ij}(-t\mathbf{x}_i \cdot \mathbf{y}_j + b)}} \mathcal{L}_{ij}$$

VLA - Flow matching

- 개념
 - 분포 간 변환을 연속적 ODE(벡터장)로 학습
 - 초기 분포(노이즈) → 목표 분포(데이터)까지 연속적 경로 이동
- 동작 원리
 1. 벡터장 $v(x,t)$: 시간·위치별 방향과 속도 정의
 2. 미분값으로 최적 방향·크기 추정
 3. 한 스텝 적분 → 갱신
 4. 반복: 노이즈 → 데이터 도달
- 특징 (Diffusion·Flow 대비)
 - Diffusion : 점진적 변환 + Flow: invertible mapping
 - 빠른 샘플링 + 표현력 우수
 - Deterministic

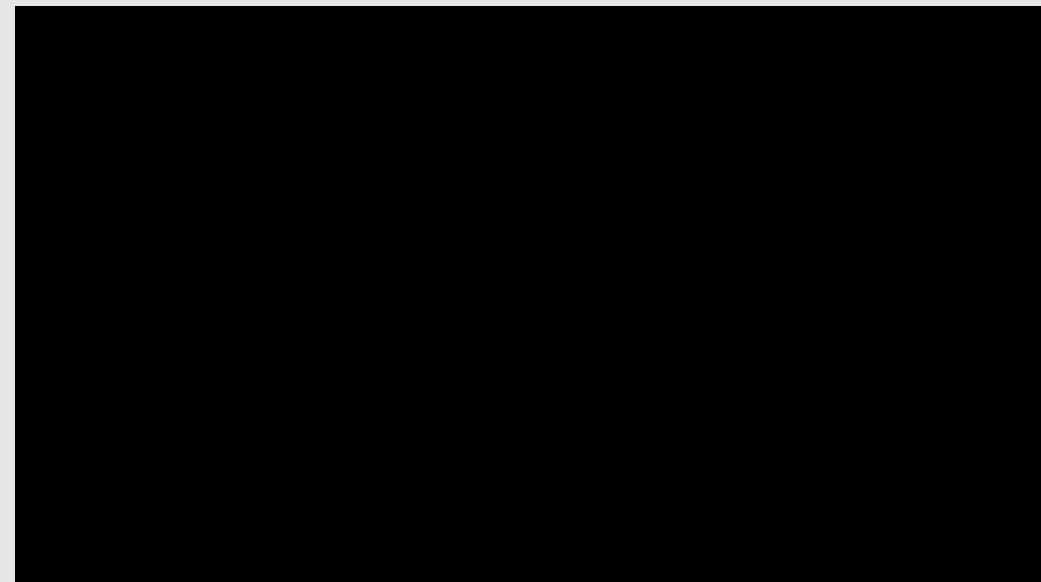
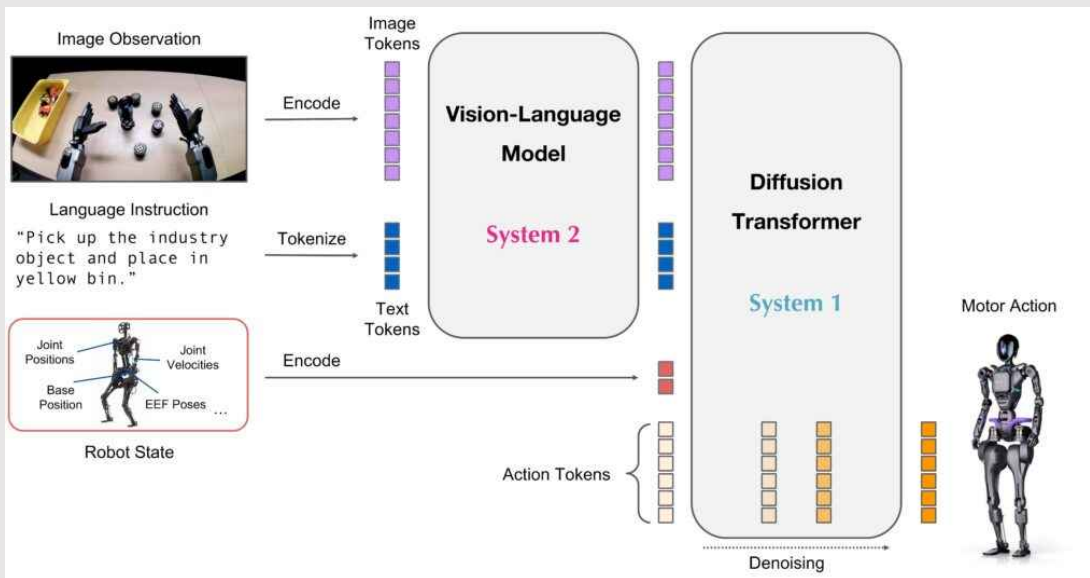
Flow Matching

- '시간에 따른 벡터장을 따라 노이즈에서 데이터로 이동하는 생성 모델'



엔비디아 GROOT N1

- 모델 유형: 개방형(Open) 범용 휴머노이드 추론·제어 모델
- 파라미터 규모: 2B (eagle2_hg_model 백본)
- 학습 데이터: Omniverse·Cosmos 시뮬레이션 데이터 + 실제 휴머노이드 로봇 데이터
- 적응성: 사전 학습된 정책을 다양한 로봇 시스템에 이식 가능 (Cross-Embodied)
- 토큰화 입력: 관측(Observation), 언어 지시(Language Instruction), 로봇 상태(Robot State)
- 이중 시스템 구조:
 - System 2: VLM 기반 고수준 계획(물리 세계 이해·추론)
 - System 1: Diffusion Transformer 기반 저수준 제어(120Hz 모터 동작)



인트로

NVIDIA의 최신 휴머노이드 로봇 논문

GR00T N1: Open Foundation Model for Generalist Humanoid Robots

리뷰

1. GR00T N1은 단일 모델로 다양한 로봇을 제어하고, 시각·언어·행동을 통합적으로 이해해 실시간 행동을 생성할 수 있는 범용 지능 로봇 모델.
 2. 리뷰에서는 GR00T N1의 구조, 데이터 전략, 기존 연구와의 차별점을 정리할 예정.
 3. 인간처럼 일하고 상황에 유연하게 대처하는 제네럴리스트 로봇이 궁극적 목표이며, 최근 파운데이션 모델 트렌드가 로봇 분야로 확산되고 있음을 강조.
- 👉 즉, “인류의 오랜 꿈인 범용 휴머노이드 로봇에 한 걸음 더 다가선 NVIDIA의 최신 연구를 리뷰한다”는 맥락이에요.

GR00T N1의 간단한 소개

GR00T N1은 NVIDIA가 공개한 범용 휴머노이드 로봇용 멀티모달 파운데이션 모델입니다. 핵심 내용은 다음과 같습니다:

1. 단순 로봇 제어가 아니라 **비전·언어·액션**을 모두 다루는 멀티모달 모델.
2. 다양한 로봇 데이터와 인간 영상을 통합 학습하여 범용성을 극대화.
3. 디퓨전 트랜스포머 기반 동작 생성 시스템으로 부드럽고 안정적인 실시간 페루프 제어 구현.
4. 기존 스크립트/시나리오 한계를 넘어, **언어 지시 → 상황 이해 → 행동 계획 및 실행** 가능.
5. 하드웨어·모델·데이터를 아우르는 **완전한 통합 구조**를 제시.

👉 요약하자면, GR00T N1은 **현실 세계의 복잡성에 대응할 수 있는 범용 로봇 지능 모델**로, 휴머노이드 로봇에 실제 적용 가능한 수준의 **정밀성과 적응력**을 보여줍니다.

GR00T N1의 핵심 개요 1

1. 입력-출력 구조

1. 입력: 이미지 + 언어 명령
2. 출력: 실제 로봇 모터 제어 명령(행동)
3. 따라서 GR00T N1은 **VLA(비전-언어-액션) 모델**

2. 이중 시스템 구조

1. **시스템 2 (생각하는 뇌)**: 환경 인식 + 언어 이해 (NVIDIA Eagle-2 VLM 탑재, 10Hz 속도)
2. **시스템 1 (반응하는 뇌)**: 실제 행동 생성 (Diffusion Transformer, DT 활용, 120Hz 속도)

3. 연결 방식

1. 시스템 2가 이미지/텍스트를 **토큰 시퀀스**로 인코딩
2. 시스템 1은 **스테이트 임베딩(로봇 상태)** + ****노이즈 액션 토큰(초기 무작위 동작)****을 입력받아
3. **셀프 어텐션**으로 내부 정보 교환
4. **크로스 어텐션**으로 시스템 2의 VLM 토큰과 결합 → 최종 모터 명령 생성

4. 핵심 아이디어

1. **Diffusion 기반 액션 생성**: 랜덤 노이즈에서 점진적으로 실제 행동으로 복원
2. **시스템 1 & 2 협업**: 상황 이해 + 실시간 반응을 분리하면서도 트랜스포머 구조로 통합

👉 한마디로, GR00T N1은 ****“언어와 시각을 이해(시스템 2) → 실시간 행동으로 변환(시스템 1)”****하는 두뇌 구조를 가진 **범용 휴머노이드 제어 모델**입니다.

GR00T N1의 핵심 개요 2

GR00T N1의 가장 큰 특징과 강점은 다음과 같습니다:

1. 엔드투엔드 학습

1. 이미지·텍스트·로봇 상태 → 모터 명령까지 **하나의 모델**로 통합 학습.
2. 시스템 1(액션 생성)과 시스템 2(언어·시각 이해)를 따로 훈련하지 않고 **공동 학습**.
3. 결과적으로 상황에 맞춰 **판단·반응이 가능한 지능형 로봇** 구현.

2. 크로스 인바디먼트 지원

1. 단일 모델·단일 가중치로 **여러 형태의 로봇 제어 가능** (단일팔, 양팔, 휴머노이드 등).
2. 각 로봇의 상태·행동을 자기 전용 인코더 → **공통 임베딩 공간**으로 변환.

3. 레이턴트 액션 스페이스

1. 겉모습이 달라도 “의도(meaning)”가 같으면 **동일한 벡터로 표현**.
2. 예: 팔·손·다양한 방식으로 “컵 옮기기”를 수행해도 → 같은 레이턴트 액션으로 학습.
3. 로봇·인간·시뮬레이션 간 **행동 언어를 통일**.

👉 요약하면, GR00T N1은 **엔드투엔드 학습 + 크로스 인바디먼트 + 의미 기반 레이턴트 액션**을 통해, 단 하나의 모델로 여러 로봇이 **일관된 방식으로 다양한 행동을 수행**할 수 있도록 만든 범용 로봇 지능입니다.

GR00TN1의 학습 데이터 전략: 데이터 피라미드

1. 데이터 피라미드 구조 (3단계)

1. **하층**: 대규모 웹·인간 영상·시뮬레이션 영상 (양 많음, 행동 정보 없음 → 레이턴트 액션 추정기로 행동 추정)
2. **중간층**: 시뮬레이터 생성 로봇 궤적 (행동 정보 정확, 양 많음, 현실감 부족)
3. **상층**: 실제 로봇 궤적 데이터 (품질 최고, 수집 어렵고 비용 큼)

2. 레이턴트 액션 스페이스 활용

1. 행동의 ****의도(meaning)****를 공통 벡터로 표현 → 사람, 로봇, 시뮬레이션 간 동일한 행동 언어로 통합.
2. 예: “컵을 집는다” → 사람은 손동작, 로봇은 관절 궤적, 시물은 수치 데이터 → 같은 레이턴트 벡터로 변환.

3. 효과

1. 실제 로봇 데이터를 적게 써도 학습 가능.
2. 다양한 출처의 데이터를 하나의 표현 공간으로 묶어 **엔드투엔드 학습** 가능.
3. 결과적으로 **유연하면서도 일반화가 잘 되는 범용 로봇 지능** 확보.

👉 한 줄로 요약하면: ****“데이터 양·품질을 피라미드 구조로 나누고, 레이턴트 액션 스페이스로 의미를 통합해 적은 실제 데이터로도 범용 학습이 가능하다”****입니다.

GR00T N1의 모델 구성과 학습 방식 : System 1, System 2 학습

****GR00T N1의 모델 구성과 학습 방식(System 1 & 2)**은 다음과 같습니다:**

1. 이중 시스템 구조

1. **시스템 2 (VLM, Eagle-2):** 이미지+텍스트 입력 → 의미 토큰(“컵, 집기, 이동, 상자”)으로 변환.
2. **시스템 1 (Diffusion Transformer, DT):** 의미 토큰 + 로봇 상태 + 노이즈 액션 토큰 → 점진적으로 복원 → 최종 모터 명령 생성.

2. 학습 과정 (3단계)

1. **VQ-VAE 학습:** 다양한 로봇 궤적으로부터 레이턴트 액션 토큰(z_τ) 생성.
2. **IDM 학습:** 행동 정보 없는 영상도 이전·이후 프레임을 보고 행동 추정(z_τ).
3. **전체 엔드투엔드 학습:** 시스템 2 의미 토큰 + 로봇 상태 + 노이즈 z_τ → DT가 행동 복원 → 모터 명령으로 학습.

3. 실험 결과

1. 단일팔, 양팔, 휴머노이드 등 다양한 로봇(141개 테스트)에서 기존 방식 대비 **더 높은 성공률** 달성.
2. 단일 로봇 전용 모델보다 **여러 로봇을 함께 학습한 단일 모델이 더 뛰어난 일반화 성능**을 보임.

4. 의의

1. RT-1의 로봇 행동 학습, RT-2의 멀티모달 일반화에 이어 GR00T N1은 **통합적이고 실용적인 지능 구조**를 제시.
2. 인간처럼 보고·이해·행동하는 범용 로봇 실현에 한걸음 더 가까워짐.

👉 **결론:** GR00T N1은 VLM(이해) + Diffusion Transformer(행동)를 통합 엔드투엔드 학습해, 다양한 로봇을 단일 모델로 제어하면서 높은 일반화 성능을 보여준 최신 범용 로봇 지능 모델입니다.

GR00TN1.5

엔비디아 GR00T-Dreams 및 GR00TN1.5 파이프라인

GR00T 비전 (로봇용 파운데이션 모델)

GR00T Dreams 시스템

GR00T Dreams → N1.5 파이프라인 (5단계)

주요 성과 및 능력 향상

<https://developer.nvidia.com/ko-kr/blog/enhance-robot-learning-with-synthetic-trajectory-data-generated-by-world-foundation-models/>

GR00T 비전(로봇용 파운데이션 모델)

목표 : 모든 로봇이 공유할 수 있는 공동 지능

GR00T N1(이전 모델)

- 사람 언어/이미지 이해
- 다양한 로봇 형태에 공동 적용 가능한 행동 생성
- 한계 : 방대한 현실 데이터 부족

GR00T N1.5 (최신 모델)

- N1의 VLA(Vision-Language-Action) 구조유지
- 훨씬 다양한 환경/작업 대응
- 새로운 상황에서 즉석 판단 및 적응 능력

GR00T-Dreams 시스템

개념 : 로봇이 스스로 상상력 발휘

기반 데이터 : 88시간 분량의 실제 시연 데이터

합성 데이터 생성

- AI 비디오 생성 모델 활용
- 827시간 분량의 가상 경험 생성(10배 증폭)
- 예 : 다른 각도, 다른 색, 다른 위치에서 컵 집는 영상 자동 생성

GR00T-Dreams → GR00T N1.5 파이프라인(5 단계)

1단계 : 현실 데이터 수집

- 88시간 분량의 사람 조정 로봇 시연 영상

2단계 : 드림즈 합성 데이터 생성

- AI 비디오 생성 모델 사용
- 827 시간 분량의 새로운 시나리오 확정

3단계 : Chain of Thought 평가 및 필터링

- Cosmos Reason 평가 시스템 사용
- 목표 달성 여부 (예: 선반 위 안착)
- 물리적 가능성(예: 관통 여부)
- 학습 가치 평가(다양성, 신규성)

4단계 : 액션 토큰 추출

- 검증된 비디오를 행동 명령 시퀀스로 변환
- 토큰화(256단계 양자화)
- 추출원리
 - 1. 2D 포즈 추정(관절 위치)
 - 2. 3D 복원(깊이 정보 추출)
 - 3. 역기구학(IK) 계산(3D → 관절각도)
 - 4. 그리퍼 상태 추출(개폐 정도)
 - 5. 물체 위치 및 궤적 추적
 - 6. 역동역학(IDM) 모델 사용(토크/힘 계산)

GR00T-Dreams → GR00T N1.5 파이프라인(5 단계)

5단계 : GR00T N1.5 학습(Post Training)

- 학습 데이터: 현실 시연 + 드림즈 합성 데이터(액션 토큰)
- 입력 데이터 구성
 - 언어 명령
 - 비전 정보(RGB, Depth, Segmentation)
 - 로봇 상태 정보(관절 위치, 그리퍼 상태)
- 행동 생성 과정
 - Vision-Language 모듈(Eagle 2 기반): 상황 이해
 - Diffusion Transformer: 행동 시퀀스 생성(두뇌 역할)
 - Flow Matching Loss: 연속적인 움직임의 흐름 학습(디노이징 기법)
- 학습 전략
 - Pre-training: 범용 동작 패턴 학습(합성+시뮬레이션+시연 데이터)
 - Post-training: 특정 로봇/작업에 맞게 세밀 튜닝(고품질 액션 토큰 사용)

주요 성과 및 능력 향상

Sim-to-Real 문제 해결

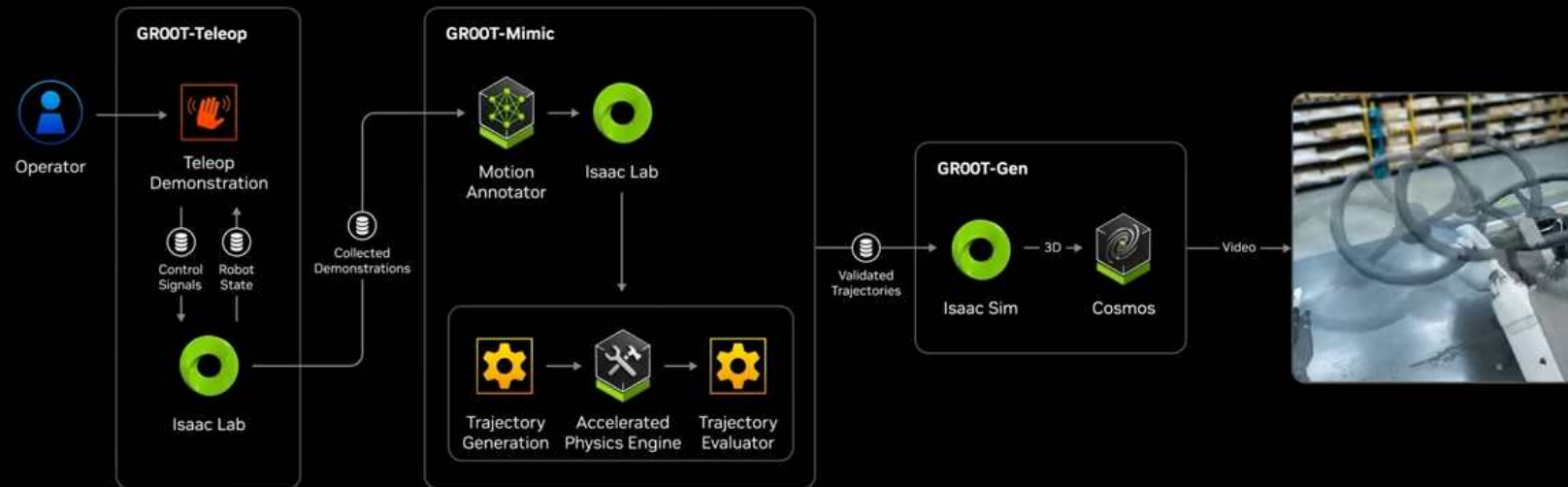
- 단순 시뮬레이터 반복 학습 초월
- 현실 친화적 행동 계획 데이터 생성(Chain-of-Thought 검증)
- 획기적인 학습 시간 단축

범용성(Generalization) 향상

- 다양한 로봇 형태/환경 반영 시나리오 대량 포함
- 싱글, 바이메뉴얼, 휴머노이드 통합 처리 가능
- 새로운 로봇에 미세 조정만으로 적용 가능

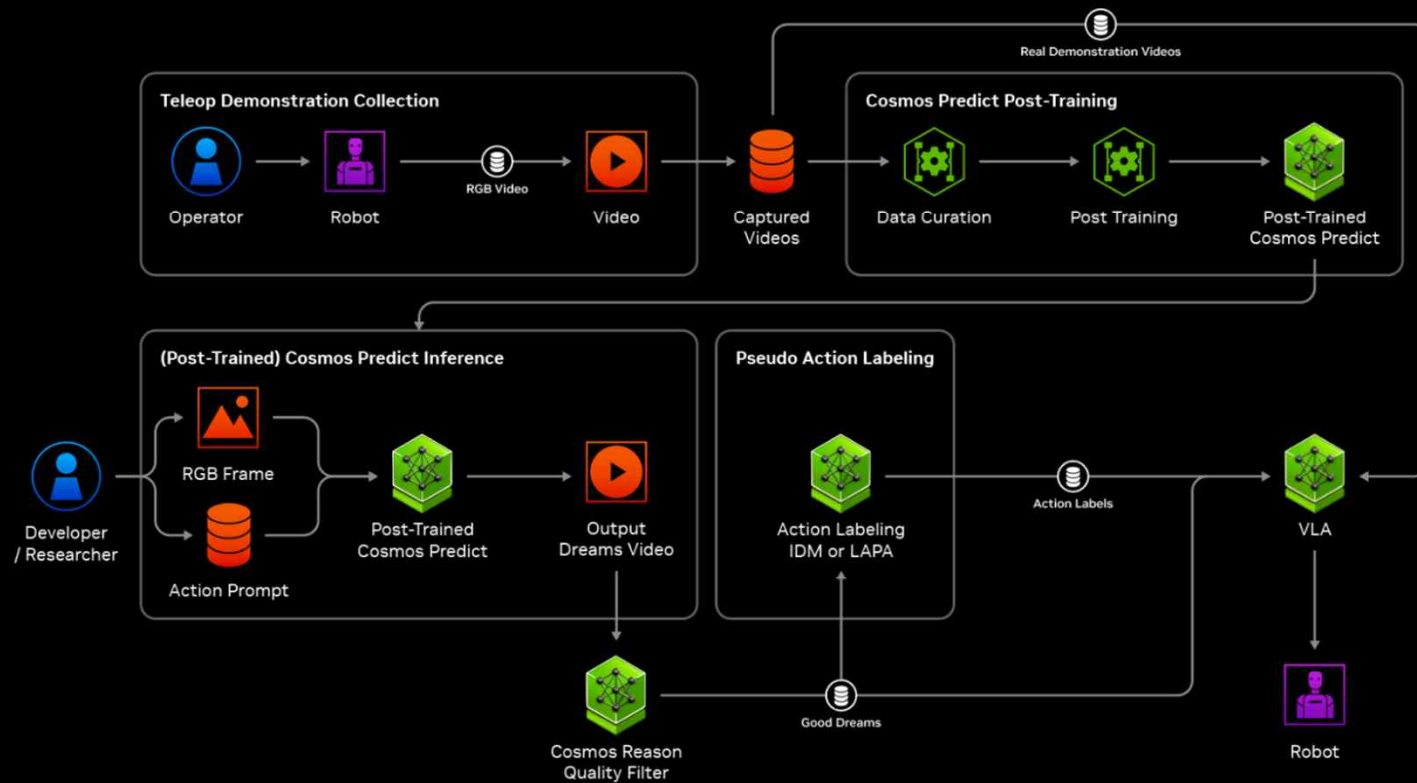
GR00TN1의 모델 구성과 학습 방식 : System 1, System 2 학습

GR00T-Dreams와 N1.5의 파이프라인 구조와 학습 방법



생각하시면 됩니다

GR00TN1의 모델 구성과 학습 방식 : System 1, System 2 학습



인트로

- 현실 세계의 복잡성과 불확실성 강조
- 로봇의 사고 능력을 위한 AI 활용 확대
- 데이터 부족이 핵심 문제
- 엔비디아 GROOT 시리즈가 그 해결책으로 등장

엔비디아 GROOT-DREAMS와 GROOT N1.5 소개

- GROOT의 목표: 로봇용 파운데이션 모델 구축
- GROOT-N1: 언어·이미지 이해 기반의 행동 생성 모델
- GROOT-Dreams: AI 비디오 생성으로 학습 데이터를 스스로 확장
- GROOT-N1.5: 다양한 상황에 적응 가능한 고도화된 로봇 모델
- 의미: AI가 스스로 상상하고 학습하는 자율 로봇 시대의 도래

GR00T-Dreams와 N1.5의 파이프라인 구조와 학습 방법 - 1

1단계 : 현실 데이터 수집(현실 기반 시연)

- 목적: 로봇 학습의 출발점인 현실 시연 데이터 확보
- 방법: 인간 조종을 통한 88시간 분량의 실제 영상 수집
- 비유: 부모를 관찰하며 배우는 아이의 초기 학습
- 한계: 현실 데이터만으로는 다양한 변수를 모두 학습하기 어려움

2단계 : Dreams의 합성 데이터 생성(AI 기반 상상 단계))

- Dreams의 역할: 현실 데이터를 기반으로 합성 데이터 확장
- 데이터 증폭 규모: 88시간 → 827시간 (약 10배 증가)
- 생성 방식: 색상·위치·환경·각도 등을 다양하게 변형
- 의미: 로봇이 상상력을 통해 학습 범위를 확장하는 단계

3단계: Chain-of-Thought 기반 평가·필터링 (Cosmos Reason)

- 문제의식: Dreams가 생성한 모든 합성 영상이 유의미하지는 않음(물리 불가능·목표 미달 등 포함).
- 해결책: 엔비디아는 **Cosmos Reason**이라는 **Chain-of-Thought(CoT)** 기반 평가 시스템으로 영상을 단계적으로 심사.
- 평가 절차(예시 포함)
 1. 목표 달성 여부: “로봇이 컵을 선반에 올려놨는가?” → 컵 이동·그리퍼에서 놓임·선반 안착을 순차 확인
 2. 물리적 타당성: 관통 여부, 갑작스런 공중부양 등 비물리적 프레임 검출
 3. 학습 가치/다양성: 기존 데이터와의 차별성, 새로운 각도·배경, 시나리오 다양성 평가
- 의의: 단순 좋다/나쁘다 이분법이 아니라, 왜 좋은지/나쁜지 근거를 설명하며 고품질 데이터만 정교하게 선별 가능.

GR00T-Dreams와 N1.5의 파이프라인 구조와 학습 방법 - 2

4단계: 액션 토큰 추출 (행동 정보화 단계)

Dreams의 검증된 비디오 데이터는 이제 로봇이 실제로 이해하고 실행할 수 있는 형태로 변환

이 과정을 통해 영상이 단순한 관찰 자료가 아닌 **로봇의 행동 명령 시퀀스(Action Token Sequence)**로 바뀜

① 2D 포즈 추정

각 프레임에서 로봇의 **관절(손목, 팔꿈치, 어깨 등)** 위치를 찾아내어 **2D 좌표 정보** 얻음. (OpenPose, HRNet 등과 유사한 방식)

② 3D 복원

Dreams가 생성한 합성 비디오에는 **Depth(깊이)** 정보가 포함되어 있어, 2D 키포인트를 **(x, y, z)** 형태의 3D 좌표로 복원

③ 역기구학(Inverse Kinematics, IK) 계산

얻은 3D 손끝 좌표를 기반으로 **각 관절을 몇 도로 움직여야 하는지** 계산

FK(Forward Kinematics): 각도 → 위치

IK(Inverse Kinematics): 위치 → 각도

④ 그리퍼(Gripper) 상태 추출

세그멘테이션 분석을 통해 **그리퍼 개폐 정도**를 파악하고, 이를 **0~100%** 또는 **양자화된 수치**로 정규화

⑤ 물체 궤적 추적

세그멘테이션으로 **물체의 위치와 이동 경로**를 추적하고, 로봇 손끝과의 **상대 위치**를 계산하여 동작의 정확성 평가

⑥ 역동역학(Inverse Dynamics Model, IDM) 적용

이전 단계에서 얻은 **관절 각도 시퀀스**를 기반으로,

실제 로봇이 움직일 수 있도록 **힘·토크·관성** 등을 고려한 **물리적 모터 명령** 생성

⑦ 행동 시퀀스의 토큰화

모든 정보를 **시간 순서로 시계열화**한 뒤, 연속적인 수치를 **256단계로 양자화(discretization)**하여 토큰으로 변환

예: x축 이동: 0~255, y축 이동: 0~255, 회전 각도: 0~255

결과적으로 **"128 91 241 5 101 127"**과 같은 **이산적 행동 토큰 시퀀스** 생성됨

GR00T-Dreams와 N1.5의 파이프라인 구조와 학습 방법 -3

최종 산출물

Dreams의 한 영상은 다음 세 요소로 구성된 완전한 학습 데이터셋이 됩니다.

1.언어 토큰 (명령어: “빨간 컵을 선반 위에 올려놔”)

2.비전 토큰 (환경의 이미지 정보)

3.액션 토큰 시퀀스 (실제 동작 명령)

이를 학습한 **GR00T-N1.5**는 사용자의 자연어 명령을 해석해, 카메라로 환경을 파악하고 **해당 액션 토큰 시퀀스를 예측·실행**하여 실제로 로봇이 행동을 수행할 수 있습니다.

핵심 요약 포인트

- Dreams의 비디오 → 로봇이 실행 가능한 행동 명령으로 변환
- 단계별 변환: 2D → 3D → IK → IDM → Tokenization
- 결과: 언어·비전·액션이 결합된 완전한 학습 데이터셋
- 의미: 로봇이 언어 이해에서 실제 물리 행동까지 수행할 수 있는 **End-to-End 학습 구조** 확립

GR00T-Dreams와 N1.5의 파이프라인 구조와 학습 방법

5단계: GR00T-N1.5 Post-Training (로봇 두뇌 학습 단계)

이 단계는 GR00T 프로젝트의 **핵심 학습 과정**으로, Dreams가 생성한 합성 데이터와 현실 시연 데이터를 함께 사용해 **GR00T-N1.5를 훈련**하는 단계입니다. GR00T-N1.5는 **Vision-Language-Action(VLA)** 모델로, 사람의 “보고-듣고-행동하는” 사고 과정을 모사하도록 설계되어 있습니다.

① 입력 데이터 구성

모델은 세 가지 입력을 동시에 받아들입니다.

언어 명령: 예) “빨간 컵을 선반 위에 올려놔.”

비전 정보: RGB, Depth, Segmentation을 포함한 카메라 영상

로봇 상태 정보: 관절 각도, 그리퍼 상태 등 Proprioceptive 정보

이 세 가지 정보를 결합하여 로봇이 현재 상황을 인식하고 다음 행동을 계획할 수 있도록 합니다.

② Vision-Language 모듈 (Eagle-2 기반)

엔비디아의 **Eagle-2** 모델이 이미지와 언어를 함께 처리하여 의미를 **토큰화된 벡터로 변환**합니다.

이미지 속 객체(예: 빨간 컵)의 위치 파악

언어 명령 속 목표(예: 선반 위에 올려놔) 이해

로봇의 현재 상태와 결합하여 **상황적 의미 표현(Semantic Embedding)** 생성

이 단계는 인간이 “본 것과 들은 지시를 통합적으로 이해하는 과정”에 해당합니다.

GR00T-Dreams와 N1.5의 파이프라인 구조와 학습 방법

5단계: GR00T-N1.5 Post-Training (로봇 두뇌 학습 단계)

③ Diffusion Transformer를 통한 행동 생성

GR00T-N1.5의 **핵심 두뇌**로, 비전·언어 해석 결과를 받아 연속된 ****행동 시퀀스(모터 명령)****를 생성합니다.
단일 명령이 아니라 시간에 따른 일련의 행동(그리퍼 열기 → 이동 → 잡기 → 올리기 → 놓기)을 계획

Flow Matching Loss를 활용해 움직임의 “연속성과 자연스러움”을 학습

단순히 목표 위치에 도달하는 것이 아니라 **“이동 경로 전체가 매끄럽고 물리적으로 자연스럽도록”** 학습

학습 시 일부러 노이즈를 섞은 행동 시퀀스를 모델이 복원하도록 훈련

결과적으로 **디퓨전 기반의 디노이징 학습**을 통해 깨끗한 액션 시퀀스를 생성

④ 다양한 로봇 형태(Embodiment) 학습

로봇마다 구조가 다르므로 GR00T-N1.5는 Dreams의 합성 데이터를 활용해

싱글암·양팔·휴머노이드 등 다양한 형태의 로봇에 맞게

상태·액션 공간을 인코딩/디코딩할 수 있도록 학습합니다.

→ 즉, **로봇 형태에 무관한 범용 행동 지능**을 달성합니다.

GR00T-Dreams와 N1.5의 파이프라인 구조와 학습 방법

5단계: GR00T-N1.5 Post-Training (로봇 두뇌 학습 단계)

⑤ Pre-training & Post-training 전략

Pre-training 단계:

Dreams의 합성 데이터, 사람 시연, 시뮬레이터 데이터를 모두 활용해 범용 동작 패턴과 언어-비전 매핑을 학습.

Post-training 단계:

특정 로봇·작업에 맞게 Dreams의 고품질 액션 토큰으로 세밀 조정.

실제 로봇이 물리적으로 수행 가능한 모터 명령으로 변환.

이 2단계 학습 전략 덕분에 GR00T-N1.5는 새로운 환경·로봇·작업에 빠르고 효율적으로 적응할 수 있습니다.

핵심 정리

GR00T-N1.5는 Vision-Language-Action 통합 구조로 동작

Flow Matching Loss + Diffusion Transformer로 자연스러운 시퀀스 학습

Eagle-2 기반 언어·비전 통합 이해

다양한 로봇 Embodiment 지원

Pre/Post-Training 전략으로 범용성과 실제 적응력 확보

GR00T-Dreams와 N1.5를 이용한 Sim-to-Real과 Generalization

엔비디아의 **GR00T-Dreams**와 **GR00T-N1.5** 파이프라인은 로봇 학습의 두 가지 난제인 **Sim-to-Real 문제**와 **Generalization(범용성)** 문제를 혁신적으로 해결했습니다.

① Sim-to-Real 문제 해결 — 현실 친화적 합성 데이터

로봇이 시뮬레이션에서 학습한 능력을 **현실에서도 동일하게 발휘**하는 것은 오랜 난제였습니다. 시뮬레이터는 현실의 미세한 마찰, 조명, 노이즈 등을 모두 재현할 수 없기 때문입니다.

GR00T-Dreams의 접근 방식:

단순히 물리 시뮬레이터에 의존하지 않고, **실제 인간의 시연 영상**을 기반으로 학습.
이를 토대로 **AI 비디오 생성 모델**이 **현실 물리 규칙**을 반영한 다양한 시나리오를 상상·생성.
Chain-of-Thought 기반 Cosmos Reason 평가 시스템을 통해 물리적으로 타당한 고품질 데이터만 선별.

결과적으로 Dreams가 만들어낸 합성 데이터는 **현실에서 바로 사용 가능한 행동 계획 데이터**가 되었으며, GR00T-N1.5는 시뮬레이션에서 학습한 전략을 **실제 로봇 하드웨어에서도 자연스럽게 적용**할 수 있게 되었습니다. 또한, 사람이 수개월에 걸쳐 수집해야 했던 데이터를 Dreams를 통해 단 며칠 만에 자동 생성·학습할 수 있어 **학습 효율이 획기적으로 향상**되었습니다.

GR00T-Dreams와 N1.5를 이용한 Sim-to-Real과 Generalization

② Generalization 문제 해결 — 범용 로봇 지능의 실현

기존 로봇 학습 모델들은 **특정 로봇, 특정 환경, 특정 작업**에 맞게 설계되어 범용성이 낮았습니다.
환경이나 하드웨어가 조금만 달라져도 처음부터 다시 학습해야 했습니다.

GR00T-Dreams와 N1.5의 접근 방식:

Dreams는 단일 작업의 변형뿐 아니라, **다양한 로봇 형태·환경·조명·물체 특성**이 포함된 **대규모 시나리오 데이터**를 생성.
GR00T-N1.5는 이러한 데이터로 학습하여 **로봇 형태에 무관한 공통 행동 정책**을 습득.
예: 싱글암, 바이매뉴얼, 휴머노이드 등 다양한 구조의 로봇에서 동일 모델 사용 가능.
결과적으로 GR00T-N1.5는 **특정 플랫폼에 종속되지 않는 범용 로봇 지능**을 구현.
이 범용성이 고도화되면, 새로운 로봇이 등장할 때마다 처음부터 재학습할 필요 없이
기존 학습된 모델을 미세 조정(Fine-tuning)만으로 적용할 수 있게 됩니다.

핵심 요약 포인트

Sim-to-Real: 현실 기반 시연 + Dreams 합성 데이터 + CoT 평가 → 현실 적용성 극대화

Generalization: 다양한 로봇 형태·환경에 대응 가능한 범용 행동 정책 학습

결과: 빠른 학습, 현실 적응성 강화, 새로운 로봇에도 즉시 확장 가능한 AI 로봇 지능 확보

즉, GR00T-Dreams와 GR00T-N1.5의 결합은 “**시뮬레이션에서 배운 로봇이 현실에서 유연하게 행동하는 시대**”를 열었으며,
“**모든 로봇이 공유할 수 있는 범용 두뇌**”의 기반을 마련했습니다.

마무리

- GR00T-Dreams & N1.5로 로봇의 자율 학습 시대 개막
- AI가 상상을 통해 현실 학습의 한계를 돌파